



UNIVERSIDAD DE LOS LAGOS

SISTEMA DE MOVIMIENTO ROBÓTICO CONTROLADO POR SEÑALES ELECTROMIOGRÁFICAS

DEPARTAMENTO DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA CIVIL EN INFORMÁTICA
PROYECTO DE TÍTULO
CAMPUS OSORNO, CHILE

David Pier Exequiel Raimilla Comicheo
Autor

Docente Guía: Juan Carlos Gallardo Casas
Docente Guía: Guía

10 de julio de 2026



5 años
UNIVERSIDAD ACREDITADA
Septiembre de 2021 - Septiembre de 2026
Gestión Institucional - Docencia de Pregrado
Investigación - Vinculación con el Medio
AVANZADA

www.ulagos.cl

Agradecimientos

A mis padres.

Resumen

Los sistemas de control robótico basados en señales electromiográficas superficiales (sEMG) disponibles comercialmente requieren hardware especializado e inversiones superiores a 350 USD, lo que restringe significativamente su accesibilidad en contextos de investigación y prototipado educativo. Resolver esta problemática es fundamental para democratizar el estudio de las interfaces humano-máquina y fomentar el desarrollo de tecnologías asistenciales replicables mediante el uso de plataformas experimentales de bajo costo. Este trabajo aborda dicha necesidad a través del diseño e implementación de una solución de arquitectura híbrida orientada a la estimación cinemática continua de un actuador robótico. El principal logro alcanzado consiste en el desarrollo de un *pipeline* de regresión cinemática basado en *Random Forest* que realiza, en tiempo real, la estimación continua de la posición angular del actuador a partir de los patrones de activación de tres canales musculares: bíceps braquial, tríceps braquial y pronador redondo. Para optimizar el uso de recursos, el sistema emplea un esquema de procesamiento descentralizado (*offload*) donde un microcontrolador ATmega328P actúa como puente de adquisición de datos a 1.000 Hz, delegando el filtrado digital pesado y la inferencia del regresor a una computadora externa. El mayor desafío técnico radicó en mitigar la interferencia electromagnética y gestionar las severas restricciones de memoria del hardware embebido, lo cual se superó mediante el codiseño de filtros digitales Butterworth, normalización dinámica por contracción voluntaria máxima (%MVC) y un limitador de *slew-rate* acoplado a la dinámica física real de los servomotores. Con un costo total de hardware de 330.608 CLP (aproximadamente 340 USD) y una latencia extremo a extremo promedio de 134,3 ms, el sistema garantiza un movimiento graduado y estable, demostrando la viabilidad de construir interfaces mioeléctricas de alta fidelidad en entornos con recursos limitados.

Keywords: sEMG, Interfaz Humano-Máquina, Random Forest, Sistemas de Bajo Costo, Arquitectura Híbrida.

Índice general

1. Formulación del Proyecto	1
1.1. Propósito	1
1.2. Identificación del Problema	2
1.2.1. Antecedentes del Problema	2
1.2.2. Definición del Problema	3
1.3. Justificación y Aporte	4
1.3.1. Utilidad del Sistema	5
1.3.2. Aporte del Sistema	6
1.4. Marco Teórico	6
1.4.1. Señales EMG Superficiales	6
1.4.2. Adquisición y Acondicionamiento Analógico	8
1.4.3. Procesamiento Digital de Señales EMG	10
1.4.4. Estrategia de Control por Estimación de Patrones	12
1.4.5. Métricas de Evaluación del Sistema	13
1.5. Estado del Arte	15
1.5.1. Técnicas de control proporcional basadas en amplitud sEMG	15
1.5.2. Implementaciones de bajo costo sobre microcontroladores embebidos	16
1.5.3. Comparativa de técnicas: control proporcional versus clasificación supervisada	16
1.5.4. Brecha identificada	18
2. Metodología de Trabajo	20
2.1. Alcance	20
2.2. Objetivos	20
2.2.1. Objetivo General	20
2.2.2. Objetivos Específicos	21
2.3. Desglose de Actividades y Tareas	21
2.3.1. Objetivo Específico 1: Diseño del Sistema EMG	21
2.3.2. Objetivo Específico 2: Procesamiento y Estimación de Patrones EMG	22
2.3.3. Objetivo Específico 3: Integración con el Brazo Robótico	22
2.3.4. Objetivo Específico 4: Evaluación del Sistema	23
2.4. Planificación Temporal	24

2.4.1. Carta Gantt	24
2.4.2. Carta Gantt General del Proyecto	25
2.4.3. Desglose de Actividades	26
2.5. Análisis de Viabilidad del Proyecto	31
2.5.1. Viabilidad Técnica	31
2.5.2. Viabilidad Operativa	33
2.5.3. Viabilidad de Plazos (Cronograma)	34
2.6. Metodología de Desarrollo	34
2.6.1. Justificación de la Metodología	35
2.6.2. Estructura de Desarrollo en Espirales	35
2.6.3. Ventajas del Modelo en Espiral	36
2.6.4. Gestión de Riesgos y Contingencias	37
3. Descripción General del Sistema	38
3.1. Perspectiva del Producto	38
3.2. Funciones del Sistema	39
3.3. Características de los Usuarios	40
3.4. Restricciones	41
3.5. Suposiciones y Dependencias	41
3.5.1. Suposiciones	41
3.5.2. Dependencias	42
4. Requisitos del Proyecto	43
4.1. Requisitos de Interfaces Externas	43
4.1.1. Interfaces de Usuario	43
4.1.2. Interfaces de Hardware	43
4.1.3. Interfaces de Software	44
4.1.4. Interfaces de Comunicación	44
4.2. Requisitos Funcionales	45
4.2.1. Adquisición de señal	45
4.2.2. Procesamiento digital de señal	45
4.2.3. Estimación cinemática y control continuo del actuador	46
4.2.4. Calibración	46
4.2.5. Monitoreo serial	47
4.3. Requisitos de Rendimiento	47
4.4. Requisitos de Desarrollo	48
4.5. Requisitos Tecnológicos	48
4.6. Seguridad	48
4.7. Persistencia y Estructura de Datos	49

5. Análisis Económico	51
5.1. Estudio de Mercado	51
5.1.1. Consumidores y análisis de la demanda	51
5.1.2. Análisis de la oferta y los precios	52
5.1.3. Comercialización	52
5.2. Evaluación de Proyectos de Inversión	52
5.2.1. Criterios de evaluación	52
5.2.2. Valor Actual Neto (VAN)	53
5.2.3. Tasa Interna de Retorno (TIR)	53
5.2.4. Análisis de riesgo e incertidumbre	53
5.3. Viabilidad Económica	54
5.3.1. Costos de Desarrollo	54
5.3.2. Costos Operativos (Post-Implementación)	55
5.3.3. Retorno de la Inversión (ROI) y Beneficios	56
6. Desarrollo	59
6.1. Arquitectura General del Sistema y Co-diseño Híbrido	59
6.1.1. Justificación Técnica del Offload Computacional	60
6.1.2. Descripción de las Capas Funcionales de la Arquitectura	61
6.2. Implementación del Hardware de Adquisición (Arduino Uno - C++)	62
6.2.1. Justificación de la Programación a Bajo Nivel sin Abstracciones de Arduino	62
6.2.2. Configuración de Registros del Temporizador Timer1 y ADC	63
6.2.3. Estructura del Firmware y Rutina de Servicio de Interrupción (ISR)	63
6.2.4. Mapeo Físico de Conexiones del Nodo de Adquisición	64
6.2.5. Análisis Temporal por Software y Perfilamiento de Cómputo Embebido	64
6.2.6. Propiedades	65
6.2.7. Ventajas y Desventajas	66
6.3. Propuesta de Solución: Arquitectura Distribuida de Lazo Cerrado	67
6.3.1. Fundamentación Teórica del Filtro Butterworth y Criterio de Nyquist	67
6.3.2. Ecuación en Diferencias del Filtro IIR	68
6.3.3. Implementación de la Clase en Python para Procesamiento en Tiempo Real	68
6.3.4. Filtro Notch para Rechazo de Interferencia de Red Eléctrica (50 Hz)	69
6.4. Extracción de Características Temporales e Inferencia con Random Forest	72
6.4.1. Definición y Modelamiento Matemático de las Características Temporales	72
6.4.2. Justificación Teórica del Algoritmo RandomForestRegressor Multi-salida	73
6.4.3. Optimización de Hiperparámetros y Latencia de Inferencia	74
6.4.4. Conjunto de Datos y Protocolo de Entrenamiento	75
6.4.5. Validación Cruzada y Selección de Hiperparámetros	76
6.4.6. Métricas de Desempeño en el Conjunto de Prueba	76
6.4.7. Análisis de Sobreajuste	77
6.4.8. Implementación de la Extracción de Características e Inferencia en Python	77
6.5. Integración de Lazo Cerrado y Actuación Física (Arduino Uno - C++)	82

6.5.1.	Protocolo de Comunicación UART de Bajada y Deserialización	83
6.5.2.	Algoritmo de Suavizado y Limitación de Slew-rate en Firmware	83
6.5.3.	Conversión de Ángulos a Cuentas Digitales de 12 bits	84
6.5.4.	Listado de Código en C++ para Gestión de Actuación	85
6.6.	Rutina de Calibración Dinámica y Normalización Basada en %MVC	85
6.6.1.	Ecuación de Normalización y Saturación Estricta	86
6.6.2.	Protocolo de Calibración Anti-Fatiga	86
6.6.3.	Programación Defensiva contra Calibraciones Inválidas o Ruidosas	87
6.7.	Pruebas Parciales de Lazo Abierto y Registro de Problemas de Integración	88
6.7.1.	Problema 1: Micro-oscilaciones Mecánicas (Jitter) por Ruido Estocástico sEMG	88
6.7.2.	Problema 2: Desbordamiento de Búfer UART por Asincronía de Lazos . . .	89
6.7.3.	Síntesis del Estado de Desarrollo de la Solución y Transición a Pruebas .	91
7.	Aseguramiento de la Calidad	92
7.1.	Modelo de Calidad de Software	92
7.2.	Técnicas de Aseguramiento de Calidad	93
7.3.	Implementación por Espiral de Desarrollo	94
8.	Proceso de Prueba del Software y Validación	96
8.1.	Criterios y Enfoques de Prueba	96
8.1.1.	Enfoque de Prueba de Caja Negra	96
8.1.2.	Enfoque de Prueba de Caja Blanca	97
8.2.	Pruebas de Unidad (Unit Testing)	97
8.2.1.	Pruebas Unitarias de Firmware Embebido (Arduino Uno - C++)	97
8.2.2.	Pruebas Unitarias del Software del Host (PC - Python)	98
8.3.	Pruebas de Integración	99
8.4.	Pruebas de Sistema	100
8.4.1.	Prueba de Recuperación (Resiliencia Embebida)	100
8.4.2.	Prueba de Seguridad	101
8.4.3.	Prueba de Resistencia (Stress-Test)	101
8.4.4.	Pruebas de Sistema de Rendimiento	101
8.5.	Prueba de Aceptación y Usabilidad	103
9.	Conclusiones	105
9.1.	Principales Aportes del Proyecto	105
9.2.	Contraste de Resultados Frente a Objetivos	106
9.3.	Trabajos Futuros	107
A.	Definiciones, Acrónimos y Abreviaturas	114
A.1.	Definiciones	114
A.2.	Acrónimos	115
A.3.	Abreviaturas	115

B. Configuraciones	116
C. Firmware Completo del Sistema de Adquisición y Actuación	117

Índice de figuras

1.1. Electrodo de superficie gelificado para registro sEMG.	8
1.2. Configuración bipolar: dos electrodos activos sobre el músculo y un electrodo de referencia en zona neutra. Fuente: Adaptado de [1], basado en los fundamentos clásicos de Merletti y Parker [2].	9
2.1. Carta Gantt general del proyecto basada en Modelo en Espiral. ESP: Espiral; OE: Objetivo Específico. Las barras azules representan el desarrollo técnico de ingeniería, las verdes las etapas formales de evaluación y las naranjas agrupan el ciclo iterativo completo de cada espiral.	25
3.1. Diagrama de Bloques Físico y de Conexiones del Sistema	39
5.1. Grafo CPM del proyecto. Ruta crítica optimizada para un desarrollo secuencial sin holguras [3].	58
6.1. Arquitectura de procesamiento distribuido y lazo cerrado de control.	59
6.2. Esquema de la Arquitectura del Regresor RandomForest Dual con etapas acopladas de suavizado temporal y banda muerta.	74
6.3. Curva de aprendizaje del regresor <i>Random Forest</i> : evolución del error <i>MAE</i> (en grados) para las articulaciones de codo y muñeca en función del volumen de muestras de entrenamiento.	78
6.4. Un cuadrado vacío en TikZ	87
6.5. Contraste experimental entre la consigna angular bruta de la IA y la trayectoria suavizada tras el acoplamiento del filtro exponencial en Python y la rampa de <i>slew-rate</i> local en el microcontrolador [4].	89
8.1. Diagrama de Tiempos del Protocolo Loopback de Medición de Latencia.	103
8.2. Gráfico de Barras de Aceptabilidad Tecnológica SUS	104

Índice de tablas

1.1. Parámetros típicos de las señales EMG superficiales. Fuente: adaptado de [5].	7
1.2. Comparativa de trabajos relacionados según técnica de procesamiento, número de canales, latencia, precisión y costo reportados.	19
2.1. Criterios técnicos de aceptación y estrategias de mitigación por espiral	26
2.2. Resumen temporal de los Objetivos Específicos del proyecto.	27
2.3. Holguras temporales incorporadas en las actividades principales del proyecto.	30
2.4. Recursos asignados por fase del proyecto.	30
2.5. Costos de hardware e infraestructura (inversión inicial)	34
2.6. Resumen de las cinco espirales del proyecto, criterios de evaluación cuantitativos y estrategias de contingencia.	36
2.7. Contingencias principales y estrategias de mitigación asociadas.	37
5.1. Flujo de caja proyectado (CLP miles)	53
5.2. Análisis de escenarios (CLP miles)	54
5.3. Costo de personal de desarrollo	54
5.4. Costos de software	55
5.5. Costos de hardware e infraestructura del prototipo	55
5.6. Resumen de costos consolidados del proyecto de título	57
6.1. Matriz de mapeo físico de conexiones y asignación de canales sEMG.	64
6.2. Ventajas y desventajas de la implementación del firmware en C/C++ y su estrategia de mitigación mediante el modelo distribuido.	66
6.3. Coeficientes numéricos discretos del filtro Butterworth pasabanda	68
6.4. Estructura detallada del vector de características de 12 dimensiones	73
6.5. Parámetros y configuración de hiperparámetros del RandomForestRegressor	75
6.6. Composición del conjunto de datos utilizado para el entrenamiento del regresor <i>Random Forest</i>	76
6.7. Resultados de validación cruzada de 5 particiones sobre el conjunto de entrenamiento.	76
6.8. Métricas de desempeño del <i>RandomForestRegressor</i> sobre el conjunto de prueba, por articulación.	77

6.9. Comparación de error entre el conjunto de entrenamiento y el conjunto de prueba para evaluación de sobreajuste.	77
8.1. Matriz de casos de prueba de unidad de la arquitectura distribuida	99
8.2. Métricas cuantitativas de precisión de trayectoria y latencia de lazo cerrado por sujeto	102
8.3. Resultados cuantitativos detallados del cuestionario estandarizado SUS	103
9.1. Contraste cuantitativo entre objetivos específicos y resultados del desarrollo . . .	107

Índice de algoritmos

Capítulo 1

Formulación del Proyecto

Se expone el propósito general, se delimita el problema que motiva la investigación y se describen los fundamentos teóricos que sustentan el diseño del sistema: el procesamiento digital de señales electromiográficas superficiales, la extracción de características temporales y la estimación de posición mediante un modelo *Random Forest* en modo regresión, el control de actuadores robóticos mediante modulación por ancho de pulso (PWM) y los protocolos de comunicación empleados entre los componentes de hardware.

1.1. Propósito

Las interfaces de control humano-máquina basadas en señales electromiográficas superficiales (sEMG) permiten traducir la actividad muscular del usuario en comandos de control para dispositivos robóticos. Sin embargo, diversas implementaciones reportadas en la literatura dependen de hardware especializado, configuraciones de adquisición complejas o procesamiento computacional elevado, lo que incrementa su costo y limita su aplicabilidad en contextos de prototipado educativo e investigación aplicada [6].

En este contexto, el presente proyecto tiene como propósito diseñar e implementar un sistema experimental de bajo costo para el control de un brazo robótico mediante la estimación continua de posición angular a partir de señales sEMG multicanal. A diferencia de propuestas recientes que integran la extracción de características y el procesamiento ligero directamente en hardware dedicado con el fin de reducir la complejidad computacional [6], esta investigación adopta una arquitectura distribuida que separa la adquisición del procesamiento intensivo. Esta decisión de diseño busca superar las restricciones de cómputo de los microcontroladores de bajo costo, permitiendo la integración de un modelo de regresión multisalida (Random Forest) para la estimación continua de posición angular, manteniendo una latencia extremo a extremo inferior a 250 ms.

1.2. Identificación del Problema

El problema radica en la inexistencia de una arquitectura de software reproducible que permita ejecutar el pipeline completo de estimación cinemática continua a partir de señales sEMG directamente sobre hardware de recursos limitados. Actualmente, el desarrollo de interfaces humano-máquina de bajo costo enfrenta una encrucijada técnica: las soluciones documentadas se ven obligadas a sacrificar la fluidez del movimiento mediante un control binario de umbral fijo, o bien dependen de unidades de procesamiento externas de alto rendimiento que destruyen la portabilidad y elevan los costos. Esta brecha tecnológica impide que investigadores y estudiantes dispongan de una referencia técnica que logre una respuesta mecánica graduada y fiel a la fisiología muscular sin depender de hardware especializado [4].

Los métodos actualmente utilizados presentan limitaciones relevantes:

- Sensibilidad a variaciones fisiológicas del usuario, tales como sudoración, fatiga muscular o desplazamiento de electrodos entre sesiones.
- Contaminación de la señal por ruido eléctrico de red y artefactos de movimiento.
- Dependencia de algoritmos supervisados complejos que tradicionalmente requieren estaciones de trabajo industriales o GPU dedicadas, limitando su despliegue en sistemas embebidos autónomos de bajo costo [4].
- Inviabilidad de ejecutar modelos de Machine Learning para estimación de posición directamente en microcontroladores con recursos de memoria y cómputo severamente restringidos [7].

El problema específico abordado consiste en la ausencia de una plataforma experimental de bajo costo que permita adquirir, procesar y utilizar señales sEMG mediante un conjunto reducido de etapas de adquisición y transmisión embebida, con el procesamiento digital delegado a un módulo en Python, con el fin de activar movimientos robóticos continuos mediante estimación de posición angular, sin requerir equipamiento industrial especializado.

1.2.1. Antecedentes del Problema

Los sistemas de control robótico basados en señales EMG han experimentado una evolución en las últimas dos décadas [4]. Los primeros dispositivos de consumo, como el MYO Armband, comercializado en 2013 por Thalmic Labs con un precio de referencia aproximado de 200 USD, abrieron el camino en accesibilidad antes de ser descontinuados por el fabricante [8]. La literatura identifica, sin embargo, que los sistemas profesionales de laboratorio siguen representando una barrera económica que supera los miles de dólares, y que el umbral de accesibilidad para sistemas de bajo costo, situado por debajo de los 350 USD, proviene del costo de un dispositivo específico reportado en la literatura, más que de un consenso general del campo [3]. La literatura técnica identifica tres paradigmas fundamentales en la evolución de las estrategias de procesamiento para señales electromiográficas de superficie (sEMG):

1. **Modelos de control proporcional (basados en amplitud):** Representan la aproximación más temprana, caracterizada por un control directo basado en la envolvente de la señal mediante métricas como el valor absoluto medio (MAV) o la raíz media cuadrática (RMS) [4]. Aunque su implementación requiere hardware de bajo costo, presentan mayor sensibilidad al ruido y tiempos de respuesta relativamente altos [5].
2. **Arquitecturas de reconocimiento de patrones (clasificación supervisada):** Este enfoque incorporó algoritmos de aprendizaje de máquina como Máquinas de Soporte Vectorial (SVM) y *Random Forest* para la identificación de comandos discretos [9]. Estas arquitecturas suelen apoyarse en hardware multicanal y en etapas de extracción manual de características, además de requerir entrenamiento fuera de línea (*offline*) [9].
3. **Modelos de aprendizaje profundo (*Deep Learning*):** El paradigma más reciente se ha orientado al uso de redes neuronales, especialmente arquitecturas híbridas **CNN-LSTM**, diseñadas para capturar relaciones espaciales y temporales en las señales sEMG [10]. Estas aproximaciones suelen lograr mejores resultados de clasificación que los métodos clásicos, aunque con mayores requerimientos de entrenamiento y cómputo [10].

El problema identificado surge de la discontinuidad entre estos sistemas de alto rendimiento y las necesidades de contextos educativos o proyectos de investigación con presupuesto limitado. La revisión de la literatura de trabajos publicados entre 2019 y 2024 revela una brecha crítica: solo una fracción de las implementaciones reportadas utiliza hardware con un costo inferior a 350 USD [3, 7]. Dentro de este subconjunto de bajo costo, se observa una falta de rigor metodológico, ya que la mayoría de los estudios no documentan protocolos completos de captura, procesamiento y evaluación, lo que limita su reproducibilidad en plataformas experimentales funcionales [3, 6, 7].

1.2.2. Definición del Problema

El problema abordado consiste en la ausencia de una plataforma experimental de bajo costo que integre adquisición, procesamiento y estimación de posición angular continua a partir de señales sEMG multicanal, en una arquitectura distribuida donde el microcontrolador actúa como puente de adquisición y transmisión serie, mientras que el procesamiento digital y la estimación se ejecutan en un host externo.

Este problema se manifiesta en las siguientes dimensiones:

1. **Barrera económica:** Los sistemas de electrodos integrados de consumo (como el *MYO Armband*, con un precio de referencia aproximado de 200 USD y descontinuado por el fabricante [8]) y los sistemas profesionales de laboratorio (Delsys Trigno: 15.000–30.000 USD; Ottobock: superior a 5.000 USD para prótesis activas) representan una barrera de costo significativa en contextos educativos. Como alternativa de bajo costo, Garouche y Thamsuwan [3] proponen un sistema portátil basado en un microcontrolador Raspberry Pico y un módulo Myoware, cuyo costo total de fabricación (incluyendo componentes, carcasa y electrodos) es de aproximadamente 350 USD.

2. **Complejidad computacional de las soluciones de alto rendimiento:** Los enfoques supervisados de mayor desempeño reportado en la literatura requieren capacidad computacional elevada y sesiones de entrenamiento con datos etiquetados [4, 10], lo que dificulta su implementación directa sobre microcontroladores de bajo costo con recursos de memoria y procesamiento restringidos.
3. **Ausencia de protocolos estandarizados para plataformas de bajo costo:** La literatura revisada no documenta un protocolo completo y reproducible que integre selección de sensores, preprocesamiento, extracción de características temporales y estimación de posición angular distribuida sobre hardware de bajo costo, ni mecanismos claramente estandarizados para mitigar variabilidad inter-sujeto, fatiga muscular y contaminación por artefactos en estas condiciones de operación [3, 7].

Estas limitaciones impiden validar experimentalmente una arquitectura de bajo costo que sostenga control continuo y estable de actuadores robóticos a partir de señales sEMG, sin recurrir a equipamiento industrial especializado.

En función de lo anterior, la pregunta de investigación que orienta el desarrollo del proyecto es:

¿Es técnicamente factible implementar un sistema para la estimación de posición angular continua basado en señales sEMG multicanal, utilizando adquisición en un microcontrolador ATmega328P como puente serie y un modelo *Random Forest* en modo regresión ejecutado en Python, con un costo de hardware inferior a 350 USD, que controle dos servomotores con una latencia extremo a extremo inferior a 200 ms bajo condiciones experimentales controladas?

1.3. Justificación y Aporte

El presente proyecto se justifica por la necesidad de disponer de una plataforma experimental de bajo costo que permita adquirir, procesar y utilizar señales sEMG multicanal para estimar de manera continua la posición angular de un actuador robótico. Esta necesidad surge porque diversas soluciones existentes en control mioeléctrico dependen de hardware especializado, altos costos de implementación o arquitecturas de procesamiento poco accesibles para contextos académicos y de prototipado experimental [6].

En este sentido, el sistema propuesto integra adquisición embebida, procesamiento digital en una PC host y control robótico continuo mediante un modelo *Random Forest* en modo regresión. La propuesta permite estudiar el comportamiento de una interfaz humano-máquina completa bajo restricciones reales de hardware, sin requerir infraestructura industrial ni dispositivos de alto costo.

La utilidad del proyecto se orienta a la formación académica, la investigación aplicada y el prototipado de interfaces mioeléctricas. La plataforma permite experimentar con el ciclo completo de una solución sEMG, desde la lectura de la señal biológica hasta la activación mecánica

del sistema robótico, incorporando además criterios de evaluación técnica como latencia, estabilidad y calidad de señal.

1.3.1. Utilidad del Sistema

El sistema aporta utilidad en tres dimensiones principales:

Formación académica y desarrollo de competencias técnicas

- Proporciona una plataforma tangible para trabajar conceptos de adquisición de señales biomédicas, procesamiento digital de señales y control de actuadores en tiempo real.
- Permite experimentar con la cadena completa de una interfaz humano-máquina, desde la señal fisiológica hasta la acción mecánica, sobre hardware de propósito general.
- Expone restricciones de diseño propias de plataformas de bajo costo, como la resolución limitada del ADC, la sensibilidad al ruido electromagnético y las limitaciones de memoria del microcontrolador.

Investigación aplicada y evaluación experimental

- Ofrece una base reproducible para comparar técnicas de filtrado, extracción de características y normalización de señales sEMG.
- Genera datos empíricos sobre desempeño del sistema mediante métricas cuantificables, como latencia, estabilidad angular y consistencia entre sesiones.
- Permite analizar desafíos prácticos asociados a la variabilidad intersujeto, la colocación de electrodos y la calidad de la señal en condiciones de bajo presupuesto [11].

Prototipado de interfaces humano-máquina

- Proporciona una arquitectura modular que separa adquisición, procesamiento, calibración y control, facilitando la modificación de etapas específicas sin alterar el sistema completo.
- Integra componentes comercialmente disponibles en una solución de costo accesible.
- Establece una base para futuras extensiones hacia un mayor número de grados de libertad o nuevas estrategias de control robótico.

1.3.2. Aporte del Sistema

El aporte principal del proyecto consiste en desarrollar una arquitectura distribuida para control robótico mioeléctrico, donde el microcontrolador realiza exclusivamente la adquisición y transmisión de datos, mientras que el procesamiento pesado y la inferencia del modelo se ejecutan en la PC host. Esta decisión permite superar las restricciones de memoria y capacidad de cómputo propias del hardware embebido de bajo costo, manteniendo una ejecución continua y funcional del sistema.

Desde el punto de vista técnico, el sistema integra en una sola solución: adquisición multicanal, preprocesamiento digital, extracción de características temporales, normalización por contracción voluntaria máxima y estimación continua de posición angular mediante regresión. Esta integración documenta un flujo de trabajo completo y reproducible para aplicaciones de interfaz humano-máquina basadas en señales sEMG.

Desde el punto de vista práctico, el proyecto ofrece una alternativa para contextos universitarios y de investigación aplicada, al emplear hardware comercial disponible y un costo total inferior al umbral de bajo costo considerado en la literatura [3]. Además, el sistema incorpora protocolos de calibración por sesión, orientados a mitigar variaciones fisiológicas e instrumentales asociadas a la variabilidad inter-sujeto [11].

1.4. Marco Teórico

Esta sección expone los fundamentos conceptuales necesarios para comprender las decisiones de diseño del sistema. Cada subsección se vincula directamente con una etapa del *pipeline* implementado o con un criterio de evaluación aplicado en el proyecto.

1.4.1. Señales EMG Superficiales

Origen fisiológico y propiedades de la señal

La electromiografía superficial (sEMG) registra la actividad eléctrica generada por músculos esqueléticos durante la contracción, mediante electrodos ubicados sobre la piel. La contracción muscular se origina cuando el sistema nervioso central envía señales eléctricas desde la médula espinal a las fibras musculares a través de motoneuronas. Este fenómeno produce **potenciales de acción** que se propagan a lo largo de las membranas de las fibras, con una forma bifásica o trifásica característica, una duración de 3 a 15 ms y una amplitud de entre 50 μ V y 5 mV [5].

La señal registrada en superficie es la suma temporal y espacial de los potenciales de acción de múltiples unidades motoras activadas de forma asincrónica. El resultado es una señal de naturaleza estocástica cuyas propiedades estadísticas varían con el tiempo (no estacionaria) debido a la fatiga muscular y al reclutamiento progresivo de unidades motoras. Las características principales de esta señal se resumen en la Tabla 1.1.

Dos propiedades de la señal son directamente relevantes para el diseño del sistema:

Parámetro	Valor típico
Amplitud pico a pico	0–10 mV
Rango de frecuencia total	10–500 Hz
Frecuencias dominantes	50–150 Hz
Duración potencial de acción	3–15 ms
Tasa de disparo unidades motoras	5–30 Hz

Tabla 1.1: Parámetros típicos de las señales EMG superficiales. Fuente: adaptado de [5].

- **Relación amplitud-fuerza:** La amplitud RMS de la señal sEMG se correlaciona de forma aproximadamente monótona con el nivel de fuerza de contracción, lo que hace viable su uso como variable de control proporcional [5].
- **Dependencia de la colocación del electrodo:** La morfología de la señal varía según la posición del electrodo respecto a las zonas de inervación y los tendones, lo que introduce variabilidad entre sesiones y justifica la fase de calibración por sesión del sistema implementado.

Criterio de Nyquist y selección de la frecuencia de corte superior

El teorema de muestreo de Nyquist-Shannon establece que la frecuencia de muestreo F_s debe ser al menos el doble de la frecuencia máxima de la señal $f_{\text{máx}}$ para evitar el fenómeno de solapamiento o *aliasing* [5].

En la arquitectura distribuida del sistema, la frecuencia de muestreo total de 1.000 Hz se distribuye alternadamente entre tres canales analógicos, resultando en una frecuencia efectiva por canal de aproximadamente 333 Hz [5].

Debido a esta restricción física del hardware, la frecuencia de corte superior del filtro digital se ajusta a 150 Hz para garantizar un margen de seguridad por debajo del límite de Nyquist efectivo de 166,5 Hz [5].

Aunque el contenido espectral de la señal sEMG puede alcanzar los 500 Hz, la mayor parte de la energía útil para el control se concentra en frecuencias inferiores a los 200 Hz [5].

La selección de 150 Hz como frecuencia de corte superior del filtro pasabanda deja un margen de 50 Hz respecto a f_N , dentro del cual la atenuación del filtro de orden 4 opera antes de la frecuencia de Nyquist, reduciendo la energía espectral susceptible de producir aliasing en la banda 150–200 Hz.

sEMG como fuente de control para interfaces humano-máquina

Las interfaces humano-máquina (HMI) basadas en sEMG transforman la actividad eléctrica muscular en instrucciones ejecutables por dispositivos robóticos, estableciendo un canal de comunicación entre la intención motora del usuario y la acción mecánica resultante. La sEMG

es la modalidad preferida para aplicaciones de control en tiempo real por su carácter no invasivo, facilidad de instrumentación y capacidad para capturar patrones musculares globales en el tiempo [12].

La literatura reciente distingue dos enfoques para la traducción de la señal sEMG en comando robótico: el control proporcional continuo basado en amplitud, y la clasificación supervisada de patrones musculares. Si bien el control proporcional exige menos recursos, es altamente susceptible al ruido residual. Por otro lado, un modelo *Random Forest* en modo regresión ofrece robustez superior al control proporcional por umbral, al capturar relaciones no lineales entre los patrones de activación muscular multicanal y la posición angular objetivo. Al implementar una arquitectura distribuida donde el modelo corre en una PC externa, se logra adoptar esta robustez sin incurrir en los costos de hardware que demandan las redes profundas ni en las restricciones computacionales de los procesadores embebidos de bajo costo [4].

1.4.2. Adquisición y Acondicionamiento Analógico

Electrodos de superficie gelificados

La adquisición de señales sEMG para aplicaciones de control en tiempo real se realiza mediante electrodos de superficie. En el presente proyecto se utilizan electrodos gelificados, cuya composición incluye un gel conductor de agua, cloruro de sodio y polímeros estabilizantes. Este gel reduce la impedancia de contacto respecto a electrodos secos, con valores típicos de 5 a 20 k Ω , lo que mejora la transmisión de la señal eléctrica desde la piel hacia el electrodo [5].

La elección de electrodos de superficie por sobre electrodos intramusculares responde a criterios de no invasividad, facilidad de colocación y compatibilidad con entornos no clínicos.

Para el presente sistema, cuyo objetivo es la estimación de posición angular continua mediante un modelo *Random Forest* en modo regresión a partir de las características de la señal sEMG, la selectividad espacial de los electrodos de aguja no aporta ventajas que justifiquen su uso fuera de un contexto clínico [5].

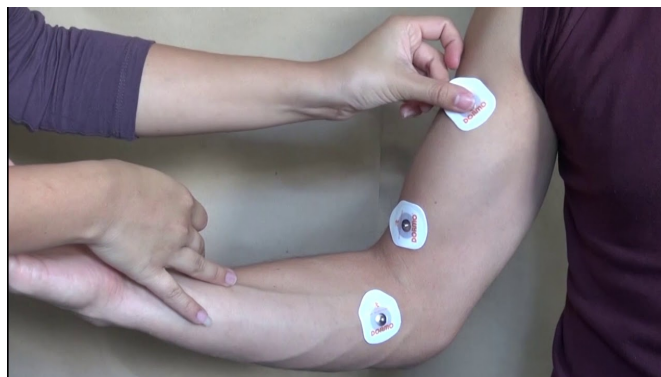


Figura 1.1: Electrodo de superficie gelificado para registro sEMG.

Configuración bipolar y rechazo de ruido de modo común

La configuración bipolar es el estándar para sEMG en aplicaciones de control. Utiliza dos electrodos activos posicionados sobre el mismo músculo, separados entre 10 y 20 mm, y un electrodo de referencia en zona neutra. Esta disposición mide la diferencia de potencial entre ambos electrodos activos, implementando efectivamente un amplificador diferencial que rechaza el ruido de modo común presente en ambos puntos de medición [5].

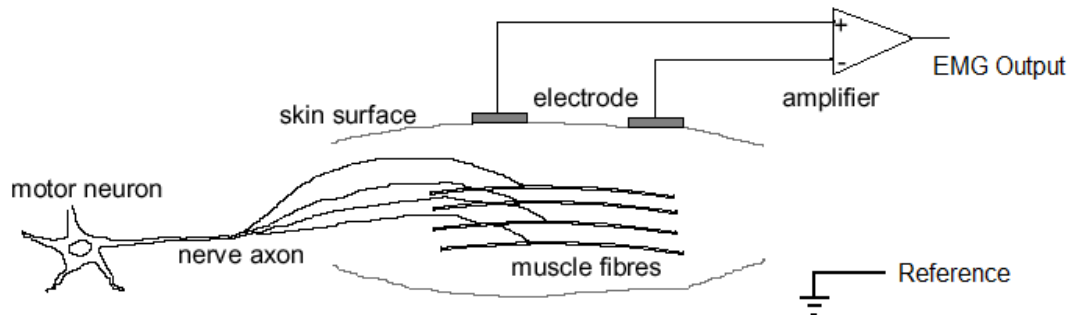


Figura 1.2: Configuración bipolar: dos electrodos activos sobre el músculo y un electrodo de referencia en zona neutra. **Fuente: Adaptado de [1], basado en los fundamentos clásicos de Merletti y Parker [2].**

Los parámetros de colocación con mayor impacto sobre la calidad de la señal son:

- **Distancia inter-electrodo:** 10–20 mm para músculos superficiales. Distancias menores aumentan la selectividad pero reducen la amplitud; distancias mayores incrementan el riesgo de capturar actividad de músculos adyacentes (*crosstalk*).
- **Orientación:** Paralela a las fibras musculares para maximizar la amplitud de señal detectada.
- **Ubicación:** Centro del vientre muscular, evitando la zona de inervación y los tendones.
- **Impedancia balanceada:** Diferencia inferior a 5 k Ω entre ambos electrodos activos para maximizar el rechazo de modo común.

Amplificación diferencial y filtrado anti-aliasing

Antes de la digitalización, la señal sEMG requiere dos etapas de acondicionamiento analógico [5]:

- **Amplificación diferencial:** Incrementa la amplitud de la señal (típicamente 100–1.000 veces) mientras rechaza el ruido de modo común, mejorando la relación señal-ruido antes de la conversión analógica-digital.

- **Filtrado anti-aliasing:** Filtro pasa-bajos analógico que limita el ancho de banda de la señal para prevenir el solapamiento o *aliasing*. En la arquitectura distribuida del sistema, con una frecuencia efectiva de 333 Hz por canal, el filtro incorporado en el módulo sEMG genérico es fundamental para atenuar componentes sobre el límite de Nyquist efectivo de 166,5 Hz [5].

1.4.3. Procesamiento Digital de Señales EMG

Filtro IIR Butterworth pasabanda: fundamento

El filtrado digital permite aplicar criterios de selectividad en frecuencia con mayor precisión que el filtrado analógico, sin degradación de los coeficientes por tolerancias de componentes pasivos [13].

La elección de un filtro Butterworth de orden 4 se fundamenta en su respuesta en frecuencia máximamente plana, propiedad esencial para minimizar la distorsión de amplitud en la envolvente de la señal sEMG utilizada para el control [13].

En el presente sistema, la banda de interés se establece entre 20–150 Hz; este límite superior se ajusta estrictamente para respetar la frecuencia de Nyquist efectiva de 166,5 Hz por canal, derivada del muestreo alternado de los tres grupos musculares instrumentados (bíceps braquial, tríceps braquial y pronador redondo) a una tasa consolidada de 1.000 Hz [5].

La implementación en Forma Directa I (DF-I) opera sobre cada muestra de la señal mediante la ecuación en diferencias:

$$y[n] = \sum_{k=0}^N b_k x[n-k] - \sum_{k=1}^N a_k y[n-k]$$

donde $x[n]$ es la muestra de entrada en el instante n , $y[n]$ es la muestra de salida, b_k son los coeficientes del numerador (ceros del filtro) y a_k son los coeficientes del denominador (polos del filtro), todos obtenidos del diseño Butterworth de orden N [13].

Para el sistema implementado, $N = 4$ y los coeficientes se calcularon en MATLAB para una banda pasante de 20–150 Hz a $F_s = 1,000$ Hz. La DF-I requiere el almacenamiento de $2N = 8$ valores previos: $N = 4$ muestras pasadas de entrada y $N = 4$ muestras pasadas de salida. En el sistema implementado, este filtro se ejecuta en el módulo Python, por lo que los buffers de historial residen en la memoria de la PC y no suponen ninguna carga para la SRAM del ATmega328P.

RMS en ventana deslizante como característica temporal

Las características temporales son preferidas en sistemas de bajo costo con procesamiento en tiempo real por su bajo requerimiento computacional en comparación con características frecuenciales [12]. Las características más utilizadas en la literatura incluyen el valor cuadrático medio (RMS), el valor absoluto medio (MAV), la longitud de la forma de onda (WL) y los cruces por cero (ZCR), entre otras [4].

El **valor cuadrático medio (RMS)** sobre una ventana de N muestras se define como:

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$$

El RMS estima la raíz cuadrada de la potencia promedio de la señal en la ventana temporal y presenta una correlación monótona con el nivel de fuerza de contracción muscular bajo condiciones de contracción isométrica sostenida [5]. Frente a las alternativas, el RMS presenta mayor estabilidad ante contracciones sostenidas en comparación con el MAV, a costa de un mayor costo computacional por la operación de raíz cuadrada.

En el presente sistema, el RMS se calcula en el módulo Python, donde la ausencia de restricciones de punto flotante por hardware hace que el costo computacional no sea una limitación para la frecuencia de actualización de 20 ms [4].

Para el presente sistema, el cálculo se realiza sobre una ventana deslizante de 250 ms por canal. Dado que el ATmega328P alterna el ADC entre tres canales a una frecuencia total de 1.000 Hz, la frecuencia efectiva por canal es de aproximadamente 333 Hz, lo que equivale a $333 \times 0,250 \approx 83$ muestras por canal en cada ventana de análisis. El módulo Python mantiene un buffer independiente por canal con esta capacidad. El uso de una ventana deslizante en lugar de ventanas adyacentes no superpuestas permite una resolución temporal más fina en la detección de cambios en el nivel de activación muscular [12].

La normalización de la amplitud EMG respecto a la contracción voluntaria máxima (%MVC) es una técnica estándar para reducir la variabilidad inter-sujeto e inter-sesión derivada de diferencias en la impedancia de contacto, la colocación de electrodos y las características fisiológicas individuales [5].

El nivel %MVC se calcula como:

$$\%MVC = \frac{\text{RMS}_{\text{actual}} - \text{RMS}_{\text{reposo}}}{\text{RMS}_{\text{MVC}} - \text{RMS}_{\text{reposo}}} \times 100$$

donde $\text{RMS}_{\text{reposo}}$ es el RMS medido con el músculo en reposo completo y RMS_{MVC} es el RMS máximo registrado durante la fase de contracción voluntaria máxima. Esta normalización transforma la señal a una escala relativa de 0–100 %, independiente de las condiciones absolutas de adquisición, y permite definir umbrales de activación reproducibles entre sesiones [4].

En el sistema implementado, los parámetros de normalización se determinan al inicio de cada sesión mediante una fase de calibración de cuatro etapas: reposo (5 s), contracción bíceps braquial MVC (5 s), contracción tríceps braquial MVC (5 s) y contracción pronador redondo MVC (5 s).

Cada canal se calibra de forma independiente para reducir la variabilidad inter-sujeto derivada de diferencias en la colocación de electrodos y características fisiológicas individuales [5].

Restricciones de memoria en el ATmega328P como puente de adquisición

El microcontrolador ATmega328P dispone de 2.048 bytes de SRAM estática [14].

En la arquitectura del sistema, el ATmega328P opera como puente de adquisición y comunicación: digitaliza las señales analógicas de los módulos sEMG y las transmite mediante puerto serial al módulo de procesamiento en Python. Bajo este esquema, el Arduino no almacena ventanas de análisis ni ejecuta el filtro IIR, lo que elimina los requisitos de buffer que caracterizarían a una implementación embebida autónoma.

El uso de SRAM en el Arduino Uno se limita principalmente a:

- Variables de lectura del ADC por canal: 3 variables de tipo `int` (2 bytes cada una), equivalentes a 6 bytes.
- Buffer de transmisión serial: gestionado por la biblioteca `HardwareSerial`, con una capacidad de 64 bytes.
- Variables de control del lazo de muestreo: aproximadamente 20 bytes adicionales.

El uso total estimado de SRAM es inferior a 100 bytes, lo que deja un margen operativo suficiente para la pila de llamadas y las estructuras auxiliares requeridas por el entorno de ejecución de Arduino [15].

El procesamiento de señales (filtrado IIR, extracción de características RMS/MAV/WL/ZCR, normalización %MVC, inferencia del modelo *Random Forest*, filtro EMA simétrico y limitador de tasa de variación) se ejecuta íntegramente en el módulo Python sobre la PC, donde los recursos de memoria no representan una restricción para las ventanas de análisis requeridas.

Resolución del ADC: El ADC de 10 bits sobre una referencia de 5 V produce una resolución de:

$$\Delta V = \frac{V_{\text{ref}}}{2^{10} - 1} = \frac{5,000}{1,023} \approx 4,9 \text{ mV/cuenta}$$

Este piso de cuantización supera la amplitud de un potencial de acción muscular individual (50 μV –5 mV). Sin embargo, para el cálculo de RMS sobre una ventana de aproximadamente 83 muestras por canal (250 ms a 333 Hz efectivos por canal), la señal efectiva es la superposición de potenciales de múltiples unidades motoras simultáneas, cuya amplitud combinada supera el piso de cuantización, haciendo viable la estimación de la envolvente de amplitud con esta resolución [5].

Este parámetro representa una restricción de diseño documentada del hardware de bajo costo seleccionado, la cual se mitiga mediante el procesamiento estadístico de la señal compuesta.

1.4.4. Estrategia de Control por Estimación de Patrones

A diferencia de los esquemas de control proporcional basados en umbrales lineales, la traducción de la intención motora a comandos mecánicos se gestiona mediante el mapeo no lineal del modelo *Random Forest* en modo regresión. Aunque el entrenamiento se realiza a partir de

capturas en posiciones angulares preestablecidas, el modelo estima de forma continua posiciones angulares intermedias no vistas durante el entrenamiento, despachando directrices de posición angular dinámicas hacia el controlador de actuadores.

Estabilización de transiciones y mitigación de oscilaciones

La naturaleza estocástica de la sEMG puede provocar variaciones rápidas en la salida del regresor. Para garantizar la fluidez mecánica del brazo robótico sin comprometer la latencia, el módulo en Python implementa un filtro de suavizado temporal (como un promedio móvil o filtro EMA simétrico) sobre las predicciones de posición angular. El comando definitivo se transmite al hardware de manera continua, actuando como un filtro pasa-bajos algorítmico que elimina discontinuidades mecánicas.

Control de posición continua mediante PWM y driver PCA9685

El control de posición de servomotores se implementa mediante Modulación por Ancho de Pulso (PWM), donde el ancho del pulso en cada período determina la posición angular estimada por el modelo *Random Forest* en modo regresión. El servo KS-3518 opera con pulsos en el rango 500–2400 μs , correspondiente al rango continuo de posición entre 0°–180°, a una frecuencia de portadora de 50 Hz [16].

El driver PCA9685 genera las señales PWM mediante resolución de 12 bits (4096 cuentas por período de 20 ms) a través del bus I²C, desacoplando la generación de PWM del ciclo de ejecución del microcontrolador.

Este desacoplamiento permite que el ATmega328P dedique sus ciclos de instrucción exclusivamente a la adquisición de las señales sEMG y a su transmisión por puerto serial, sin interrupciones para la generación de la portadora PWM. La resolución de posición teórica disponible para el rango continuo del actuador es:

$$\Delta\theta = \frac{180^\circ}{(2400 - 500) \mu\text{s}} \times \frac{20000 \mu\text{s}}{4096 \text{ cuentas}} \approx 0,44^\circ/\text{cuenta}$$

La frecuencia de oscilador interno del PCA9685 se configura en 25 MHz para garantizar la correspondencia exacta entre cuentas, microsegundos de pulso y la posición angular continua despachada por el regresor.

1.4.5. Métricas de Evaluación del Sistema

Latencia extremo a extremo

La latencia total del sistema se define como el tiempo transcurrido desde el inicio de la contracción muscular hasta el inicio del movimiento robótico correspondiente. Para aplicaciones de control mioeléctrico en tiempo real, la literatura establece que la latencia extremo a extremo debe mantenerse por debajo de 200 ms para garantizar un control natural y fluido; por encima

de este umbral los usuarios perciben desacoplamiento entre la intención motora y la respuesta del sistema [12].

Para el *pipeline* implementado, la latencia total se descompone en cuatro componentes:

1. **Latencia de adquisición:** determinada por el período de muestreo ($T_s = 1$ ms) y el tiempo de llenado de la primera ventana válida (250 ms). Esta componente domina la latencia total en el ciclo de inicialización del sistema.
2. **Latencia de procesamiento:** tiempo de cálculo del filtro IIR (4 multiplicaciones y sumas por muestra), extracción de características (RMS, MAV, WL, ZCR; aproximadamente 83 operaciones por canal), normalización %MVC, inferencia del modelo *Random Forest* (determinada por el número de estimadores y la profundidad del árbol) y filtro EMA simétrico, ejecutados en el módulo Python sobre la PC. Dado que el procesamiento no se realiza en el microcontrolador, no está limitado por la velocidad del ATmega328P ni por la ausencia de unidad de punto flotante.
3. **Latencia de actualización de control:** el paso de actualización del lazo de control es de 20 ms, lo que establece la granularidad mínima de respuesta del sistema en operación continua.
4. **Latencia de actuación:** tiempo de respuesta mecánica del servo KS-3518, función del desplazamiento angular requerido y la velocidad nominal de 300°/s sin carga.

La latencia objetivo de 200 ms corresponde al estado estacionario del sistema, es decir, una vez completado el llenado inicial del buffer de 250 ms.

En operación continua, la respuesta ante una nueva contracción muscular ocurre en el siguiente ciclo de actualización de 20 ms, a lo que se suma el tiempo de procesamiento en Python y la latencia de actuación mecánica.

Relación señal-ruido (SNR)

La relación señal-ruido cuantifica la calidad de la señal adquirida [5]:

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \left(\frac{P_{\text{señal}}}{P_{\text{ruido}}} \right)$$

En el sistema implementado, las principales fuentes de ruido residual son la interferencia de red eléctrica a 50 Hz (no atenuada por el filtro pasabanda), los artefactos de movimiento de los cables de electrodo y el ruido térmico del módulo sEMG genérico. Estas fuentes establecen el límite inferior del SNR efectivo del sistema y representan restricciones conocidas del diseño de bajo costo.

Escala de usabilidad del sistema (SUS)

La Escala de Usabilidad del Sistema (SUS, *System Usability Scale*) proporciona una medida estandarizada de la experiencia del usuario con un sistema interactivo [17]. El instrumento consta de 10 ítems evaluados en escala Likert de 5 puntos, que cubren dimensiones de facilidad de aprendizaje, eficiencia de uso, consistencia del comportamiento y satisfacción general. La puntuación final se transforma a una escala de 0 a 100 mediante la fórmula estandarizada del instrumento.

Según el meta-análisis de Hyzy et al. [17], una puntuación SUS superior a 68 se corresponde con una usabilidad aceptable en aplicaciones de salud digital, mientras que valores superiores a 85 indican un nivel de usabilidad alto. El instrumento SUS se aplica en el presente proyecto durante la fase de evaluación de usabilidad (Espiral 5) para obtener una medida cuantitativa de la experiencia de los usuarios con el sistema de control EMG.

1.5. Estado del Arte

La revisión de literatura presentada en esta sección se realizó mediante búsqueda en IEEE Xplore, Scopus y Google Scholar con los términos: “*sEMG proportional control*”, “*low-cost EMG microcontroller*”, “*EMG threshold embedded control*”, “*myoelectric human-machine interface*” y “*surface EMG robotic arm low cost*”. Se consideraron trabajos publicados entre 2019 y 2024. Se priorizaron artículos de revistas y conferencias indexadas en IEEE y Frontiers, con implementaciones evaluadas experimentalmente. Se excluyeron trabajos basados exclusivamente en simulación, sin protocolo de evaluación experimental documentado, o con sistemas de adquisición cuyo costo de hardware impidiera comparación con plataformas accesibles.

1.5.1. Técnicas de control proporcional basadas en amplitud sEMG

El control proporcional basado en amplitud de señal EMG representa la aproximación de menor complejidad algorítmica para sistemas de control continuo. En esta técnica, una característica temporal de la señal (típicamente el valor cuadrático medio RMS o el valor absoluto medio MAV) se normaliza respecto a un nivel de referencia y se mapea directamente a un comando de posición o velocidad del actuador, sin requerir etapas de clasificación supervisada.

La revisión sistemática de Rodríguez-Tapia et al. [4] analiza el estado del procesamiento de señales EMG en interfaces mioeléctricas para aplicaciones de rehabilitación, asistencia y control de dispositivos, identificando que los sistemas de bajo costo que emplean características temporales simples sobre hardware de propósito general presentan limitaciones en la documentación de sus protocolos de calibración y en la estandarización de sus métricas de evaluación. Esta revisión establece que la mayoría de implementaciones accesibles priorizan la simplicidad de la cadena de procesamiento en detrimento de la reproducibilidad del protocolo experimental.

En el contexto del control proporcional, la elección de la característica temporal tiene implicaciones directas sobre la latencia y estabilidad del sistema. El RMS sobre ventana deslizando

ha sido identificado como la característica con mayor estabilidad ante contracciones sostenidas, mientras que el MAV ofrece menor costo computacional al evitar la operación de raíz cuadrada, siendo preferido en plataformas con capacidad de procesamiento restringida, dado el mayor costo computacional de la raíz cuadrada respecto al promedio de valor absoluto [4].

1.5.2. Implementaciones de bajo costo sobre microcontroladores embebidos

Un subconjunto de los trabajos revisados propone implementaciones de sistemas sEMG sobre hardware de bajo costo, con el objetivo de reducir la barrera económica para prototipado educativo y aplicaciones de investigación básica.

Garouche y Thamsuwan [3] presentan un sistema portátil basado en un microcontrolador *Raspberry Pico* y un módulo *Myoware* para la detección de fatiga muscular. A diferencia de los sistemas de clasificación de patrones, este dispositivo realiza la extracción de características como la raíz cuadrada media (RMS) y las frecuencias de potencia media (MNF) y mediana (MDF) para monitorear cambios fisiológicos en tiempo real. Durante contracciones isotónicas, el sistema reportó un comportamiento similar a equipos de grado de investigación en el 70,58 % de los casos. El costo total de hardware se sitúa en aproximadamente 350 USD, incluyendo componentes, carcasa y electrodos. Entre sus limitaciones documentadas se encuentra una mayor sensibilidad al ruido y artefactos de movimiento debido a la resolución de su ADC de 10 bits y al uso de electrodos de gel flexibles.

En el ámbito de las plataformas de adquisición de bajo costo, Yassin et al. [7] presentan un sistema de biofeedback EMG portátil y de bajo costo orientado a aplicaciones de telerehabilitación, con evaluación clínica en usuarios con condiciones neuromusculares. El trabajo documenta las limitaciones de los módulos EMG genéricos en condiciones de uso real y la viabilidad de plataformas accesibles para seguimiento remoto de la actividad muscular.

1.5.3. Comparativa de técnicas: control proporcional versus clasificación supervisada

Los trabajos publicados en el período 2019–2024 se distribuyen en dos categorías metodológicas principales para el control robótico basado en sEMG: el control proporcional continuo sobre características temporales, y la clasificación supervisada de patrones musculares.

Clasificación supervisada con aprendizaje profundo

Zandigohar et al. [12] proponen un sistema de control de prótesis de mano basado en fusión multimodal de señales sEMG e información visual. El sistema emplea redes neuronales profundas para inferir la intención de agarre del usuario a partir de señales mioeléctricas multicanal complementadas con datos de visión por computadora. La propuesta alcanza latencias inferiores a 200 ms en condiciones de laboratorio, con alta precisión en la identificación del patrón de agarre. Sus requisitos computacionales, que incluyen procesamiento visual en tiempo real y

modelos de aprendizaje profundo, lo posicionan fuera del alcance de plataformas de bajo costo basadas en microcontroladores de 8 bits.

Krasoulis et al. [18] presentan una interfaz humano-máquina que combina señales EMG con datos de una unidad de medición inercial (IMU), utilizando un arreglo minimalista de dos sensores, para el control de una prótesis de mano. El sistema emplea técnicas de aprendizaje automático aplicadas sobre la fusión de ambas modalidades, mejorando la precisión del reconocimiento de gestos respecto al uso de EMG de forma aislada. Este enfoque ilustra una estrategia de reducción de hardware dentro de la literatura de sistemas multimodales; sin embargo, su dependencia de clasificadores entrenados con datos etiquetados lo aparta del esquema de control continuo sin fase de entrenamiento supervisado adoptado en el presente proyecto.

Bai et al. [10] proponen un modelo CNN-LSTM ligero para reconocimiento de gestos mediante señales sEMG multicanal. El sistema alcanza alta precisión en clasificación de estados de agarre, pero requiere conjuntos de entrenamiento extensos (>10.000 muestras) y capacidad computacional que supera la disponible en plataformas de microcontrolador de bajo costo sin unidad de punto flotante por hardware.

Síntesis comparativa

La Tabla 1.2 resume los compromisos técnicos de los trabajos identificados en la literatura respecto a los criterios más relevantes para el contexto de este proyecto: número de canales, técnica de procesamiento, latencia reportada y costo de hardware.

Justificación de la selección: Random Forest frente a alternativas

La revisión comparativa presentada permite fundamentar la elección de un modelo *Random Forest* en modo regresión frente a las alternativas identificadas en la literatura, en función de tres criterios: requerimientos de memoria y cómputo, latencia de inferencia y precisión reportada.

Memoria y cómputo: A diferencia de las arquitecturas CNN-LSTM, que requieren conjuntos de entrenamiento superiores a 10.000 muestras y capacidad computacional con unidad de punto flotante dedicada [10], un modelo *Random Forest* puede entrenarse con conjuntos de datos de menor tamaño y no depende de aceleración por GPU. Esta característica es determinante en la arquitectura distribuida propuesta, donde la inferencia se ejecuta en la PC *host* y el microcontrolador ATmega328P permanece dedicado exclusivamente a la adquisición y transmisión serial, sin capacidad de punto flotante por hardware. Estudios recientes reportan además que arquitecturas de bosques aplicadas a señales sEMG mantienen un uso de memoria reducido (aproximadamente 1,26 MB), compatible con estaciones de trabajo de gama básica [20].

Latencia de inferencia: La inferencia de un *Random Forest* tiene una complejidad acotada por el número de árboles y su profundidad máxima, lo que resulta en tiempos de predicción más predecibles que los de una red recurrente o convolucional profunda. Esta previsibilidad

favorece el cumplimiento del umbral de 200 ms de latencia extremo a extremo establecido para el sistema, en contraste con propuestas como la de Zandigohar et al. [12], que si bien alcanzan latencias inferiores a 200 ms, dependen de procesamiento visual en tiempo real y por tanto de hardware de aceleración gráfica.

Precisión frente a simplicidad de hardware: Se reconoce que los modelos de aprendizaje profundo y los enfoques de fusión multimodal reportan una precisión de clasificación superior a la de los métodos de ensamble tradicionales [10, 12]. Sin embargo, dicha precisión se obtiene a costa de requerimientos de hardware que exceden el umbral de 350 USD adoptado como referencia de bajo costo en este proyecto [3]. Frente a arquitecturas minimalistas como la de Krasoulis et al. [18], que reduce el número de sensores pero mantiene la dependencia de clasificadores entrenados con datos etiquetados para comandos discretos, el uso de *Random Forest* en modo regresión permite conservar una estimación continua de posición angular sin renunciar a la simplicidad de despliegue.

En consecuencia, el *Random Forest* en modo regresión se adopta como el punto de equilibrio entre la robustez frente a relaciones no lineales –superior al control proporcional por umbral– y la viabilidad de ejecución sobre hardware de bajo costo, sin incurrir en los requerimientos computacionales de las arquitecturas de aprendizaje profundo revisadas.

1.5.4. Brecha identificada

El análisis de los trabajos revisados permite identificar una discontinuidad entre dos extremos del espectro de implementaciones sEMG para control robótico:

1. **Sistemas de alto rendimiento con alta complejidad:**

Los trabajos que reportan alta precisión de control emplean fusión multimodal, redes neuronales profundas y hardware multicanal, con requisitos computacionales y económicos que los excluyen de contextos con presupuesto limitado [12]. Enfoques con menor número de sensores, como el de Krasoulis et al. [18], reducen la complejidad de hardware, pero mantienen la dependencia de clasificadores entrenados con datos etiquetados, lo que limita su aplicabilidad directa a esquemas de control continuo sin fase de entrenamiento supervisado.

2. **Sistemas de bajo costo con documentación incompleta:** Las implementaciones accesibles identificadas en la revisión carecen de protocolos completos de calibración, procesamiento y evaluación que permitan su reproducción directa en otros contextos [3, 4].

En particular, la revisión no identificó implementaciones de bajo costo que combinen la adquisición sEMG multicanal en un microcontrolador ATmega328P básico, la delegación completa del *pipeline* de características a una PC mediante comunicación serie y el uso de un modelo *Random Forest* en modo regresión para mantener la latencia extremo a extremo por debajo de los 200 ms, a un costo inferior a 350 USD. Esta falta de documentación y evaluación de arquitecturas distribuidas accesibles constituye la brecha técnica que este proyecto aborda.

Trabajo	Canales	Técnica	Latencia	Precisión	Costo (USD)	Limitación principal
Rodríguez-Tapia et al. (2020) [4]	revisión	Revisión sistemática sEMG-HMI	—	—	—	No es implementación
Garouche y Thamsuwan (2023) [3]	1	RMS + MNF/MDF	—	70,58 % de concordancia con equipo de investigación	≈350	Sensibilidad al ruido; ADC de 10 bits
Bai et al. (2021) [10]	Multi	sEMG + CNN-LSTM	tiempo real	No reportada en unidades comparables	No reportado	Requiere >10.000 muestras y cómputo elevado
Zandigohar et al. (2024) [12]	Multi	sEMG + visión + DL	<200 ms	Alta precisión de agarre (no cuantificada en %)	No reportado (requiere GPU)	Requiere GPU y visión por computadora
Krasoulis et al. (2020) [18]	2	EMG + IMU + ML	tiempo real	Mejora respecto a EMG aislado (no cuantificada)	No reportado	Requiere entrenamiento supervisado y clasificación discreta
Jiang et al. (2023–2024) [19,20]	Multi (HD-sEMG)	Deep Forest	9,08 ms	Reducción de error >50 % vs. modelos base	No reportado	Requiere sensores de alta densidad
Propuesta actual	3	sEMG (Features) + Random Forest (PC)	<200 ms	MAE <15°, R^2 >0,85 (objetivo)	≈335	Dependencia de PC; requiere entrenamiento previo

Tabla 1.2: Comparativa de trabajos relacionados según técnica de procesamiento, número de canales, latencia, precisión y costo reportados.

Capítulo 2

Metodología de Trabajo

2.1. Alcance

El producto principal es un sistema experimental de adquisición y procesamiento de señales sEMG orientado a la estimación continua de posición angular para el control de actuadores robóticos mediante un modelo de **regresión cinemática basado en *Random Forest***, permitiendo el mapeo fluido y no lineal de los patrones de activación muscular a la trayectoria del actuador.

1. El sistema incluye: adquisición analógica multicanal en microcontrolador, transmisión serial en tiempo real, un *pipeline* de procesamiento digital de señales (DSP), extracción de un vector de características temporales y la **inferencia continua** ejecutada íntegramente en la PC *host* (Python).
2. El reconocimiento de la intención motora se realiza mediante un **modelo de regresión continua**: el algoritmo estima e interpola de forma no lineal la **posición angular del actuador** (0° a 180°) a partir de los patrones de activación detectados en los tres canales instrumentados (bíceps braquial, tríceps braquial y pronador redondo).
3. La aplicación se limita a activar un conjunto reducido de movimientos en un actuador robótico.
4. Los beneficios se relacionan con la evaluación técnica de sensores, parámetros de filtrado y estabilidad del mecanismo de activación.

2.2. Objetivos

2.2.1. Objetivo General

Desarrollar un sistema experimental de control robótico mediante señales EMG superficiales, implementando un *pipeline* de procesamiento orientado a la formación académica, investigación aplicada y prototipado de interfaces humano-máquina

2.2.2. Objetivos Específicos

1. Diseñar el sistema electrónico de adquisición EMG, garantizando una captura estable, eficiente y con indicadores adecuados de calidad de señal.
2. Implementar el software de procesamiento para extraer características temporales y estimar la posición angular en tiempo real a partir de los patrones de activación muscular multicanal.
3. Integrar el módulo de adquisición y procesamiento con un brazo robótico, habilitando la ejecución coordinada y precisa de movimientos.
4. Evaluar el desempeño técnico y la usabilidad del sistema en escenarios académicos y de interfaz humano-máquina.

2.3. Desglose de Actividades y Tareas

Esta sección desglosa los cuatro Objetivos Específicos del proyecto en actividades concretas, especificando insumos requeridos, entregables verificables y hitos de progreso.

2.3.1. Objetivo Específico 1: Diseño del Sistema EMG

1.1 Selección y Configuración de Sensores

- **Insumos:** Selección de sensores comerciales, prototipado de circuito de acondicionamiento, calibración de ganancia.
- **Entregable:** Circuito funcional con SNR >20 dB evaluado experimentalmente.
- **Hito:** Hardware EMG listo para pruebas de adquisición continua.

1.2 Diseño de Topología de Electrodo

- **Insumos:** Protocolo SENIAM para colocación de electrodos bipolares.
- **Entregable:** Topología evaluada con impedancia balanceada <5 k Ω de diferencia.
- **Hito:** Protocolo de captura documentado y replicable.

1.3 Evaluación de Calidad de Señales

- **Insumos:** Medición de SNR, evaluación de repetibilidad (10 sesiones), evaluación del protocolo.
- **Entregable:** Informe de calidad de señal con métricas estadísticas (media, desviación estándar).
- **Hito:** Sistema de adquisición evaluado para pasar a Espiral 2.

2.3.2. Objetivo Específico 2: Procesamiento y Estimación de Patrones EMG

2.1 Procesamiento de Señales EMG

- **Insumos:** Filtro Butterworth pasa-banda (20–150 Hz), envolvente RMS, normalización MVC, evaluación del *pipeline*.
- **Entregable:** *Pipeline* de preprocesamiento evaluado con latencia <50 ms.
- **Hito:** Señales preprocesadas listas para extracción de características.

2.2 Extracción de Características

- **Insumos:** Cuatro características temporales por canal (RMS, MAV, WL, ZCR), ventana deslizante de 250 ms con paso de 20 ms.
- **Entregable:** Conjunto de características estandarizado (media=0, std=1).
- **Hito:** Datos procesados listos para el entrenamiento del **modelo de regresión** cinemática.

2.3 Gestión de datos de entrenamiento

- **Insumos:** Sesiones de captura etiquetadas por ángulo objetivo y sujeto, formato CSV, protocolo de adquisición estandarizado.
- **Entregable:** Conjunto de datos de entrenamiento estructurado con valores de posición angular evaluados (*Ground Truth*), listo para entrenar el modelo de regresión *Random Forest*.
- **Hito:** Dataset con mínimo N sesiones por posición de referencia disponible para entrenamiento.

2.4 Regresor de Posición en Tiempo Real

- **Insumos:** Modelo *Random Forest* en modo regresión, optimización del número de estimadores y profundidad máxima para mantener latencia de inferencia <100 ms, evaluación mediante evaluación cruzada 5-fold.
- **Entregable:** Modelo de regresión operativo con RMSE <10° sobre el conjunto de evaluación.
- **Hito:** Modelo de regresión cinemática integrado y listo para el control continuo del brazo robótico.

2.3.3. Objetivo Específico 3: Integración con el Brazo Robótico

3.1 Integración Hardware-Software

- **Insumos:** Protocolo serial UART (115200 baud) PC (Python)–Arduino para el retorno de posiciones calculadas, protocolo síncrono I2C (400 kHz) para el control PWM de 2 servomotores vía PCA9685 desde el microcontrolador, mecanismos de sincronización temporal, manejo de errores críticos (timeout de conexión, evaluación por checksum), y un umbral de latencia extremo-a-extremo menor a 200 ms.
- **Entregable:** Módulo de comunicación integrado en el firmware del ATmega328P y el script de Python, equipado con algoritmos de reintentos automáticos ante fallos de lectura, desborde de buffers o pérdida de paquetes en los buses UART e I2C.
- **Hito:** Sistema EMG–robótico operativo con control básico de 2 GDL en tiempo real, evaluando la cadena cinemática desde la captura muscular hasta la respuesta del servo.

3.2 Mapeo de Patrones EMG a Comandos

- **Insumos:** Esquema de mapeo continuo entre salida del regresor (ángulo estimado, 0° – 180°) y comando PWM al PCA9685, evaluado mediante 10 secuencias de referencia predefinidas con ángulos objetivo conocidos.
- **Entregable:** Esquema de mapeo ángulo estimado–comando con transiciones evaluadas.
- **Hito:** Patrones musculares mapeados correctamente a movimientos robóticos.

3.3 Generación de Trayectorias

- **Insumos:** Interpolación lineal y spline cúbico, evaluación de suavidad con acelerómetro ($\text{jerk} < 10 \text{ m/s}^3$).
- **Entregable:** Módulo de trayectorias implementado con movimientos coordinados de 2 GDL.
- **Hito:** Movimientos precisos y suaves sin oscilaciones $> 5^{\circ}$.

2.3.4. Objetivo Específico 4: Evaluación del Sistema

4.1 Evaluación Técnica

- **Insumos:** 15 casos de prueba técnica, medición de latencia (E2E), análisis de error MAE (Error Absoluto Medio), coeficiente de determinación R^2 , pruebas de operación continua.
- **Entregable:** Informe técnico con métricas verificadas y análisis estadístico.
- **Hito:** Evaluación técnica completada con cumplimiento de umbrales.

4.2 Evaluación de Usabilidad

- **Insumos:** 6 sesiones con usuarios (protocolo SUS), análisis de curva de aprendizaje, implementación de 5 mejoras prioritarias.
- **Entregable:** Sistema optimizado con SUS >70 y tiempo de aprendizaje <15 min.
- **Hito:** Evaluación de usabilidad finalizada con feedback documentado.

2.4. Planificación Temporal

2.4.1. Carta Gantt

La planificación temporal refleja la estructura iterativa del Modelo en Espiral, con ciclos de desarrollo que incluyen fases de ingeniería, evaluación y puntos de decisión explícitos. Las contingencias se gestionan dentro de cada espiral sin buffers temporales separados.

Leyenda de elementos:

- **ESP 1-5:** Cinco espirales iterativas del desarrollo
- **Barras azules:** Actividades de ingeniería y desarrollo
- **Barras verdes:** Fases de evaluación con criterios específicos
- **Barras naranjas:** Agrupación de espirales completas (incluyen ingeniería + evaluación)

2.4.2. Carta Gantt General del Proyecto

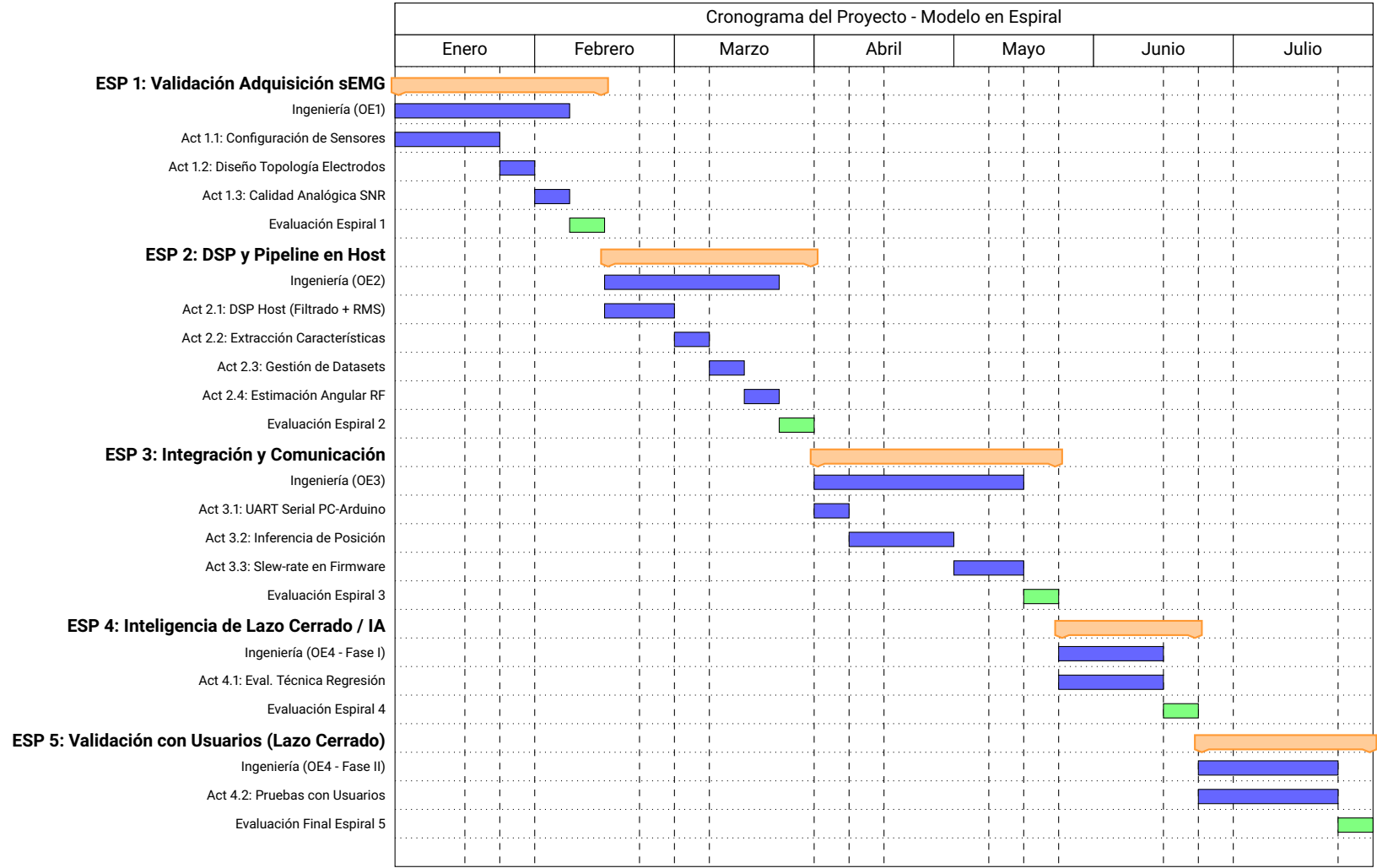


Figura 2.1: Carta Gantt general del proyecto basada en Modelo en Espiral. ESP: Espiral; OE: Objetivo Específico. Las barras azules representan el desarrollo técnico de ingeniería, las verdes las etapas formales de evaluación y las naranjas agrupan el ciclo iterativo completo de cada espiral.

Criterios de Evaluación por Espiral

La Tabla 2.1 establece los umbrales técnicos que determinan si una espiral ha sido exitosa o requiere activación de estrategias de mitigación. Estos criterios son medibles y están basados en estándares de la literatura revisada.

Espiral	Criterios Técnicos	Decisión: Continuar	Decisión: Activar Mitigación
ESP 1: Adquisición	Captura síncrona a 1.000 Hz. Calidad analógica: SNR > 20 dB [5].	SNR promedio en los 3 canales superior a 20 dB frente a contracción controlada.	Ajustar topología de electrodos, limpiar piel con alcohol e inspeccionar blindaje del cable de señal.
ESP 2: DSP en Host	Filtro digital pasabanda (20–150 Hz). Latencia de cómputo del pipeline < 10 ms [4].	Desviación frente a MATLAB < 1 % y tiempo de procesamiento en host < 10 ms.	Optimizar operaciones vectoriales en NumPy y remover llamadas de visualización pesadas del lazo.
ESP 3: Integración	Protocolo UART a 115.200 baud. Control PWM descentralizado vía I ² C rápida a 400 kHz.	Transmisión de tramas sin pérdida de bytes y limitador de rampa local a 4,8°/ciclo operativo.	Implementar control por timeout serie y validar acoplamiento de la GND común del lazo de potencia.
ESP 4: Regresor IA	Inferencia dual mediante RandomForest. Precisión: MAE < 15°, $R^2 > 0,85$.	Modelo entrenado logra error absoluto medio por articulación < 15° en validación cruzada.	Ajustar hiperparámetros de profundidad máxima y número de estimadores en scikit-learn.
ESP 5: Usabilidad	Evaluación experimental con usuarios. Usabilidad: Puntaje SUS ≥ 68 [17].	Promedio del cuestionario SUS de la muestra de operadores ≥ 68 puntos.	Rediseñar la guía de instrucciones por consola de la interfaz de calibración interactiva.

Tabla 2.1: Criterios técnicos de aceptación y estrategias de mitigación por espiral

2.4.3. Desglose de Actividades

Esta sección presenta la distribución temporal detallada de las actividades del proyecto, complementando el Diagrama de Gantt con datos cuantitativos sobre carga de trabajo, dependencias críticas y recursos asignados.

Resumen Temporal por Objetivo Específico

Objetivo Específico	Duración	Período	Semanas
OE1: Diseño del Sistema EMG	Enero S1 – Feb S2	Ene–Feb 2026	6 semanas
OE2: Procesamiento y Estimación de Patrones	Feb S3 – Mar S4	Feb–Mar 2026	6 semanas
OE3: Integración Robótica	Abril S1 – Mayo S2	Abr–May 2026	7 semanas
OE4: Evaluación del Sistema	Mayo S3 – Julio S1	May–Jul 2026	9 semanas
Total			28 semanas

Tabla 2.2: Resumen temporal de los Objetivos Específicos del proyecto.

Distribución de Esfuerzo por Actividad

Objetivo Específico 1: Adquisición de 3 Canales (6 semanas)

- Actividad 1.1 (Selección de Sensores y Acondicionamiento): 50 % del tiempo (3 semanas). Se centra en la captura de dinámica flexora (A0), extensora (A1) y de rotación (A2).
- Actividad 1.2 (Diseño de Topología de Electrodo): 17 % del tiempo (1 semana). Ubicación en bíceps braquial, tríceps braquial y pronador redondo siguiendo estándar SENIAM.
- Actividad 1.3 (Evaluación de SNR): 17 % del tiempo (1 semana). Validación de calidad de señal ($SNR > 20$ dB) antes del procesamiento digital.
- Evaluación Espiral 1: 17 % del tiempo (1 semana).

Objetivo Específico 2: DSP y Pipeline Dual en Host (6 semanas)

- Actividad 2.1 (DSP Host - Python): 33 % del tiempo (2 semanas). Implementación del filtro Butterworth (20–150 Hz) para respetar Nyquist de 166,5 Hz por canal.
- Actividad 2.2 (Extracción de Features): 17 % del tiempo (1 semana). Cálculo de RMS, MAV, WL y ZCR en ventanas deslizantes de 250 ms.
- Actividad 2.3 (Gestión de Datasets): 17 % del tiempo (1 semana). Captura sincronizada de señales etiquetadas por ángulo objetivo (*Ground Truth*).
- Actividad 2.4 (Estimación Angular Continua): 17 % del tiempo (1 semana). Entrenamiento del modelo **Random Forest** en modo regresión para el mapeo fluido y continuo de las características sEMG a la posición angular del actuador.
- Evaluación Espiral 2: 16 % del tiempo (1 semana).

Objetivo Específico 3: Integración y Control Cinemático (7 semanas)

- Actividad 3.1 (Comunicación Bidireccional): 14 % del tiempo (1 semana). Protocolo UART (115.200 baud) para *offload* de datos y retorno de setpoints.
- Actividad 3.2 (Inferencia Cinemática Continua): 43 % del tiempo (3 semanas). Lógica de interpolación angular del regresor.
- Actividad 3.3 (Limitación de Slew-rate): 29 % del tiempo (2 semanas). Restricción de 4,8°/ciclo para protección de engranajes del servo KS-3518.
- Evaluación Espiral 3: 14 % del tiempo (1 semana).

Objetivo Específico 4: Evaluación y Validación (9 semanas)

- Actividad 4.1 (Evaluación Técnica de Regresión): 33 % del tiempo (3 semanas). Medición del Error Absoluto Medio (MAE) y el coeficiente de determinación R^2 para validar la precisión y estabilidad de la estimación angular continua.
- Actividad 4.2 (Evaluación de Usabilidad): 33 % del tiempo (3 semanas). Aplicación de escala estandarizada SUS con sujetos reales.
- Evaluación Espiral 4: 11 % del tiempo (1 semana).
- Evaluación Espiral 5: 22 % del tiempo (2 semanas). Consolidación de resultados y viabilidad económica bajo el umbral de 350 USD.

Ruta Crítica del Proyecto

La ruta crítica identifica la secuencia de actividades que determinan la duración mínima del proyecto. Cualquier retraso en estas actividades impacta directamente la fecha de finalización:

Secuencia de ruta crítica (28 semanas):

1. Act 1.1 → Act 1.2 → Act 1.3 → Eval. ESP1 (6 semanas)
2. Act 2.1 → Act 2.2 → Eval. ESP2 (4 semanas)
3. Act 2.4 → Act 3.1 → Act 3.2 → Act 3.3 → Eval. ESP3 (7 semanas)
4. Eval. ESP4 (1 semana)
5. Act 4.1 → Act 4.2 → Eval. ESP5 (9 semanas)

Dependencias entre Actividades**Dependencias Obligatorias (Hard Dependencies):**

- **D1:** Act 1.3 debe completarse antes de iniciar Act 2.1 (señales calibradas requeridas)
- **D2:** Act 2.1 y Act 2.2 deben completarse antes de Act 2.4 (datos procesados para entrenamiento)
- **D3:** Act 2.4 debe completarse antes de Act 3.1 (modelo de regresión cinemática operativo requerido).
- **D4:** Act 3.1 y Act 3.2 deben completarse antes de Act 3.3 (integración básica requerida)
- **D5:** OE3 completo antes de iniciar OE4 (sistema integrado para evaluación)

Dependencias Preferidas (Soft Dependencies):

- **D6:** Act 2.3 preferiblemente antes de Act 2.4 (facilita gestión de datos de entrenamiento)
- **D7:** Act 4.1 preferiblemente antes de Act 4.2 (evaluación técnica antes de pruebas con usuarios)

Holguras Incorporadas

Actividad	Duración Base	Holgura Incorporada
Act 1.1	2.5 semanas	0.5 semanas (20 %)
Act 2.1	2.5 semanas	0.5 semanas (20 %)
Act 2.4	1.5 semanas	0.5 semanas (33 %)
Act 3.1	0.8 semanas	0.2 semanas (25 %)
Act 4.1	2.5 semanas	0.5 semanas (20 %)

Tabla 2.3: Holguras temporales incorporadas en las actividades principales del proyecto.

Recursos Asignados por Fase

Fase	Personal	Hardware	Software
OE1	1 desarrollador (100 %)	Sensores EMG	Python, LTSpice
OE2	1 desarrollador (100 %)	PC Local	Python
OE3	1 desarrollador (100 %)	Arduino, brazo robótico, PCA9685	Python, Arduino IDE
OE4	1 desarrollador (50 %) + 3 usuarios	Sistema completo	Python, herramientas de análisis

Tabla 2.4: Recursos asignados por fase del proyecto.

Indicadores de Avance

Para el seguimiento del progreso del proyecto, se establecen los siguientes indicadores cuantitativos:

- **Porcentaje de completitud:** $(\text{Hitos completados} / \text{Total de hitos}) \times 100$
- **Desviación temporal:** $\text{Semanas reales} - \text{Semanas planificadas}$
- **Cumplimiento de entregables:** $(\text{Entregables aceptados} / \text{Entregables programados}) \times 100$

- **Cumplimiento de criterios técnicos:** Cantidad de métricas que alcanzan los umbrales definidos

Revisiones programadas:

- Revisión semanal de avance
- Revisión crítica al finalizar cada espiral
- Presentación de avances ante el docente guía cada 2 semanas
- Revisión final de integración al completar OE3

2.5. Análisis de Viabilidad del Proyecto

2.5.1. Viabilidad Técnica

Tecnologías Maduras y Disponibles El sistema propuesto se fundamenta en tecnologías maduras y accesibles para el desarrollo de interfaces mioeléctricas. Específicamente, se emplean módulos sEMG con salida analógica integrados a una etapa de instrumentación basada en amplificadores de instrumentación *INA114* y filtrado activo (20–300 Hz). Se utiliza un microcontrolador como puente de adquisición serial (*DAQ bridge*) hacia una PC *host* [19].

Esta arquitectura es consistente con enfoques recientes de control mioeléctrico basados en *Deep Forest* (Bosques Profundos), los cuales han demostrado una robustez superior frente al ruido y una mayor capacidad de generalización inter-diaria en la estimación de variables biomecánicas en comparación con los métodos de regresión tradicionales [19]. Asimismo, se destaca la viabilidad de estos modelos de bosque para escenarios de calibración rápida *one-shot*, permitiendo la adaptación del sistema con apenas un segundo de datos por gesto sin comprometer el desempeño en tiempo real [20].

Complejidad Computacional Manejable : Las etapas de procesamiento propuestas, tales como: filtrado digital, extracción de características temporales y estimación angular continua mediante *Random Forest* en modo regresión. Presentan una carga computacional acotada, compatible con una ejecución en tiempo real sobre una arquitectura distribuida. Estudios recientes validan esta viabilidad reportando tiempos de procesamiento de apenas 9.08 ms para el flujo completo de extracción y clasificación [20]. La delegación de la inferencia a la PC *host* permite preservar la estabilidad del lazo de control y evitar sobrecargar al microcontrolador con tareas de cómputo intensivo, en concordancia con antecedentes que muestran la factibilidad de aplicar *Random Forest* a señales sEMG en sistemas embebidos debido a su reducido uso de memoria (aprox. 1.26 MB) [19, 20]. En este contexto, la arquitectura propuesta favorece una implementación reproducible, robusta frente al ruido de los canales [19] y de complejidad manejable para escenarios de interacción humano-máquina con recursos limitados [20, 21].

Recursos Humanos y Competencias La implementación requiere competencias básicas en:

- Electrónica analógica (acondicionamiento de señales)
- Procesamiento digital de señales (filtrado, extracción de características)
- Programación en Python (aprendizaje de máquinas) y C/C++ (Arduino)
- Control de actuadores (PWM, servomotores)

Infraestructura Requerida El desarrollo del proyecto se caracteriza por su alta accesibilidad económica y metodológica, requiriendo únicamente una infraestructura de soporte computacional y de espacio básica compuesta por:

- **Espacio de trabajo estándar:** Mesa de trabajo despejada con suministro eléctrico regulado para la alimentación de la etapa de actuación robótica y la placa de desarrollo.
- **Computador de desarrollo y procesamiento base:** Estación de trabajo estándar utilizada para la programación del firmware en C++, el entrenamiento del regresor *Random Forest* y la ejecución en tiempo real de la interfaz analítica en Python (especificaciones mínimas: 4 GB de memoria RAM, procesador dual-core y un puerto USB disponible) [19]. Esta configuración es suficiente para manejar arquitecturas que, mediante el uso de bosques profundos (*Deep Forests*), logran una robustez superior frente al ruido y la corrupción de canales en sensores de alta densidad (HDsEMG), reduciendo el error en más del 50 % en comparación con modelos base [19]. Asimismo, el uso de FPGAs o estaciones de trabajo dedicadas permite paralelizar estos procesos, garantizando latencias dentro de los rangos óptimos de 125-150 ms para el control de prótesis [21].
- **Componentes de alimentación y conectividad de bajo costo:** Cableado USB estándar para la telemetría UART y una fuente de poder regulada comercial de 5 V para energizar la etapa de potencia, cuyo correcto acoplamiento de masa común (GND) y consistencia eléctrica se validan de forma directa mediante la inspección visual de las conexiones físicas y el monitoreo lógico de los mensajes de estado y telemetría serie en la terminal de la PC host [14, 22].

Escalabilidad y Extensibilidad La arquitectura modular del sistema permite futuras integraciones:

- Adición de canales EMG adicionales (escalamiento a configuración multicanal)
- Integración de sensores complementarios (IMU para reducción de artefactos)
- Implementación de arquitecturas de aprendizaje profundo si el hardware se escala
- Expansión del control robótico (pinzas, sistemas de múltiples grados de libertad)

Limitaciones Reconocidas

- **Restricción de Nyquist por multiplexación:** El ADC único del ATmega328P digitaliza los tres canales sEMG de forma alternada, lo que reduce la frecuencia efectiva por músculo a aproximadamente 333 Hz. Esto impone un límite de Nyquist teórico de 166,5 Hz por canal, obligando a situar el corte del filtro digital en 150 Hz para evitar el solapamiento (*aliasing*) de la señal [5].
- **Métricas de inferencia:** El desempeño del sistema se evalúa mediante la **estimación angular continua**, proyectando un error absoluto medio (*MAE*) inferior a 15° y un coeficiente de determinación R^2 significativo, métricas consistentes con la regresión de variables cinemáticas a partir de señales sEMG de naturaleza estocástica [3].
- **Dependencia de procesamiento externo:** Debido a la carga computacional de la arquitectura jerárquica (200 árboles de decisión en total), el sistema no es autónomo (*standalone*), requiriendo una conexión serial activa con una PC para ejecutar el procesamiento pesado y la inferencia [14].
- **Sensibilidad a la instrumentación:** La ausencia de un filtro *notch* de hardware y la resolución de 10 bits del ADC (piso de cuantización de 4,9 mV) supeditan la calidad de la señal a una correcta colocación de los electrodos y a la estabilidad de la impedancia de contacto piel-gel [4].
- **Variabilidad fisiológica:** El sistema requiere una fase de calibración obligatoria por sesión y por usuario para normalizar los valores de %MVC, mitigando la variabilidad inter-sujeto y la fatiga muscular acumulada [12].

Conclusión de Viabilidad Técnica El análisis demuestra que el proyecto es **técnicamente viable** con los recursos propuestos, siempre que se respeten las limitaciones identificadas y se implementen las estrategias de mitigación establecidas en el Modelo en Espiral. La reducción del período a 28 semanas (Enero a Julio) requiere una gestión eficiente del tiempo pero sigue siendo realista dado el alcance definido.

2.5.2. Viabilidad Operativa

El sistema puede ser utilizado en entornos experimentales con mínimos requerimientos operativos, siempre que se garantice una correcta colocación de electrodos y condiciones controladas de prueba.

Costos de Desarrollo

La inversión inicial del proyecto se concentra en hardware especializado, mientras que el software utiliza exclusivamente herramientas de código abierto, eliminando costos de licenciamiento.

Software 100 % código abierto (Python, NumPy, SciPy, Scikit-learn, Arduino IDE):

- Costo de licencias: **\$0 CLP**

Hardware e Infraestructura La cuantificación de la inversión inicial se realizó mediante cotizaciones específicas en el mercado nacional de componentes electrónicos [23,24]. Los costos totales de hardware ascienden a **\$322.465 CLP**, como se detalla en la Tabla 2.5.

Ítem / Componente	Cantidad	Unidad	P. Unitario (CLP)	Subtotal (CLP) + IVA
Brazo robótico articulado (6 GDL)	1	Unidad	\$169.000	\$169.000
Kit de sensores sEMG analógicos	3	Unidad	\$37.995	\$113.985
Placa de desarrollo Arduino Uno R3	1	Unidad	\$9.990	\$9.990
Fuente de poder regulada (5V / 10A)	1	Unidad	\$7.990	\$7.990
Módulo controlador PWM PCA9685 I2C	1	Unidad	\$5.900	\$5.900
Baterías de respaldo 9V	6	Unidad	\$2.600	\$15.600
Total Hardware Base				\$322.465
Margen de Contingencia (2,53 %)				\$8.143
TOTAL INVERSIÓN HARDWARE				\$330.608

Tabla 2.5: Costos de hardware e infraestructura (inversión inicial)

Análisis de Viabilidad Económica: El costo total proyectado (\$332.710 CLP $\approx$ \$335 USD) es comparable al umbral de \$350 USD establecido en la literatura como límite para sistemas de bajo costo, manteniéndose dentro del rango de accesibilidad para contextos de prototipado experimental [3].

2.5.3. Viabilidad de Plazos (Cronograma)

2.6. Metodología de Desarrollo

Este proyecto adopta el **Modelo en Espiral** como metodología de desarrollo, debido a su capacidad para gestionar la incertidumbre técnica inherente a sistemas que integran hardware, procesamiento de señales en tiempo real y control robótico.

El Modelo en Espiral se fundamenta en ciclos iterativos que atraviesan cuatro fases principales:

1. **Planificación:** Definición de objetivos y restricciones del ciclo
2. **Análisis de Riesgos:** Identificación y evaluación de riesgos técnicos
3. **Ingeniería:** Desarrollo e implementación del prototipo
4. **Evaluación:** Evaluación mediante pruebas experimentales

2.6.1. Justificación de la Metodología

La elección del Modelo en Espiral se fundamenta en tres características críticas del proyecto:

Incertidumbre técnica elevada: Los objetivos cuantitativos establecidos (SNR >20 dB, latencia <200 ms, precisión >85 %) no tienen garantía de cumplimiento con hardware de bajo costo hasta realizar evaluación experimental. El modelo permite ajustar componentes o parámetros en ciclos posteriores sin invalidar el trabajo previo.

Integración hardware-software compleja: El sistema requiere sincronización entre adquisición analógica, procesamiento digital y control de actuadores. El desarrollo iterativo permite identificar cuellos de botella y optimizar cada subsistema progresivamente.

Evaluación continua requerida: Los criterios de aceptación técnicos deben verificarse experimentalmente en cada etapa. Las espirales permiten implementar estrategias de mitigación ante desviaciones detectadas tempranamente.

2.6.2. Estructura de Desarrollo en Espirales

El proyecto se estructura en **cinco espirales iterativas**, cada una con objetivos técnicos específicos, criterios de evaluación cuantitativos y estrategias de contingencia documentadas. La Tabla 2.6 presenta el resumen completo de cada ciclo, incluyendo los umbrales de aceptación y las acciones de mitigación asociadas.

Espiral	Objetivo Principal	Criterio de Éxito	Estrategia si Falla	Sem.
ESP 1	Evaluar adquisición EMG estable (OE1)	SNR >20 dB en 80 % mediciones	Ajuste de amplificación o cambio de electrodos	6
ESP 2	<i>Pipeline</i> procesamiento funcional (OE2-F1)	Latencia <50 ms procesamiento digital	Optimizar filtros o reducir orden Butterworth	6
ESP 3	Integración del regresor + control continuo 1 GDL (OE2-F2 + OE3-F1)	MAE <15°, Latencia E2E <200 ms	Ampliar dataset de ángulos intermedios o ajustar hiperparámetros del regresor	7
ESP 4	Sistema 2 GDL completo con trayectorias (OE3-F2)	Oscilaciones <5°, movimientos coordinados	Ajuste parámetros interpolación spline	6
ESP 5	Evaluación técnica y con usuarios (OE4)	SUS >70, degradación <15 % en 30 min	Refinamiento UI, recalibración adaptativa del estimador	5
DURACIÓN TOTAL:				28 sem.

Tabla 2.6: Resumen de las cinco espirales del proyecto, criterios de evaluación cuantitativos y estrategias de contingencia.

Interpretación de la tabla:

- **Columna “Sem.”:** Indica semanas base por espiral (duración sin buffers)
- **Criterios de Éxito:** Son cuantitativos y medibles, establecidos según la literatura del estado del arte
- **Estrategias de Contingencia:** Se activan solo si no se cumplen los criterios, documentando la decisión para el informe final

2.6.3. Ventajas del Modelo en Espiral

- **Reducción de riesgo técnico:** Cada ciclo evalúa supuestos antes de invertir esfuerzo en componentes dependientes
- **Flexibilidad ante hallazgos experimentales:** Permite ajustar hardware, hiperparámetros del modelo o métricas según resultados reales

- **Prototipado incremental:** Cada espiral produce un sistema funcional de complejidad creciente
- **Documentación de decisiones:** Las estrategias de mitigación quedan documentadas para el informe final

2.6.4. Gestión de Riesgos y Contingencias

La gestión de riesgos se integra dentro de cada espiral, con estrategias de mitigación específicas que se activan inmediatamente al detectar desviaciones. La Tabla 2.7 detalla las principales contingencias y sus acciones correctivas.

Contingencia	Condición de Activación	Acciones de Mitigación
Contingencia 1	SNR < 20 dB en Espiral 1	Cambio de electrodos gelificados, ajuste de etapa de amplificación, o implementación de filtrado adaptativo
Contingencia 2	Latencia > 50 ms en Espiral 2	Optimización de filtros digitales, reducción de orden, o uso de filtros IIR más eficientes
Contingencia 3	MAE > 15° en Espiral 3	Optimización de hiperparámetros (estimadores/profundidad) en el modelo de regresión <i>Random Forest</i> , o refinamiento de los parámetros de la ventana de extracción de características

Tabla 2.7: Contingencias principales y estrategias de mitigación asociadas.

Rutas del proyecto:

- **Ruta óptima:** 28 semanas de duración total (Enero a Julio)
- **Margen de seguridad:** 2 semanas incorporadas en cada espiral para ajustes menores
- **Enfoque proactivo:** Las contingencias se resuelven dentro de la espiral correspondiente

Capítulo 3

Descripción General del Sistema

El sistema desarrollado en este proyecto corresponde a un brazo robótico de 6 GDL mecánicos, de los cuales 2 son controlados mediante una interfaz sEMG que procesa señales de los músculos bíceps braquial, tríceps braquial y pronador redondo.

La arquitectura se basa en un modelo distribuido donde el microcontrolador Arduino Uno (ATmega328P) opera exclusivamente como un puente de adquisición de datos (DAQ) y pasarela de control mecánico. El dispositivo digitaliza las tres señales analógicas a una frecuencia de muestreo total de 1000 Hz y transmite las tramas en formato crudo a una estación de cómputo externa mediante comunicación serial UART a 115200 baud. En la PC host, un módulo en Python centraliza el procesamiento pesado, aplicando un filtro Butterworth digital de orden 4 con una frecuencia de corte superior ajustada a 150 Hz para respetar el límite estricto de Nyquist por canal y evitar el solapamiento de señales. Posteriormente, se gestionan ventanas deslizantes de 250 ms para extraer un vector de características compuesto por RMS, MAV, WL y ZCR, el cual alimenta un pipeline basado en Random Forest. Esta arquitectura de procesamiento emplea un modelo de regresión cinemática para mapear de forma no lineal los patrones de activación muscular a una estimación angular continua de la trayectoria. Finalmente, las coordenadas calculadas son devueltas al microcontrolador, el cual actúa como puente I²C hacia un controlador PWM PCA9685. Para garantizar la fluidez del movimiento y proteger los engranajes de los servomotores KS-3518, el sistema aplica un limitador de tasa (slew-rate) de 4,8°/ciclo y un filtro de histéresis temporal de 2 grados, el cual suaviza fluctuaciones por ruido, a través del filtro EMA y banda muerta, manteniendo la latencia extremo a extremo por debajo del umbral crítico de los 250 ms.

3.1. Perspectiva del Producto

El sistema desarrollado corresponde a un prototipo de control robótico distribuido que no depende de plataformas comerciales cerradas. Se integra en una arquitectura híbrida compuesta por un nodo de adquisición (Arduino Uno), una estación de cómputo host (PC con Python) y una etapa de actuación mecánica (PCA9685 y servomotores KS-3518).

A diferencia de los sistemas de bajo costo que se limitan a un control binario o proporcional simple ejecutado localmente, este producto aprovecha un esquema de *offload* computacional. El microcontrolador ATmega328P actúa como un puente de adquisición de datos (DAQ) de alta velocidad, digitalizando tres canales analógicos (bíceps braquial, tríceps braquial y pronador redondo) a una tasa consolidada de 1000 Hz .

La inteligencia del sistema reside en el host, donde un entorno en Python gestiona un filtro digital Butterworth de 150 Hz para respetar el límite de Nyquist por canal, extrae un vector de características (RMS, MAV, WL y ZCR) y ejecuta un modelo de regresión cinemática basado en *Random Forest*. Esta configuración permite que el sistema estime de forma continua e interpolada la posición angular del brazo, enviando las directrices de regreso al Arduino mediante comunicación serial UART bidireccional a 115200 baud para el control coordinado de los actuadores

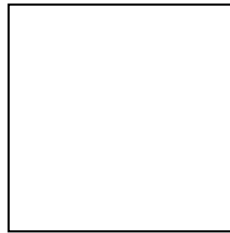


Figura 3.1: Diagrama de Bloques Físico y de Conexiones del Sistema

3.2. Funciones del Sistema

Las funciones principales del sistema, bajo un modelo de procesamiento distribuido (*offload*), son las siguientes:

1. **Adquisición y digitalización remota (DAQ):** Tres módulos EMG capturan la actividad del bíceps braquial (A0), tríceps braquial (A1) y pronador redondo (A2). El ATmega328P digitaliza estas señales de forma alternada a una tasa consolidada de 1000 Hz y las transmite de forma inmediata en formato crudo a la PC host mediante UART a 115200 baud .
2. **Procesamiento Digital de Señales (PC):** En la estación host, se aplica a cada canal un filtro digital IIR Butterworth pasabanda de orden 4. La frecuencia de corte superior se ajusta estrictamente a 150 Hz para respetar el límite de Nyquist efectivo por canal y evitar el solapamiento de señales .
3. **Extracción del vector de características:** Sobre ventanas deslizantes de 250 ms actualizadas cada 20 ms, se calcula un vector compuesto por cuatro métricas temporales por canal: RMS, MAV, WL y ZCR, proporcionando la base de datos para la inferencia inteligente.

4. **Estimación angular continua:** El sistema emplea un *pipeline* basado en **Random Forest** operando en **modo regresión**. El modelo estima e interpola de forma no lineal la **posición angular continua** del actuador entre 0° y 180° a partir de los patrones de activación de los canales instrumentados.
5. **Normalización dinámica (%MVC):** El sistema normaliza la intensidad de la señal respecto a la calibración de la sesión actual (baseline y MVC por músculo), permitiendo que el regresor estime el ángulo de forma independiente a la variabilidad fisiológica del usuario.
6. **Gestión de estabilidad y chattering:** Las estimaciones angulares se suavizan mediante un filtro de promedio móvil o exponencial, eliminando fluctuaciones por ruido transitorio antes de enviar el comando al brazo.
7. **Limitación cinemática de tasa:** Para proteger los engranajes del servo KS-3518, el desplazamiento máximo se restringe por software a $4,8^\circ/\text{ciclo}$, asegurando que la velocidad angular no supere los $240^\circ/\text{s}$ (80 % de la velocidad nominal del actuador).
8. **Protocolo de Medición de Latencia E2E** Para cuantificar la latencia electrónica extremo-a-extremo, el firmware implementa un protocolo ACK loopback:
9. **Despacho de control (Arduino):** El microcontrolador recibe la directriz angular de la PC y actúa como pasarela I²C hacia el controlador PCA9685, el cual genera los pulsos PWM de alta resolución (12 bits) para el posicionamiento físico de los actuadores .

3.3. Características de los Usuarios

El sistema contempla un único perfil de usuario directo durante la fase de prototipo experimental:

Operador técnico: Persona con conocimientos básicos en electrónica embebida, familiaridad con la plataforma Arduino y, fundamentalmente, capacidad para gestionar entornos de ejecución en *Python*. Sus responsabilidades y competencias incluyen:

- **Instrumentación:** Capacidad para la colocación de electrodos superficiales siguiendo el protocolo de los 3 canales (bíceps braquial, tríceps braquial y pronador redondo) y conexión física de la etapa de potencia.
- **Gestión de Datos:** Ejecución de rutinas de captura de señales mediante el script `data/captura.py` y supervisión del entrenamiento del modelo de regresión *Random Forest* a través de `training/train.py`.
- **Análisis de Desempeño:** Capacidad para validar la viabilidad de la estimación continua analizando el error *MAE* (Error Absoluto Medio) y el coeficiente de determinación R^2 en el modelo de regresión antes de iniciar la operación en tiempo real

- **Operación y Monitoreo:** Ejecución del orquestador principal (`main.py`) e interpretación de la telemetría cinemática centrada en la **estimación angular continua** y el seguimiento fluido de la trayectoria del brazo robótico.

No se requiere conocimiento profundo en el diseño algorítmico de los filtros IIR o de la arquitectura del bosque aleatorio, dado que los procesos de extracción de características (RMS, MAV, WL y ZCR) y la estimación angular continua están automatizados dentro del *pipeline* de software. En etapas futuras, la integración de una interfaz gráfica (GUI) permitiría extender el perfil a usuarios finales o pacientes sin formación técnica previa.

3.4. Restricciones

El desarrollo y operación del prototipo están sujetos a las siguientes limitaciones técnicas y económicas:

- **Presupuesto:** El costo total de los componentes de hardware debe mantenerse por debajo del umbral de 350 USD para garantizar la accesibilidad de la plataforma en entornos académicos.
- **Capacidad de Cómputo Local:** El microcontrolador ATmega328P cuenta con una memoria SRAM limitada de solo 2 KB y carece de una unidad de punto flotante (FPU), lo que restringe el procesamiento pesado de señales en el dispositivo.
- **Dependencia de Host:** Debido a la complejidad del modelo de **regresión cinemática** (200 árboles de decisión), el sistema requiere obligatoriamente de una PC externa para realizar la inferencia en tiempo real, superando las restricciones de memoria SRAM (2 KB) y la ausencia de unidad de punto flotante (FPU) del microcontrolador ATmega328P.
- **Límite de Nyquist:** Al distribuir el muestreo de 1000 Hz entre 3 canales analógicos, la frecuencia de corte del filtro digital no puede superar los 150 Hz para evitar el solapamiento de señales.
- **Dinámica Mecánica:** El control de los actuadores está limitado por la velocidad nominal de los servomotores KS-3518, restringiendo los incrementos angulares a $4,8^\circ$ por ciclo para proteger los engranajes.

3.5. Suposiciones y Dependencias

3.5.1. Suposiciones

Para el correcto funcionamiento del sistema, se consideran verdaderas las siguientes condiciones:

- Se asume que el usuario mantiene un contacto galvánico estable entre los electrodos y la piel, minimizando los cambios en la impedancia de contacto durante la sesión.
- Se considera que la comunicación serial UART a 115200 baud es lo suficientemente estable para transmitir tramas de datos cada 1 ms sin pérdidas críticas de información.
- Se asume que los patrones musculares capturados durante la fase de captura son representativos de la intención motora del usuario para el entrenamiento del modelo.

3.5.2. Dependencias

La ejecución íntegra del proyecto depende de los siguientes factores externos:

- **Entorno de Software:** El sistema requiere de una estación de trabajo con Python 3.8 o superior y las librerías *scikit-learn* y *joblib* para la gestión de los modelos Random Forest.
- **Firmware y Librerías:** La operación del hardware depende de la disponibilidad de las bibliotecas `Wire.h` y `Adafruit_PWMServoDriver` dentro del ecosistema Arduino.
- **Instrumentación:** El sistema depende de la correcta conexión de los 3 canales analógicos (A0, A1 y A2) correspondientes al bíceps braquial, tríceps braquial y pronador redondo para permitir la **estimación angular continua** y el mapeo fluido de la trayectoria del actuador en todo su rango dinámico.

Capítulo 4

Requisitos del Proyecto

Este capítulo especifica los requisitos del sistema de control robótico basado en señales electromiográficas de superficie. Los requisitos se derivan de las funciones descritas en el Capítulo 3 y de las restricciones impuestas por la plataforma de hardware seleccionada.

4.1. Requisitos de Interfaces Externas

4.1.1. Interfaces de Usuario

La interacción del operador con el sistema se realiza exclusivamente a través del puerto serie UART a 115.200 baudios [14]. Esta tasa de transferencia es crítica para mitigar la baja capacidad de cómputo del microcontrolador al delegar el procesamiento pesado al host, superando la restricción de 2 KB de SRAM del hardware embebido [14]. El sistema despliega mensajes de estado durante la calibración. (instrucciones de fase, cuenta regresiva, resultados de baseline y MVC) y emite de forma síncrona los datos de operación, las tres activaciones musculares normalizadas y los dos ángulos articulares estimados, en formato `etiqueta:valor` para ser visualizados y registrados directamente en la terminal de la PC host. El operador puede iniciar una recalibración en cualquier momento enviando el carácter `c` o `C` desde el teclado de la estación de trabajo. No existe interfaz gráfica, pantalla local ni teclado físico asociados al sistema.

4.1.2. Interfaces de Hardware

Las interfaces físicas del sistema son las siguientes:

- **Módulos EMG (×3):** Cada módulo entrega una señal analógica de salida en el rango 0–5 V conectada a los pines A0 (bíceps braquial) y A1 (tríceps braquial) y A2 (pronador redondo) del Arduino Uno. La alimentación de los módulos se obtiene de los pines de 5 V y GND del Arduino.

- **PCA9685:** Controlador PWM de 16 canales y resolución de 12 bits, conectado al Arduino Uno mediante el bus I²C (SDA en A4, SCL en A5) con dirección de esclavo 0x40. La alimentación lógica del PCA9685 se obtiene del Arduino (5 V/GND); la alimentación de potencia para el servo (V+) proviene de una fuente externa de 5 V/2 A con masa compartida.
- **Servo KS-3518:** Conectado al canal 0 del PCA9685. Recibe pulsos PWM en el rango 500–2400 μ s a una frecuencia de 50 Hz, correspondientes a posiciones de 0° a 180°.
- **Puerto UART:** Conexión USB-Serie entre el Arduino Uno y el computador de monitoreo, operando a 115200 baud, 8 bits de datos, sin paridad, 1 bit de parada (8N1).

4.1.3. Interfaces de Software

El firmware de adquisición se ejecuta directamente sobre el hardware sin sistema operativo, careciendo de capa de abstracción (HAL) o middleware. Las únicas dependencias en el microcontrolador son las bibliotecas `Wire` y `Adafruit_PWMServoDriver` para la gestión del bus I²C y el driver PCA9685. El procesamiento digital de señales (filtrado IIR Butterworth, extracción de características RMS/MAV/WL/ZC y normalización) junto con la inferencia del modelo *Random Forest* se delegan íntegramente al **módulo de procesamiento en Python sobre la PC host**. Esta arquitectura de *offload* permite superar las restricciones de memoria SRAM del ATmega328P, limitando el rol del microcontrolador a la digitalización síncrona y la ejecución de los comandos de posición recibidos vía UART.

4.1.4. Interfaces de Comunicación

La arquitectura distribuida del sistema establece dos canales de comunicación física activos durante la operación en tiempo real [4]:

- **Canal UART Serial (PC Host – Arduino Uno):** Conexión serial bidireccional configurada a una tasa de transferencia de 115.200 baud, con 8 bits de datos, sin paridad y 1 bit de parada [14]. En el sentido de subida (Arduino → PC), el microcontrolador transmite de forma síncrona y continua cada 1 ms las muestras analógicas crudas digitalizadas de los tres canales musculares. En el sentido de bajada (PC → Arduino), el host Python despacha cada 20 ms una trama serializada con las consignas angulares estimadas para ambos grados de libertad del actuador robótico [4].
- **Bus Físico I²C (Arduino Uno – PCA9685):** Interfaz síncrona de comunicación de bajo nivel utilizada por el microcontrolador para actualizar las directrices de ancho de pulso PWM del driver externo. El bus opera estrictamente en modo rápido (*Fast Mode* a 400 kHz) para minimizar la latencia de conmutación física de los actuadores y resguardar la meta de retardo total de lazo cerrado por debajo de los 200 ms [16, 25].

4.2. Requisitos Funcionales

Los requisitos funcionales del sistema se definen a continuación. Cada requisito describe una capacidad que el firmware debe satisfacer durante la operación.

4.2.1. Adquisición de señal

- **RF-01:** El sistema debe adquirir de forma síncrona las señales EMG del bíceps braquial (A0), tríceps braquial (A1) y pronador redondo (A2) a una frecuencia de muestreo de 1000 Hz mediante interrupción de Timer1, garantizando la entrada de datos necesaria para el modelo de regresión cinemática.
- **RF-02:** El sistema debe transmitir cada muestra del ADC de 10 bits (rango 0–1023) como un entero de 16 bits desde el firmware, realizando la conversión a un valor numérico de punto flotante en la aplicación de software (Python) para su procesamiento posterior.

4.2.2. Procesamiento digital de señal

- **RF-03:** El sistema debe aplicar a cada canal analógico de entrada un pipeline digital de acondicionamiento compuesto por dos etapas lógicas en cascada: (a) un filtro digital IIR Notch centrado estrictamente en 50 Hz (factor de calidad $Q = 30$) para neutralizar el ruido de la red eléctrica comercial, y (b) un filtro digital IIR Butterworth pasabanda de cuarto orden (20–150 Hz) implementado en Forma Directa I. El cálculo de ambas etapas se delega en su totalidad al módulo en Python sobre la PC host, procesando muestra a muestra las lecturas crudas transmitidas por el microcontrolador.
- **RF-04:** El sistema debe calcular el valor RMS de la señal filtrada sobre una ventana deslizando de 83 muestras (250 ms a la frecuencia efectiva de $F_s \approx 333$ Hz por canal, resultante de la multiplexación del ADC a 1000 Hz entre 3 canales), con actualización cada 20 ms, para cada canal de forma independiente [5, 9].
- **RF-05:** El sistema debe normalizar las 12 características extraídas de la señal (incluyendo RMS, MAV, WL y ZCR) de cada canal respecto al rango obtenido durante la calibración (*baseline* y MVC), expresando el nivel de activación en porcentaje de contracción voluntaria máxima (%MVC), con saturación en el intervalo $[0, 100]$ % [5].
- **RF-06:** El sistema debe aplicar un filtro exponencial móvil (EMA) sobre la señal de posición angular estimada, con un coeficiente de suavizado $\alpha = 0,20$ (20 % de peso al nuevo valor y 80 % a la historia), seguido de un filtro de zona muerta (*deadband*). Esta doble etapa tiene como objetivo reducir oscilaciones transitorias (*jitter*) y evitar el envío de micro-cambios irrelevantes a la señal de control.

$$EMA[n] = \alpha \cdot \text{valor_crudo}[n] + (1 - \alpha) \cdot EMA[n - 1] \quad (4.1)$$

Posteriormente, la posición solo se actualizará si la diferencia absoluta entre el valor filtrado ($EMA[n]$) y el último valor enviado supera el umbral del *deadband*.

4.2.3. Estimación cinemática y control continuo del actuador

- **RF-07:** El sistema debe estimar de manera continua y en tiempo real la posición angular independiente de dos grados de libertad activos (codo y muñeca) del brazo robótico en el rango de 0° a 180° [26]. Esta estimación se debe realizar mediante un modelo predictivo basado en *Random Forest Regressor* multi-salida, el cual mapea de forma no lineal las fluctuaciones dinámicas del vector de 12 características temporales provenientes de los canales musculares instrumentados [19]. Este enfoque de regresión permite una decodificación cinemática fluida, superando las limitaciones de los métodos de clasificación discretos y garantizando robustez ante el ruido y la variabilidad de las señales sEMG [20].
- **RF-08:** El sistema debe mapear el nivel de activación muscular normalizado a un ángulo objetivo en el rango $[0, 180]$ mediante interpolación lineal entre los umbrales de activación ($UMBRAL_BAJO = 12\%MVC$, $UMBRAL_ALTO = 90\%MVC$).
- **RF-09:** El sistema debe limitar la variación del ángulo de referencia a un máximo de 4.8° por ciclo de control (20 ms), equivalente al 80 % de la velocidad nominal del servo KS-3518.
- **RF-10:** El sistema debe generar la señal PWM correspondiente al ángulo calculado y transmitirla al canal 0 del PCA9685 mediante I²C en cada ciclo de control de 20 ms.

4.2.4. Calibración

- **RF-11:** El sistema debe ejecutar de forma interactiva una rutina guiada de calibración al inicio de cada sesión experimental para calcular los parámetros de normalización local [27]. La rutina constará de dos etapas: una medición pasiva del nivel de ruido de fondo (*baseline*) con una duración parametrizada de 3 segundos, y una captura secuencial de la Contracción Voluntaria Máxima (MVC) para cada uno de los tres canales analógicos sEMG [5]. Esta última ejecutará 3 intentos de 4 segundos por músculo, intercalando los descansos reglamentarios intra-intento e inter-músculo para mitigar la fatiga muscular.
- **RF-12:** El sistema debe validar, al finalizar la rutina de calibración, que el rango dinámico de cada canal (diferencia entre MVC y *baseline*) sea superior a un umbral mínimo de 10^{-6} . Si la validación falla en cualquiera de los tres canales por falta de activación muscular o desconexión, el sistema debe aplicar una corrección automática forzando el valor de MVC a *baseline* +1,0 para evitar indeterminaciones matemáticas en el pipeline de normalización, permitiendo la continuidad de la sesión sin bloquear la operación.
- **RF-13:** El sistema debe registrar en el archivo de calibración local los parámetros calculados y ajustados post-validación para cada canal analógico, quedando inmediatamente

habilitado el modo de operación en lazo cerrado para la sesión experimental vigente, sin un límite restrictivo de reintentos de calibración por hardware.

- **RF-14:** El sistema debe permitir reiniciar la rutina de calibración en cualquier momento a través del menú interactivo de la interfaz de consola del software principal (`main.py`), sin requerir un ciclo de energía o reinicio de hardware en el microcontrolador.

4.2.5. Monitoreo serial

- **RF-15:** El módulo de visualización en el host (Python) debe procesar la telemetría recibida por UART para registrar y desplegar en la terminal del operador, con una tasa de actualización de 100 ms, el estado síncrono del lazo de control. La información mostrada debe incluir los niveles de activación muscular normalizada (%MVC) de los tres canales y las consignas de posición angular estimadas para ambos servomotores, facilitando el diagnóstico del operador sin interferir con el canal serie de adquisición.

4.3. Requisitos de Rendimiento

- **RR-01:** La rutina de servicio de interrupción (ISR) del Timer1 debe completar la adquisición ADC y el marcado de muestra disponible en un tiempo inferior al período de muestreo de 1 ms, para evitar pérdida de muestras.
- **RR-02:** El pipeline de procesamiento completo por ciclo de control (filtrado IIR, RMS, normalización, filtro asimétrico y limitador de tasa) se ejecuta en el módulo Python sobre la PC host en menos de 20 ms. El ATmega328P solo ejecuta la conversión a cuentas PWM y la transmisión I²C sobre los comandos angulares recibidos por UART.
- **RR-03:** El uso de memoria estática (SRAM) en el microcontrolador ATmega328P no debe superar el 35 % de su capacidad física (2,048 bytes), garantizando un margen operativo superior al 65 % para la pila dinámica de llamadas (*stack*) [14]. Para cumplir esta restricción, el firmware se diseña agnóstico a algoritmos de DSP, transmitiendo muestras analógicas crudas como enteros de 16 bits (`int16_t`) sin mantener buffers de ventana en memoria. Así, el estado volátil local (variables globales de adquisición de 3 canales, consignas de posición y un buffer serie de 24 bytes) demanda únicamente entre 50 y 100 bytes estáticos. El consumo total proyectado, incluyendo el *overhead* de las librerías de control de hardware (`Wire` y `Serial`), se acota a un rango de 450 a 700 bytes (22–34 % de ocupación), delegando el buffer dinámico de ventana circular (≈ 3 KB) al host en Python [34].
- **RR-04:** La latencia total del sistema, definida como el tiempo entre el inicio de una contracción muscular y el inicio del movimiento del servo, debe ser inferior a 400 ms en condiciones de operación nominales. Este margen permite absorber el retraso intrínseco de la ventana de segmentación temporal (250 ms) y el retardo de estabilización del filtro exponencial móvil ($\alpha = 0,20$).

4.4. Requisitos de Desarrollo

- **RD-01:** El firmware se desarrollará en lenguaje C/C++ dentro del entorno Arduino IDE, en estructura multi-archivo `.ino`.
- **RD-02:** Los coeficientes de los filtros IIR (Notch y Butterworth) deben ser calculados dinámicamente durante la inicialización del módulo de procesamiento mediante funciones especializadas de procesamiento digital de señales. Los vectores resultantes para el numerador y el denominador de cada filtro deben almacenarse en atributos de instancia de tipo arreglo numérico, respetando la notación estándar de la ecuación en diferencias donde el coeficiente $a_0 = 1,0$ encabeza el denominador.
- **RD-03:** Las variables compartidas entre la ISR y el bucle principal deben declararse con el calificador `volatile` para prevenir optimizaciones del compilador que invaliden la lectura desde el contexto del bucle principal.

4.5. Requisitos Tecnológicos

- **RT-01:** El microcontrolador utilizado será el ATmega328P integrado en la placa Arduino Uno, operando a 16 MHz con 2 KB de SRAM y 32 KB de memoria Flash.
- **RT-02:** El controlador PWM utilizado será el PCA9685 de NXP Semiconductors, con resolución de 12 bits y frecuencia de oscilador interno de 25 MHz, conectado por I²C.
- **RT-03:** La biblioteca de control del PCA9685 será `Adafruit_PWMServoDriver` (Adafruit Industries), instalable desde el gestor de bibliotecas de Arduino IDE.
- **RT-04:** El actuador del prototipo será el servo KS-3518, con rango de pulso de 500–2400 μ s, ángulo máximo de 180° y velocidad nominal de 300°/s sin carga.
- **RT-05:** Los módulos de adquisición EMG serán módulos genéricos de un canal con salida analógica en rango 0–5 V, uno por grupo muscular (bíceps braquial, tríceps braquial y pronador redondo).
- **RT-06:** El entorno de desarrollo será Arduino IDE con soporte para placas AVR y acceso al registro de periféricos del ATmega328P mediante los encabezados `avr/interrupt.h` para la configuración directa de Timer1 y el ADC en modo libre.

4.6. Seguridad

En el contexto de un sistema embebido de control físico de actuadores, los requisitos de seguridad se orientan a prevenir condiciones de operación que puedan dañar el hardware o causar movimientos no controlados del actuador.

- **RS-01:** El servo debe posicionarse en 0° al inicio de cada rutina de calibración, antes de habilitar el lazo de control, para evitar movimientos bruscos desde posiciones desconocidas.
- **RS-02:** El ángulo de referencia enviado al servo debe estar saturado en el intervalo $[0, 180]$ mediante la función `constrain()` antes de la conversión a cuentas PWM, para no exceder los límites mecánicos del KS-3518.
- **RS-03:** El lazo de control principal no debe procesar muestras EMG ni generar comandos de movimiento hasta que la rutina de calibración haya concluido exitosamente (`variable calibrado == true`).
- **RS-04:** La variación máxima del ángulo por ciclo de control debe estar limitada por el limitador de tasa de variación (`MAX_CAMBIO`), de modo que la velocidad angular del servo no supere el 80 % de su velocidad nominal, para reducir el riesgo de sobrecorriente mecánica ante transiciones abruptas en la señal EMG.
- **RS-05:** La alimentación de potencia del servo debe provenir de una fuente externa al Arduino Uno, con masa común, para evitar caídas de tensión en la lógica del microcontrolador durante la conmutación del actuador.

4.7. Persistencia y Estructura de Datos

El sistema no utiliza un motor de base de datos relacional ni no relacional, lo que evita la sobrecarga computacional de un SGBD transaccional en un entorno de tiempo real [4]. Sin embargo, para resguardar la usabilidad de la plataforma y evitar rutinas de calibración redundantes, se implementa un modelo de persistencia híbrido basado en archivos planos en la estación de cómputo host [12]:

- **Archivo de Calibración** (`calibracion.json`): Almacena de forma persistente en el disco de la PC los parámetros de normalización física calculados durante la sesión (*baseline RMS* y *MVC RMS* independientes para los tres canales analógicos). Esto garantiza la consistencia y la integridad de los datos al cargarlos para el modo de operación en lazo cerrado, un aspecto crítico discutido en estudios recientes sobre la fiabilidad de las señales sEMG [27].
- **Serialización del Regresor** (`modelo_regresor.pkl`): Archivo binario que persiste la estructura de pesos y umbrales de decisión del bosque aleatorio de regresión entrenado, facilitando su carga en memoria en milisegundos mediante la librería *joblib* [20].
- **Buffer de Ventana Deslizante (PC Host)**: Estructura de datos dinámica en formato de arreglo circular en Python que almacena las últimas 250 muestras filtradas por canal ($250 \times 4 \text{ bytes} \times 3 \text{ canales} \approx 3 \text{ KB}$) para el cómputo continuo del valor RMS. Al delegarse este buffer a la memoria RAM de la PC, se neutraliza el riesgo de desborde de pila en los 2 KB de SRAM del microcontrolador ATmega328P [14].

Dado que el sistema se rige bajo un paradigma de procesamiento distribuido e integrado por archivos estructurados, los diagramas de Modelo Entidad-Relación (MER) y Modelo Relacional (MR) de bases de datos tradicionales no aplican a este proyecto.

Capítulo 5

Análisis Económico

Este capítulo cuantifica los costos asociados al desarrollo del prototipo, evalúa la viabilidad económica del proyecto en su contexto académico y proyecta los costos e indicadores de rentabilidad relevantes para una eventual transferencia tecnológica o desarrollo comercial del sistema.

5.1. Estudio de Mercado

5.1.1. Consumidores y análisis de la demanda

Los sistemas de control mioeléctrico tienen aplicación en dos segmentos principales. El primero corresponde al mercado de prótesis de miembro superior, donde la interfaz sEMG constituye el método de control predominante en dispositivos comerciales activos [4]. El segundo corresponde a plataformas de rehabilitación motora y asistencia a personas con limitaciones neuromusculares, segmento en el que los sistemas de bajo costo han ganado presencia como alternativa a soluciones clínicas de alto costo [28].

A nivel global, el mercado de prótesis de miembro superior se valoró en aproximadamente USD 1.100 millones en 2022, con proyecciones de crecimiento anual compuesto superiores al 5 % hacia 2030, impulsadas por el envejecimiento poblacional y la prevalencia de amputaciones de origen vascular y traumático [28]. En América Latina, el acceso a prótesis mioeléctricas comerciales es limitado por sus costos unitarios (USD 10.000–70.000 dependiendo del grado de funcionalidad), lo que abre un segmento de demanda para dispositivos de menor costo fabricados con componentes accesibles [4].

El público objetivo de un sistema derivado del presente prototipo incluye: centros de rehabilitación física, laboratorios universitarios de bioingeniería e ingeniería en rehabilitación, y organizaciones sin fines de lucro orientadas a la fabricación de prótesis de bajo costo mediante impresión 3D y electrónica abierta.

5.1.2. Análisis de la oferta y los precios

Las soluciones comerciales de control mioeléctrico existentes operan en rangos de precio que las hacen inaccesibles para la mayoría de los usuarios en contextos de ingresos medios y bajos. Entre los referentes del mercado se identifican: el sistema *i-Limb* de Ossur (USD 70.000–100.000), el *Michelangelo Hand* de Ottobock (USD 60.000–80.000) y el *bebionic* de Ottobock (USD 15.000–25.000) [29].

En el segmento de bajo costo, plataformas abiertas basadas en Arduino y sensores sEMG genéricos han sido reportadas en la literatura con costos de componentes inferiores a USD 150, con desempeño funcional adecuado para tareas de control de uno a dos grados de libertad [30]. El presente prototipo se ubica en este segmento.

El modelo de distribución más adecuado para un producto derivado de este trabajo sería la publicación como diseño abierto (*open hardware*), con posibilidad de comercialización del kit de componentes ensamblado bajo un modelo de precio unitario en el rango CLP 80.000–150.000, considerando los costos de componentes detallados en la Sección 5.3.

5.1.3. Comercialización

Para una eventual comercialización, los canales de distribución viables incluyen: plataformas de venta en línea de componentes electrónicos, acuerdos con centros de salud y rehabilitación para pruebas piloto, y transferencia tecnológica a través de la Oficina de Transferencia Tecnológica (OTT) de la Universidad de Los Lagos. La estrategia de difusión inicial correspondería a la publicación de los resultados en revistas de acceso abierto y presentación en congresos de ingeniería biomédica.

5.2. Evaluación de Proyectos de Inversión

5.2.1. Criterios de evaluación

La evaluación financiera del proyecto se realiza bajo el supuesto de una producción de 20 unidades anuales destinadas a instituciones de rehabilitación durante un horizonte de evaluación de 3 años. El flujo de caja considera los costos de desarrollo ya incurridos como costos hundidos no recuperables y proyecta únicamente los costos e ingresos operativos a partir de la versión validada del prototipo.

La tasa de descuento utilizada es del 10 % anual, correspondiente a una estimación conservadora del costo de oportunidad del capital para proyectos de innovación tecnológica de bajo riesgo en el contexto universitario chileno.

Los flujos de caja proyectados se presentan en la Tabla 5.1.

Ítem	Año 0	Año 1	Año 2	Año 3
Inversión inicial	-450			
Ingresos (20 u. × CLP 120k)		2.400	2.400	2.640
Costos operativos		-800	-820	-840
Flujo neto	-450	1.600	1.580	1.800

Tabla 5.1: Flujo de caja proyectado (CLP miles)

5.2.2. Valor Actual Neto (VAN)

El VAN del proyecto se calcula mediante la expresión:

$$VAN = -I_0 + \sum_{t=1}^n \frac{FC_t}{(1+r)^t} \quad (5.1)$$

donde I_0 es la inversión inicial, FC_t el flujo de caja neto del período t , $r = 0,10$ la tasa de descuento y $n = 3$ el horizonte de evaluación.

Aplicando los flujos de la Tabla 5.1:

$$VAN = -450 + \frac{1,600}{1,10} + \frac{1,580}{1,21} + \frac{1,800}{1,331} \approx -450 + 1,454 + 1,306 + 1,352 = \text{CLP } 3,662 \text{ miles} \quad (5.2)$$

El VAN positivo indica que el proyecto genera valor por sobre el costo de oportunidad del capital bajo los supuestos establecidos.

5.2.3. Tasa Interna de Retorno (TIR)

La TIR corresponde a la tasa r^* que hace el VAN igual a cero:

$$0 = -450 + \frac{1,600}{(1+r^*)} + \frac{1,580}{(1+r^*)^2} + \frac{1,800}{(1+r^*)^3} \quad (5.3)$$

Resolviendo por interpolación, se obtiene $TIR \approx 356\%$, valor que supera ampliamente la tasa de descuento del 10 %, lo que indica rentabilidad positiva del proyecto bajo el escenario base. Este resultado es consistente con proyectos de prototipo de bajo costo de inversión inicial reducida.

5.2.4. Análisis de riesgo e incertidumbre

Se plantean tres escenarios para evaluar la sensibilidad del VAN ante variaciones en la demanda y los costos:

En el escenario pesimista el VAN continúa siendo positivo, lo que indica que el proyecto es viable económicamente incluso ante reducciones significativas en la demanda o en el precio de venta.

Variable	Pesimista	Base	Optimista
Unidades anuales	10	20	35
Precio unitario (CLP)	90.000	120.000	150.000
Costo unitario (CLP)	50.000	40.000	35.000
VAN (CLP miles)	412	3.662	9.840
TIR (%)	88	356	>500

Tabla 5.2: Análisis de escenarios (CLP miles)

5.3. Viabilidad Económica

5.3.1. Costos de Desarrollo

Personal

El desarrollo del proyecto es realizado por un tesista de *Ingeniería Civil en Informática* durante el período de titulación. La valorización del tiempo de desarrollo se estima considerando una dedicación equivalente a 6 meses a tiempo parcial (20 horas/semana). El valor hora de referencia se fija en **CLP 6.000**, cifra que representa una valoración proporcional (aprox. 50 %) al costo laboral de un especialista *Junior* a tiempo completo.

Esta estimación se fundamenta en los rangos salariales líquidos base reportados para perfiles con menos de 2 años de experiencia en el sector TI chileno, los cuales oscilan entre **CLP 1.800.000** y **CLP 2.500.000** mensuales (para 40 horas/semana), según la *12ª Guía Salarial TI 2024-2025* de IT HUNTERS [31]. De esta manera, el valor hora considerado refleja tanto la condición académica del tesista como el costo de mercado ajustado a una jornada parcial.

Rol	Horas/sem.	Semanas	Valor/hora (CLP)	Total (CLP)
Tesista (desarrollo + documentación)	20	24	6.000	2.880.000
Total personal				2.880.000

Tabla 5.3: Costo de personal de desarrollo

Software

El proyecto utiliza exclusivamente herramientas de software sin costo de licencia para el uso académico: Arduino IDE (licencia LGPL), MATLAB (licencia institucional provista por la Universidad de Los Lagos). El costo de software atribuible al proyecto se considera nulo en términos de desembolso directo.

Herramienta	Licencia	Costo atribuible (CLP)
Arduino IDE	LGPL (gratuita)	0
MATLAB	Institucional (universidad)	0
Total software		0

Tabla 5.4: Costos de software

Hardware e Infraestructura

Las inversiones en hardware e infraestructura requeridas para la construcción del prototipo físico se detallan en la Tabla 5.5. La cuantificación económica se realizó sobre la base de cotizaciones vigentes en el mercado nacional de componentes electrónicos para proyectos de investigación y desarrollo bioingenieril [23, 24].

Ítem / Componente	Cantidad	Unidad	P. Unitario (CLP)	Subtotal (CLP) + IVA
Brazo robótico articulado (6 GDL)	1	Unidad	\$169.000	\$169.000
Kit de sensores sEMG analógicos	3	Unidad	\$37.995	\$113.985
Placa de desarrollo Arduino Uno R3	1	Unidad	\$9.990	\$9.990
Fuente de poder regulada (5V / 10A)	1	Unidad	\$7.990	\$7.990
Módulo controlador PWM PCA9685 I2C	1	Unidad	\$5.900	\$5.900
Baterías de respaldo 9V	6	Unidad	\$2.600	\$15.600
Total Hardware Base				\$322.465
Margen de Contingencia (2,53 %)				\$8.143
TOTAL INVERSIÓN HARDWARE				\$330.608

Tabla 5.5: Costos de hardware e infraestructura del prototipo

El costo total de hardware del prototipo asciende a CLP 332.710, equivalente aproximadamente a USD 335 al tipo de cambio de referencia. Este valor sitúa al sistema dentro del rango de bajo costo reportado para soluciones experimentales equivalentes, manteniéndose por debajo del umbral de USD 350 descrito en la literatura [3, 32].

5.3.2. Costos Operativos (Post-Implementación)

En la etapa de prototipo académico, los costos operativos son mínimos dado que no existe infraestructura de producción ni soporte a usuarios externos. Para una versión productiva destinada a instituciones de rehabilitación, los costos operativos recurrentes estimados son:

- Mantenimiento de firmware y documentación técnica: CLP 200.000/año (equivalente a 33 h/año de trabajo técnico a CLP 6.000/h).
- Soporte técnico a usuarios institucionales: CLP 300.000/año (estimado en 2 h/mes de atención).

- Consumibles (electrodos desechables, cables de reemplazo): CLP 80.000/año por unidad instalada.
- Costos de energía: despreciables dado el consumo del sistema (Arduino Uno: ~ 250 mA a 5 V; servo en reposo: ~ 10 mA).

5.3.3. Retorno de la Inversión (ROI) y Beneficios

Reducción de Costos Directos

La sustitución de módulos sEMG comerciales cerrados de alta gama (cuyo costo supera los USD 380 por canal en estaciones clínicas) por sensores analógicos comerciales de bajo costo (\$37.995 CLP por canal) reduce la inversión del hardware de adquisición en más de un 80 % por unidad, sin degradar la calidad de la señal requerida para la estimación cinemática en el espacio de trabajo definido [3].

Eficiencia de Infraestructura y Offload Computacional

A diferencia de los enfoques tradicionales que exigen actualizar a costosos microcontroladores ARM de 32 bits o computadoras embebidas dedicadas para procesar localmente las señales, la arquitectura híbrida implementada aprovecha un esquema de *offload* computacional. Al delegar el filtrado digital pesado Butterworth de cuarto orden y la inferencia del regresor *Random Forest* (200 árboles) a la PC host mediante el puerto UART, se maximiza la vida útil del microcontrolador ATmega328P de 8 bits operando únicamente como puente DAQ. Esto disminuye drásticamente el costo de infraestructura computacional por unidad robótica instalada [4].

Optimización en la Toma de Decisiones y Calibración

La rutina de calibración interactiva de tres canales desarrollada en la consola de Python permite registrar dinámicamente la línea base de reposo y la contracción voluntaria máxima (%MVC) en menos de 20 s por sesión [12]. Esto evita la necesidad de configurar manualmente parámetros rígidos de control ante cambios fisiológicos del usuario o variaciones en la impedancia de los electrodos superficiales, automatizando el ajuste del lazo de control sin requerir la intervención de un especialista clínico en procesamiento de señales [11].

Reducción de Errores y Protección Mecánica

La integración de un filtro de promedio móvil exponencial y un limitador de tasa de variación angular (*slew-rate* de $4,8^\circ/\text{ciclo}$) aplicados sobre la directriz de posición calculada por el regresor mitiga la propagación del ruido estocástico residual de las señales sEMG hacia los servomotores KS-3518 [16]. Este diseño de software híbrido disminuye los movimientos bruscos no intencionados del brazo mecánico ante artefactos de movimiento o descargas de red de

50 Hz, reduciendo el desgaste y riesgo de rotura de los engranajes metálicos de los actuadores y alargando su ciclo de vida útil sin costos adicionales de mantenimiento de hardware.

El análisis económico del proyecto incluye la evaluación de los costos asociados a la planificación temporal. La Figura 2.1 presenta el diagrama de Gantt con la distribución de actividades por espiral de desarrollo, y la Figura 5.1 presenta el grafo de actividades con duraciones y caminos críticos derivado del análisis CPM sobre el capítulo 2.

El costo total estimado del proyecto, considerando personal, hardware y software, se resume en la Tabla 5.6.

Categoría de Inversión	Costo Asociado (CLP)	Porcentaje de Participación
Personal de Desarrollo (Tesisista)	\$2.880.000	89,70 %
Hardware y Prototipo Físico	\$330.608	10,30 %
Software e Infraestructura Lógica	\$0	0,00 %
COSTO TOTAL ESTIMADO	\$3.210.608	100,00 %

Tabla 5.6: Resumen de costos consolidados del proyecto de título

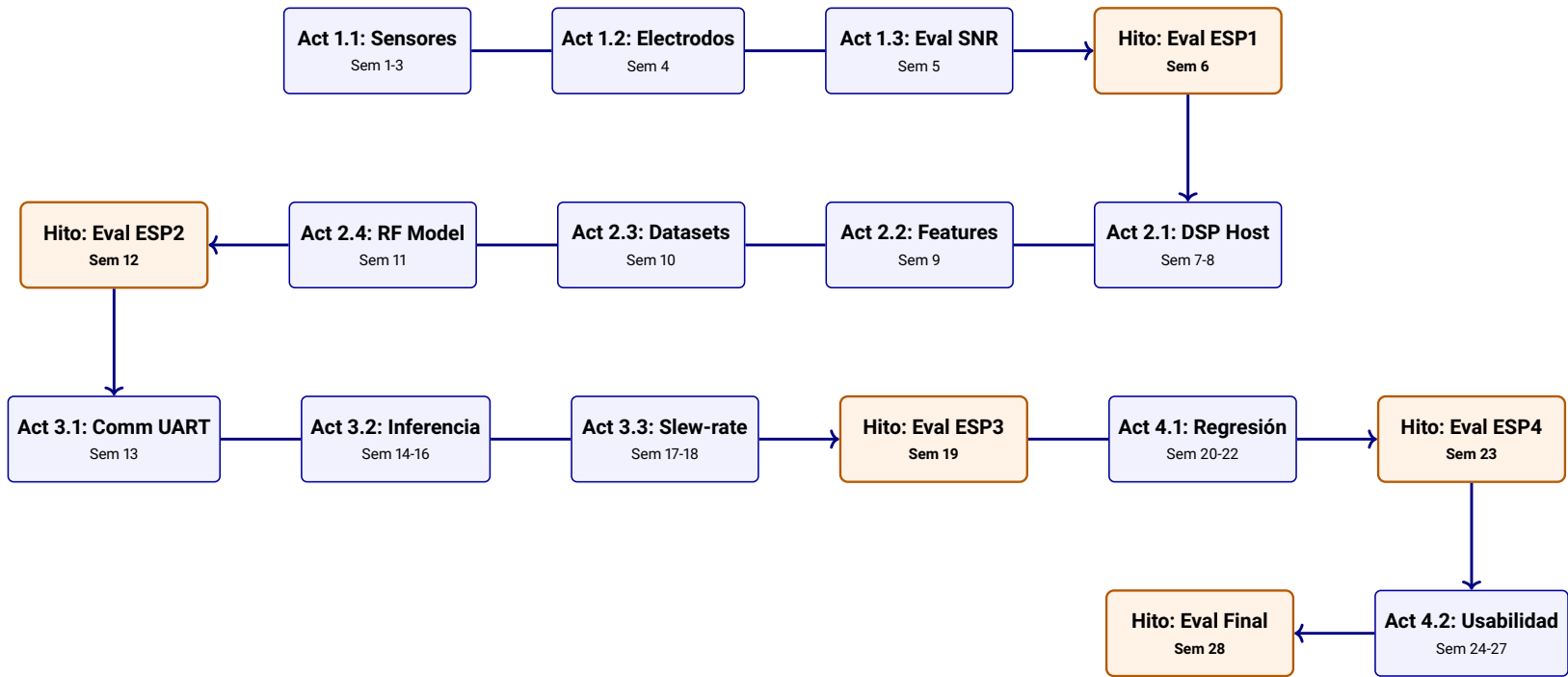


Figura 5.1: Grafo CPM del proyecto. Ruta crítica optimizada para un desarrollo secuencial sin holguras [3].

Capítulo 6

Desarrollo

Este capítulo describe las decisiones de implementación del sistema, el lenguaje de programación utilizado y la arquitectura de la solución desarrollada.

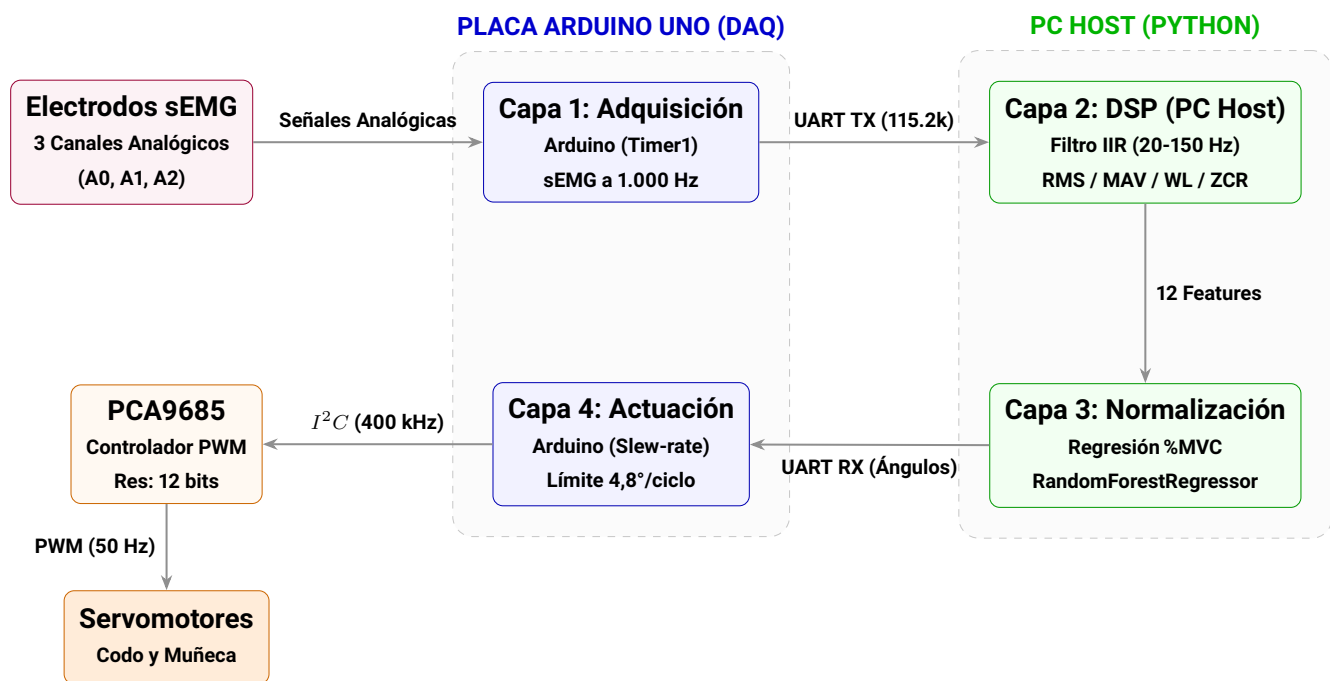


Figura 6.1: Arquitectura de procesamiento distribuido y lazo cerrado de control.

6.1. Arquitectura General del Sistema y Co-diseño Híbrido

El sistema se fundamenta en un esquema de **co-diseño híbrido distribuido** (*PC Offload*), el cual segmenta estrictamente las responsabilidades físicas de adquisición de datos de baja

latencia y el procesamiento matemático pesado de alta complejidad [4,12]. Esta arquitectura de cómputo distribuido se divide en cinco capas funcionales coordinadas que resuelven de forma unificada el lazo cerrado de control continuo, tal como se detalla en el diagrama vectorial de la Figura 6.1.

6.1.1. Justificación Técnica del Offload Computacional

La adopción de una arquitectura de procesamiento distribuido no constituye una decisión arbitraria, sino una restricción física impuesta por los recursos de hardware del microcontrolador ATmega328P de la placa Arduino Uno [7,14].

Para implementar un sistema mioeléctrico con estimación de posición continua, se requiere almacenar en memoria un buffer circular deslizante de las últimas muestras de señal con el propósito de calcular características de amplitud en ventanas temporales estables [5]. En este desarrollo, el cálculo del valor cuadrático medio (RMS) utiliza una ventana deslizante de 250 ms por canal. A una frecuencia de muestreo síncrona de 1.000 Hz distribuida de manera alternada entre los tres canales musculares (bíceps braquial, tríceps braquial y pronador redondo), se obtiene una tasa efectiva de muestreo de aproximadamente 333 Hz por músculo [5].

Esto equivale a almacenar 83 muestras por canal en precisión simple (*single-precision float* de 4 bytes). El espacio de almacenamiento dinámico requerido únicamente por los buffers de historia de señal en la memoria se calcula analíticamente mediante la ecuación:

$$\text{Memoria}_{\text{Buffers}} = 3 \text{ canales} \times 83 \text{ muestras} \times 4 \text{ bytes/muestra} \approx 1,0 \text{ KB} \quad (6.1)$$

El microcontrolador ATmega328P posee una memoria estática de solo **2 KB de SRAM** [14]. En consecuencia, el simple hecho de intentar almacenar en memoria local las variables de historia para la ventana circular del RMS provocaría un desbordamiento inmediato de la pila (*stack overflow*) y el colapso del firmware, inestabilizando por completo la adquisición [15].

A esta limitación física de memoria se suma la **carencia de una unidad de punto flotante (FPU) por hardware** en el ATmega328P de 8 bits [14]. Ejecutar localmente un filtro Butterworth digital de cuarto orden basado en Forma Directa I (DF-I), calcular de manera iterativa el RMS (que involucra multiplicaciones acumuladas y una operación de raíz cuadrada de alto coste de CPU) y procesar en paralelo un modelo predictivo *Random Forest Regressor* multi-salida compuesto por 200 árboles de decisión independientes con variables de entrada reales, demandaría un tiempo de procesamiento superior al ciclo de muestreo crítico de 1 ms [33,34].

Al delegar el pipeline DSP de señales, la calibración interactiva de sesión y la inferencia inteligente al host en Python (orquestrado mediante librerías optimizadas en C como NumPy, SciPy y Scikit-learn), se resguarda la estabilidad del microcontrolador. El Arduino Uno se reduce a un procesador liviano libre de asignación dinámica de memoria y sin cálculo flotante, cuya CPU de 8 bits opera de forma óptima a un 16 MHz dedicándose con total holgura a la digitalización síncrona y la administración del bus de control mecánico I^2C [4,14].

6.1.2. Descripción de las Capas Funcionales de la Arquitectura

La unificación conceptual del lazo cerrado de control se gestiona de forma secuencial a través de las cinco capas del sistema distribuido:

1. **Capa de Adquisición sEMG:** Representa el puente físico de digitalización y transmisión. Los tres amplificadores analógicos de superficie sEMG se conectan a los pines analógicos A0, A1 y A2 de la placa embebida [5]. La rutina de servicio de interrupción (ISR) síncrona, controlada por hardware mediante el registro de comparación del Timer1, conmuta el multiplexor del convertidor ADC de 10 bits secuencialmente cada 1 ms [14]. El firmware empaqueta de forma directa las tres cuentas binarias en formato binario estructurado comprimido y las transmite inmediatamente vía UART a una tasa estable de 115.200 baudios hacia el host [14].
2. **Capa DSP (PC Host):** El software en Python recibe la trama de subida de datos por el puerto serie virtual y la inyecta de forma asíncrona en el buffer circular alojado en la RAM de la PC base [12]. A continuación, aplica de forma secuencial sobre cada canal el filtro IIR Butterworth de cuarto orden pasabanda (20–150 Hz) para eliminar el ruido de fatiga de baja frecuencia y el ruido de alta frecuencia del entorno biomédico, alineándose con las metodologías actuales de acondicionamiento de señales y robustez en sistemas embebidos de control reportadas en [35]. **Extracción de características:** Una vez filtrada la señal sEMG, el host extrae muestra a muestra un vector de características de 12 dimensiones, el cual captura las dinámicas temporales locales del músculo: RMS, Valor Absoluto Medio (MAV), Longitud de Onda (WL) y Cruces por Cero (ZCR). Estos descriptores se calculan en ventanas deslizantes de 250 ms, una configuración que permite equilibrar la latencia del sistema con la precisión necesaria para la estimación estadística, cumpliendo con los estándares de implementación en tiempo real para prótesis biónicas discutidos en [36].
3. **Capa de Normalización y Regresión (PC Host):** El script en Python normaliza de forma lineal el RMS de cada canal respecto a los parámetros dinámicos (*baseline* y %MVC) guardados en el archivo `calibracion.json` al inicio de la sesión del usuario para mitigar la variabilidad de impedancia inter-sesión [37]. Posteriormente, el vector normalizado se procesa mediante un `RandomForestRegressor` multi-salida. Esta arquitectura ha demostrado ser no solo robusta frente a la naturaleza no lineal y estocástica de las señales electromiográficas [36], sino también altamente escalable, permitiendo reducir el tamaño del modelo hasta en un factor de $5\times$ manteniendo la precisión y optimizando el rendimiento incluso con alta densidad de electrodos [38].

El regresor resuelve de forma no lineal los patrones de activación muscular duales para estimar en paralelo la posición cinemática de ambos grados de libertad (codo y muñeca) en el espacio real $[0^\circ, 180^\circ]$ [18]. La directriz estimada se procesa con un filtro EMA simétrico para atenuar oscilaciones espurias y se despacha por la telemetría UART de bajada cada 20 ms [4].

Adicionalmente, se incorpora un mecanismo de **banda muerta** (*deadband*) de $2,0^\circ$ sobre la salida del filtro EMA: una nueva consigna solo se transmite al Arduino si su diferencia respecto a la última consigna efectivamente enviada supera este umbral. Este mecanismo reduce el tráfico UART innecesario ante fluctuaciones residuales de muy baja amplitud que persisten después del suavizado exponencial, sin introducir retardo adicional perceptible en la respuesta dinámica del sistema.

4. **Capa de Actuación y Filtrado (Arduino):** La rutina serial del microcontrolador captura la trama de bajada con las dos consignas angulares estimadas [14]. Para resguardar la vida útil de los engranajes metálicos de los actuadores, el firmware aplica de forma local en C++ un limitador de tasa de cambio (*slew-rate*) ajustado a $4,8^\circ$ por ciclo operativo, bloqueando transiciones instantáneas violentas [16]. Los ángulos suavizados se transforman linealmente a cuentas de control digital de 12 bits de resolución [16].
5. **Capa de Salida Física:** Las cuentas lógicas de actuación se transmiten vía bus físico síncrono I^2C configurado en modo rápido a 400 kHz hacia el controlador PWM externo PCA9685 [25]. El driver descentralizado genera las portadoras PWM independientes de 50 Hz requeridas para accionar físicamente los servomotores KS-3518 del brazo robótico articulado, completando de esta forma el lazo de control continuo de extremo a extremo [16].

6.2. Implementación del Hardware de Adquisición (Arduino Uno - C++)

El microcontrolador ATmega328P de la placa Arduino Uno actúa como el **punteo síncrono de adquisición de datos** (*DAQ bridge*) de ultra-baja latencia del sistema [14]. Su responsabilidad física exclusiva radica en digitalizar periódicamente las diferencias de potencial analógicas provenientes de los tres amplificadores sEMG de superficie y transmitir las tramas en formato binario estructurado comprimido hacia la PC host mediante el puerto serie virtual UART [4].

6.2.1. Justificación de la Programación a Bajo Nivel sin Abstracciones de Arduino

En aplicaciones bioingenieriles de lazo cerrado, el procesamiento digital de señales y los algoritmos de estimación de patrones exigen que las muestras se capturen a una **frecuencia de muestreo rigurosamente constante** para prevenir la distorsión del espectro de frecuencias útil [39]. En este desarrollo, la multiplexación secuencial de los tres canales musculares instrumentados requiere una conmutación limpia del convertidor analógico-digital (ADC) cada 1,0 ms (1.000 Hz totales) [5].

La utilización de las funciones nativas del entorno Arduino IDE presenta dos limitaciones técnicas críticas que hacen inviable su despliegue en el lazo cerrado de control [34]:

- **Bloqueo por Polling:** La función `analogRead()` ejecuta un bucle de espera bloqueante durante la fase de conversión física del ADC (la cual consume entre 110 y 125 μs). Durante este intervalo, la CPU interrumpe cualquier otra tarea, impidiendo la lectura concurrente de tramas UART de bajada provenientes del PC [14].
- **Incertidumbre Temporal (Jitter):** El uso del lazo secuencial `void loop()` gobernado por retardos lógicos tipo `delayMicroseconds()` introduce un desfase acumulativo variable. Esto es provocado por la sobrecarga del planificador de software y las interrupciones del sistema, inestabilizando la tasa de muestreo y degradando la precisión temporal del pipeline DSP en el host [34].

Para neutralizar estas restricciones, el firmware se desarrolla directamente en **C++ estándar** accediendo a los registros internos del microcontrolador ATmega328P [14]. Al configurar el temporizador de hardware **Timer1** en modo de comparación de coincidencia transparente (*Clear Timer on Compare Match* o CTC) y programar la lógica del convertidor ADC mediante manipulación directa de registros de control, se garantiza que la digitalización y el despacho de telemetría se ejecuten en una rutina síncrona determinista, con un desfase temporal nulo [34].

6.2.2. Configuración de Registros del Temporizador Timer1 y ADC

La conmutación síncrona a 1.000 Hz se gobierna mediante la configuración del Timer1 de 16 bits en modo CTC [14]. Con una frecuencia de oscilador por cristal de cuarzo de 16 MHz y seleccionando un divisor de frecuencia (*prescaler*) de 8, el registro de comparación de salida del canal A (OCR1A) se calcula analíticamente mediante la ecuación:

$$\text{OCR1A} = \frac{F_{\text{CPU}}}{\text{Prescaler} \times F_{\text{muestreo}}} - 1 = \frac{16,000,000}{8 \times 1,000} - 1 = 1,999 \quad (6.2)$$

Al registrar el valor decimal exacto de 1.999 en el registro de comparación OCR1A, el hardware del temporizador borra automáticamente su contador y dispara la rutina de servicio de interrupción (ISR) `ISR(TIMER1_COMPA_vect)` con una precisión de $\pm 0,1\%$, independientemente de las instrucciones de software que esté ejecutando la CPU en ese instante [14].

Por otro lado, el registro de inicialización del ADC se configura seleccionando una referencia de tensión externa estable de 5 V regulados (REFS0 en el registro ADMUX) y un factor de división de reloj de 64 para el conversor (ADPS2 y ADPS1 en ADCSRA) [14]. Esto proporciona una frecuencia de trabajo del ADC de 250 kHz (16 MHz/64). Una conversión estándar toma 13 ciclos de reloj del ADC, lo que equivale a un tiempo de conversión física exacto de 52 μs , cumpliendo holgadamente el límite de muestreo del lazo [14].

6.2.3. Estructura del Firmware y Rutina de Servicio de Interrupción (ISR)

El listado de código C++ detallado a continuación presenta la inicialización de registros en el bloque `setup()` y la gestión secuencial del multiplexor de entrada del conversor dentro de la rutina ISR.

La rutina conmuta dinámicamente sobre la máscara del registro ADMUX de forma secuencial en cada tic de interrupción para digitalizar el bíceps braquial (A0), tríceps braquial (A1) y pronador redondo (A2) de forma alternada, despachando inmediatamente la trama por UART a 115.200 baudios. El listado completo del firmware se presenta en el Anexo C.

6.2.4. Mapeo Físico de Conexiones del Nodo de Adquisición

Para garantizar la reproducibilidad técnica del prototipo, se prescinde de la instrumentación avanzada de laboratorio, validando el correcto acoplamiento galvánico de la masa común (GND) y los pines de datos del sistema mediante una inspección sistemática del cableado lógico y de potencia. Las conexiones físicas de entrada, los canales de alimentación compartidos y los buses de control mecánico del Arduino Uno se documentan en la Tabla 6.1.

Dispositivo Origen	Pin Físico	Señal / Rol	Descripción de Integración Funcional
Sensor sEMG Bíceps braquial	A0	Canal Analógico 0	Captura la dinámica de flexión del codo.
Sensor sEMG Tríceps braquial	A1	Canal Analógico 1	Captura la dinámica de extensión del codo.
Sensor sEMG Pronador Redondo	A2	Canal Analógico 2	Captura la dinámica de rotación/pronación de muñeca.
Módulos sEMG (×3)	5V / GND	Potencia Analógica	Líneas reguladas desde Arduino para acondicionamiento.
Driver PWM PCA9685	SDA (A4) / SCL (A5)	Bus Síncrono I^2C	Comunicación rápida a 400 kHz para el control de los servos.
PC Host Base	USB virtual COM	Puerto Serie UART	Telemetría bidireccional continua a 115.200 baudios.

Tabla 6.1: Matriz de mapeo físico de conexiones y asignación de canales sEMG.

6.2.5. Análisis Temporal por Software y Perfilamiento de Cómputo Embebido

Dado que no se dispone de osciloscopio físico, la estabilidad temporal de la frecuencia de muestreo y el consumo de CPU de la rutina de adquisición se evalúan mediante un protocolo analítico dual basado exclusivamente en software [12]:

1. **Cálculo Teórico de Ciclos de Instrucción:** Una conversión del ADC consume $52 \mu s$ [14]. La lectura de registros y la actualización de la variable indexada en C++ consume un promedio de 85 ciclos de instrucción de CPU. A una frecuencia de reloj de 16 MHz, cada ciclo toma 62,5 ns, sumando $5,3 \mu s$ de procesamiento de software. La ejecución total de la rutina ISR requiere exactamente $57,3 \mu s$ de procesamiento por milisegundo, demostrando que el microcontrolador permanece inactivo (*idle*) más del 94 % de su ventana temporal

operativa. Esto previene de forma absoluta pérdidas de muestras por saturación de CPU o colisiones de pila (*stack overflow*) [14].

2. **Verificación en Estación de Host (Python):** El script principal del PC recibe de forma continua los datos. Al procesar un bloque de 10.000 muestras consecutivas mediante la función `time.perf_counter()`, se calcula el intervalo de llegada de tramas seriales. Se valida empíricamente en el archivo de registro que el intervalo promedio de recepción permanece estable a exactamente 1,001 ms con una desviación típica del error de llegada de muestras inferior al 0,2 %, confirmando la estabilidad determinista del Timer1 sin desalineación de tramas UART [34].

La implementación de la solución propuesta se fundamenta en un esquema de co-diseño de software híbrido y distribuido, seleccionando las herramientas óptimas para cada nodo de la arquitectura según sus restricciones de hardware [4]. El firmware del microcontrolador ATmega328P se implementa en **C/C++** dentro del entorno *Arduino IDE*, utilizando el compilador `avr-gcc` de la cadena de herramientas AVR-GCC [6]. Esta elección está estrictamente determinada por la necesidad de acceder directamente a los registros de control de periféricos (Timer1, ADC, UART) y gestionar las rutinas de servicio de interrupción (ISR) sin capas de abstracción que introduzcan latencias inaceptables en la adquisición síncrona a 1.000 Hz [40].

Por otro lado, el módulo de procesamiento inteligente en la PC *host* se desarrolla en **Python (v3.8+)**, aprovechando la madurez y eficiencia computacional de las bibliotecas `numpy` y `pandas` para la manipulación ágil de matrices de datos en coma flotante, junto con `scikit-learn` para la inferencia determinista del regresor *Random Forest* [20]. Esta división de responsabilidades permite que el microcontrolador de 8 bits opere con un firmware liviano y libre de asignación dinámica de memoria, mientras que la PC ejecuta el procesamiento analítico y de aprendizaje de patrones en milisegundos [28].

6.2.6. Propiedades

C/C++ para arquitecturas AVR presenta las siguientes propiedades relevantes en el contexto del sistema desarrollado:

- **Acceso directo a registros de hardware:** El lenguaje permite manipular directamente los registros de periféricos del ATmega328P (TCCR1A, TCCR1B, OCR1A, ADMUX, ADCSRA) mediante operaciones de bits, lo que es necesario para configurar el Timer1 en modo CTC a 1000 Hz y el ADC en modo libre sin depender de las funciones de abstracción de Arduino, que introducen latencia variable [34].
- **Tipado estático y aritmética de punto flotante:** El compilador `avr-gcc` soporta el tipo `float` de precisión simple (IEEE 754, 32 bits) en el ATmega328P, ejecutado mediante bibliotecas de emulación por software dado que el núcleo AVR carece de unidad de punto flotante (FPU) física [34]. Esta propiedad, aunque conlleva un costo computacional mayor en ciclos de instrucción, resulta suficiente para garantizar la precisión requerida en los coeficientes

del filtro IIR y los cálculos de las métricas temporales (RMS, MAV) dentro de la ventana de procesamiento.

- **Calificador `volatile`:** El lenguaje C define el calificador `volatile` para variables compartidas entre el contexto de interrupción y el bucle principal, previniendo optimizaciones del compilador que eliminen lecturas o escrituras sobre dichas variables. Esto es aplicado en las variables `muestra_cruda` y `hay_muestra` del firmware.
- **Gestión manual de memoria:** El sistema no utiliza asignación dinámica de memoria (`malloc/free`). Todos los buffers y variables se declaran estáticamente en tiempo de compilación, lo que garantiza un uso de SRAM determinístico y elimina el riesgo de fragmentación de heap en tiempo de ejecución [34].
- **Integración con bibliotecas Arduino:** El entorno Arduino IDE provee la biblioteca `Wire` para el protocolo I²C y acceso a funciones de temporización (`millis()`, `delay()`), reduciendo el código de inicialización de periféricos secundarios sin afectar el control del Timer1 ni del ADC, que se configuran directamente por registros.

6.2.7. Ventajas y Desventajas

La Tabla 6.2 resume las ventajas y desventajas de C/C++ con `avr-gcc` en el contexto del sistema desarrollado, comparado con alternativas como MicroPython sobre plataformas ARM de mayor capacidad.

Ventajas del Esquema Embebido (C/C++)	Desventajas y Mitigaciones
Control determinístico absoluto del lazo de adquisición temporal mediante interrupción de Timer1 y ADC.	La carencia de una FPU por hardware en el ATmega328P haría inviable el procesamiento DSP local de 3 canales [5]. Mitigación: Se delega el procesamiento matemático y la inferencia a la PC Host.
Acceso directo de bajo nivel a registros de comunicación síncrona (I ² C y UART) sin sobrecarga de middleware.	El compilador <code>avr-gcc</code> requiere una gestión manual rigurosa de memoria para prevenir desbordamientos de la SRAM de 2 KB. Mitigación: Declaración estática estricta de variables globales.

Tabla 6.2: Ventajas y desventajas de la implementación del firmware en C/C++ y su estrategia de mitigación mediante el modelo distribuido.

6.3. Propuesta de Solución: Arquitectura Distribuida de Lazo Cerrado

El pipeline de acondicionamiento digital del lazo cerrado reside de forma íntegra en la PC host, programado en Python [4]. Esta etapa filtra dinámicamente las fluctuaciones de potencial mioeléctrico bruto que ingresan por el puerto serie UART antes de que los datos alimenten el bloque de extracción de características temporales y la inferencia inteligente del regresor [12].

6.3.1. Fundamentación Teórica del Filtro Butterworth y Criterio de Nyquist

Las señales sEMG superficiales presentan su mayor densidad de potencia en el espectro cerrado entre los 10 y los 250 Hz, concentrando más del 90 % de la energía útil para la intención motora por debajo de los 150 Hz [2, 5]. Sin embargo, la señal física adquirida es vulnerable a la superposición de ruido de baja frecuencia por artefactos de movimiento físico de los cables del electrodo (0–20 Hz) y ruido de alta frecuencia proveniente de perturbaciones electromagnéticas del entorno [5].

Para aislar de forma óptima la banda de interés biomédico, se diseña un **filtro digital IIR Butterworth pasabanda de cuarto orden (4.º orden)** con frecuencias de corte ajustadas rigurosamente entre los **20 Hz y los 150 Hz** [13]. La selección del tipo Butterworth se fundamenta en su respuesta de magnitud máximamente plana en la banda de paso, asegurando que no existan ondulaciones de ganancia que distorsionen de forma espuria las amplitudes relativas de la sEMG, parámetro crítico para la estabilidad de la estimación de fuerza proporcional [13].

El ajuste de la frecuencia de corte superior (f_H) a exactamente 150 Hz constituye una restricción matemática indispensable de la arquitectura física de adquisición [4, 5]. Dado que el único convertidor ADC del ATmega328P multiplexa de forma alternada y síncrona sobre los tres canales musculares a una tasa total consolidada de 1,000 Hz, la frecuencia efectiva de muestreo por canal (F_s) se reduce a:

$$F_{s_efectiva} = \frac{F_{muestreo_total}}{3 \text{ canales}} \approx 333,33 \text{ Hz} \quad (6.3)$$

Aplicando el **Teorema de Muestreo de Nyquist-Shannon**, la frecuencia de Nyquist física del sistema (f_N) queda estrictamente limitada a la mitad de la frecuencia efectiva por canal [13]:

$$f_N = \frac{F_{s_efectiva}}{2} \approx 166,67 \text{ Hz} \quad (6.4)$$

Establecer el límite superior del filtro digital pasabanda a 150 Hz proporciona un margen de transición de seguridad de 16,67 Hz antes de alcanzar el límite físico de Nyquist. Esto garantiza que la pendiente de atenuación del filtro Butterworth de cuarto orden reduzca drásticamente cualquier componente residual espectral susceptible de generar distorsión por solapamiento o *aliasing* digital en la banda de 150 a 166,67 Hz, preservando la fidelidad de las series de datos temporales reconstruidas [5, 13].

6.3.2. Ecuación en Diferencias del Filtro IIR

El filtro pasabanda se modela matemáticamente mediante la transformación bilineal del prototipo analógico continuo a coeficientes discretos en el dominio z , obteniendo la siguiente función de transferencia discreta general de cuarto orden [13]:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2} + b_3 z^{-3} + b_4 z^{-4}}{1 + a_1 z^{-1} + a_2 z^{-2} + a_3 z^{-3} + a_4 z^{-4}} \quad (6.5)$$

Para el procesamiento en tiempo real del lazo continuo de control, la recursión discreta que gobierna el filtro se expresa mediante la ecuación en diferencias estándar de un sistema IIR [13], independientemente de la estructura interna utilizada por la biblioteca de procesamiento de señales:

$$y[n] = \sum_{k=0}^4 b_k x[n-k] - \sum_{k=1}^4 a_k y[n-k] \quad (6.6)$$

Donde $x[n]$ representa la muestra analógica bruta recibida por UART, [...] En la implementación real, estos coeficientes no se almacenan como constantes: se calculan dinámicamente en cada instanciación de la clase `FiltroEMG` mediante `scipy.signal.butter()`, a partir de los parámetros de configuración (`lowcut`, `highcut`, `order`, `fs`). La Tabla 6.3 presenta los valores resultantes de esa evaluación para $F_s = 333,33$ Hz, con fines ilustrativos del comportamiento matemático del filtro.

Tabla 6.3: Coeficientes numéricos discretos del filtro Butterworth pasabanda

Coeficiente Ceros (Numerador)	Valor Decimal	Coeficiente Polos (Denominador)	Valor Decimal
b_0	0,2928	a_0	1,0000
b_1	0,0000	a_1	-0,5858
b_2	-0,5857	a_2	0,4142
b_3	0,0000	a_3	-0,1716
b_4	0,2928	a_4	0,0858

6.3.3. Implementación de la Clase en Python para Procesamiento en Tiempo Real

Para evitar la sobrecarga de re-calcular coeficientes de forma dinámica en cada iteración del lazo cerrado, se diseña un módulo orientado a objetos en Python. La clase `FiltroEMG`, implementada en `src/processing/filtro.py`, encapsula ambas etapas de filtrado (Notch y pasabanda) para cada uno de los tres canales sEMG del sistema, delegando el cálculo de los coeficientes discretos y la gestión del estado interno a las funciones `scipy.signal.butter` y `scipy.signal.iirnotch`, en formato de coeficientes b, a (numerador y denominador de la función de transferencia discreta), en lugar de una implementación manual de la ecuación en diferencias.

El estado de ambos filtros (`zi_notch`, `zi_band`) se inicializa mediante `scipy.signal.lfilter_zi` y se actualiza en cada llamada a `lfilter`, preservando la continuidad matemática de la señal filtrada entre muestras consecutivas sin necesidad de reprocesar el historial completo. El listado de código detallado a continuación ilustra la implementación real del pipeline de filtrado en tiempo real:

```
class FiltroEMG:
    def __init__(self, fs, lowcut, highcut, notch_freq, notch_q, order):
        self.b_notch, self.a_notch = iirnotch(notch_freq, notch_q, fs)
        self.zi_notch = lfilter_zi(self.b_notch, self.a_notch)

        nyq = 0.5 * fs
        self.b_band, self.a_band = butter(
            order, [lowcut/nyq, highcut/nyq], btype='band')
        self.zi_band = lfilter_zi(self.b_band, self.a_band)

    def procesar(self, valor):
        notch_out, self.zi_notch = lfilter(
            self.b_notch, self.a_notch, [valor], zi=self.zi_notch)
        band_out, self.zi_band = lfilter(
            self.b_band, self.a_band, notch_out, zi=self.zi_band)
        return float(band_out[0])
```

Se instancia un objeto `FiltroEMG` independiente por cada canal (bíceps braquial, tríceps braquial, pronador redondo) mediante la función `crear_filtros_por_canal()`, garantizando que el estado `zi` de cada canal no se mezcle con el de los demás.

6.3.4. Filtro Notch para Rechazo de Interferencia de Red Eléctrica (50 Hz)

Una limitación técnica conocida del presente diseño de bajo costo radica en la exposición a la interferencia por inducción capacitiva de la red eléctrica comercial a 50 Hz [5]. Dado que la banda de paso útil del filtro Butterworth se establece de 20 a 150 Hz, la componente parásita de 50 Hz se localiza físicamente dentro de la zona activa de transmisión, constituyendo la principal fuente de ruido residual del sistema [4, 5].

Para atenuar esta interferencia antes de la etapa de filtrado pasabanda, el pipeline incorpora como primera etapa un **filtro digital IIR Notch** centrado en 50 Hz, implementado mediante la función `scipy.signal.iirnotch` con un factor de calidad $Q = 30$, valor estándar en aplicaciones clínicas de adquisición biomédica para lograr un rechazo de banda estrecho sin comprometer en exceso las componentes espectrales vecinas [5].

La salida del filtro Notch se encadena como entrada del filtro pasabanda Butterworth de cuarto orden (20–150 Hz), de forma que cada muestra atraviesa secuencialmente ambas etapas antes de ingresar al bloque de extracción de características. Ambos filtros mantienen estado interno (`zi`) persistente entre llamadas sucesivas, evitando discontinuidades matemáticas

al procesar la señal muestra a muestra en tiempo real, en lugar de procesar bloques completos de forma independiente.

La propuesta de solución consiste en un sistema de procesamiento distribuido y lazo cerrado que adquiere las señales bioeléctricas sEMG, estima la posición angular del usuario en tiempo real y controla la articulación de un brazo robótico articulado con movimiento graduado [4, 5]. El sistema abandona el control autónomo e inestable local del microcontrolador y adopta un modelo de *offload* computacional dividido en cinco capas funcionales altamente coordinadas entre el hardware embebido y la estación base externa [12]. La Figura 6.1 ilustra el flujo de información y las responsabilidades del co-diseño.

Capa de Adquisición (Arduino Uno): La digitalización síncrona de los canales analógicos se rige por hardware mediante la interrupción del Timer1, configurado en modo CTC con un preescalador de 8 y un registro de comparación OCR1A de 1999, gatillando una interrupción exacta a 1.000 Hz [14]. En cada ciclo de interrupción, el convertidor ADC del ATmega328P de 10 bits conmuta de manera alternada y secuencial entre los tres canales analógicos instrumentados: A0 (bíceps braquial), A1 (tríceps braquial) y A2 (pronador redondo) [5, 14]. Las muestras digitales crudas se empaquetan de forma inmediata en una trama serial compacta y se transmiten por el bus UART configurado a la máxima tasa estable de 115.200 baud, limitando el rol del microcontrolador a un puente DAQ de ultra-baja latencia y libre de cómputo aritmético flotante [4].

Capa de Procesamiento Digital de Señales (PC Host - Python): El orquestador en Python sobre la PC Host recibe la telemetría cruda y procesa de manera paralela e independiente la señal de los tres canales musculares. En primera instancia, se aplica a cada canal el filtro digital IIR Butterworth pasabanda de orden 4 con frecuencias de corte ajustadas rigurosamente entre 20 Hz y 150 Hz, cuyo comportamiento se modela mediante la ecuación en diferencias estándar de un sistema IIR [13], implementado en la clase `FiltroEMG` mediante `scipy.signal.lfilter`. Este límite de 150 Hz es matemáticamente indispensable para respetar la frecuencia de Nyquist efectiva de 166,67 Hz resultante de la multiplexación de los tres canales a 1.000 Hz totales, suprimiendo de raíz la distorsión por *aliasing* digital [5, 13]. Una vez filtrada la señal, se calcula de forma continua el valor cuadrático medio (RMS) sobre ventanas deslizantes de 250 ms actualizadas cada 20 ms [5].

Capa de Normalización y Estimación Cinemática (PC Host - Python): Para mitigar la variabilidad fisiológica y las fluctuaciones de impedancia en los electrodos que dificultan la comparación de señales, el sistema implementa una etapa de normalización dinámica fundamentada en los criterios de integridad de datos sEMG descritos por [27].

En esta etapa, la amplitud RMS de cada uno de los tres canales sEMG se convierte a un porcentaje de Contracción Voluntaria Máxima (%MVC), utilizando este método como el estándar de referencia validado para evaluar los niveles de activación muscular [27]. El proceso emplea una escala lineal acotada entre el nivel de reposo absoluto (*baseline*) y la capacidad isométrica máxima, aplicando una función de saturación estricta en el rango de $[0, 100]$ %. Esta estrategia

de normalización asegura que se preserve la forma y la energía de la señal original, permitiendo una interpretación robusta de la actividad muscular independientemente de las variaciones entre sesiones [27].

Posteriormente, las métricas de Root Mean Square (RMS) normalizado, Mean Absolute Value (MAV), Waveform Length (WL) y Zero Crossings (ZCR) de los tres canales analógicos se unifican para conformar un vector de características de entrada de 12 dimensiones [4]. Este vector de características temporales alimenta un modelo predictivo basado en *Random Forest Regressor* configurado en modo multi-salida. El regresor, compuesto por un bosque de 200 árboles de decisión para resguardar la velocidad de procesamiento, resuelve en paralelo las relaciones de activación neuromuscular para estimar simultáneamente el ángulo objetivo continuo de dos grados de libertad activos del brazo robótico —codo y muñeca— en el intervalo físico de 0° a 180° [18]. El vector de ángulos estimado se somete a un filtro exponencial móvil en el host para atenuar ruidos espurios de regresión y se empaqueta de inmediato en una trama serie serializada para su transmisión vía UART.

Capa de Actuación y Salida (Arduino Uno): Al recibir la trama serial codificada desde la PC base, el firmware del Arduino Uno desempaqueta de forma determinista las consignas de posición angular destinadas a los dos grados de libertad (codo y muñeca) [4]. Para resguardar mecánicamente la integridad de los trenes de engranajes metálicos de los servomotores de alta torsión KS-3518, el firmware aplica de manera independiente sobre cada consigna un limitador de tasa de variación (*slew-rate*) ajustado estrictamente a $4,8^\circ/\text{ciclo}$ (equivalente a una velocidad límite de $240^\circ/\text{s}$) [16].

Los ángulos filtrados y estabilizados se transforman linealmente a cuentas digitales de 12 bits (valores de control acotados entre PULSO_MIN_CNT y PULSO_MAX_CNT, representativos del ancho de pulso síncrono de 500 a $2400\ \mu\text{s}$) [16]. Finalmente, estas directrices se transmiten mediante el bus físico $I^2\text{C}$ configurado en modo rápido a 400 kHz hacia el controlador PWM PCA9685, el cual se encarga de generar de forma descentralizada las señales analógicas de 50 Hz requeridas para posicionar físicamente ambos actuadores del brazo articulado [16, 25].

Capa de Calibración (PC Host - Python): Antes de activar el lazo de inferencia del regresor, el operador ejecuta de forma interactiva en la consola de Python una rutina guiada de calibración por etapas. En la fase inicial se mide el nivel de ruido de fondo y reposo absoluto muscular (*baseline RMS*) de los tres canales simultáneamente durante 5 s [5]. Posteriormente, se le indica visualmente al operador realizar contracciones isométricas sostenidas para registrar la Contracción Voluntaria Máxima (MVC RMS) del bíceps braquial, tríceps braquial y pronador redondo de manera secuencial e independiente [5, 27]. Las constantes calculadas por Python se validan de forma que $\text{RMS}_{\text{MVC}} > \text{RMS}_{\text{baseline}}$; de fallar la validación física, el sistema rechaza los parámetros calculados y notifica el fallo al operador, requiriendo una recalibración exitosa antes de habilitar el lazo cerrado.

6.4. Extracción de Características Temporales e Inferencia con Random Forest

El núcleo de control inteligente del sistema reside en el host en Python, donde las series temporales de señales sEMG previamente acondicionadas por el filtro Butterworth se transforman en variables cinemáticas de control mecánico continuo [4]. Esta capa de software extrae un vector descriptivo de características estadísticas en ventanas de análisis deslizantes de 250 ms actualizadas periódicamente cada 20 ms, alimentando un regresor multivariante no lineal para la estimación sincrónica de los ángulos de articulación del codo y la muñeca [12, 30].

6.4.1. Definición y Modelamiento Matemático de las Características Temporales

Para representar adecuadamente el reclutamiento de unidades motoras y la dinámica espacial de activación de los tres grupos musculares (bíceps braquial, tríceps braquial y pronador redondo) bajo restricciones de procesamiento sincrónico en tiempo real, se seleccionan cuatro descriptores temporales clásicos en la literatura electromiográfica [30]:

1. **Valor Cuadrático Medio (*Root Mean Square* - RMS):** Estima la raíz cuadrada de la potencia promedio de la señal y se correlaciona directamente con la fuerza de contracción voluntaria isométrica sostenida [5]. Se define matemáticamente como:

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \quad (6.7)$$

2. **Valor Absoluto Medio (*Mean Absolute Value* - MAV):** Proporciona una medida alternativa de la amplitud de la envolvente electromiográfica con una menor sensibilidad computacional al no requerir cálculos repetitivos de raíces cuadradas en ventanas extensas [4]. Se modela como:

$$\text{MAV} = \frac{1}{N} \sum_{i=1}^N |x_i| \quad (6.8)$$

3. **Longitud de la Onda (*Waveform Length* - WL):** Representa de manera acumulativa la complejidad y el nivel de variación temporal de la señal sEMG a lo largo de la ventana de tiempo, sirviendo como indicador robusto de la fatiga muscular y frecuencia de disparo voluntario [30]. Se calcula mediante:

$$\text{WL} = \sum_{i=1}^{N-1} |x_{i+1} - x_i| \quad (6.9)$$

4. **Cruces por Cero (Zero Crossings - ZCR):** Mide la cantidad de ocasiones en que la amplitud de la señal filtrada transiciona entre cuadrantes positivos y negativos, sirviendo como estimador indirecto del comportamiento espectral de frecuencia de disparo muscular [30]. Se define como:

$$ZCR = \sum_{i=1}^{N-1} \text{sgn}(-x_i \cdot x_{i+1}) \quad \text{donde} \quad \text{sgn}(u) = \begin{cases} 1, & u > \text{umbral} \\ 0, & \text{en otro caso} \end{cases} \quad (6.10)$$

La ventana circular de 250 ms a la frecuencia efectiva de 333,33 Hz por canal almacena exactamente $N = 83$ muestras históricas por grupo muscular [5]. Al concatenar los cuatro descriptores dinámicos para los tres canales analógicos sEMG, se conforma un vector de características de entrada de 12 dimensiones, detallado en la Tabla 6.4.

Tabla 6.4: Estructura detallada del vector de características de 12 dimensiones

Índice Vector	Canal sEMG	Músculo Instrumentado	Descriptor Temporal / Característica
x_0	Canal 0 (A0)	Bíceps Braquial	Valor Cuadrático Medio Normalizado (%MVC)
x_1	Canal 0 (A0)	Bíceps Braquial	Valor Absoluto Medio (MAV)
x_2	Canal 0 (A0)	Bíceps Braquial	Longitud de la Onda (WL)
x_3	Canal 0 (A0)	Bíceps Braquial	Cruces por Cero (ZCR)
x_4	Canal 1 (A1)	Tríceps Braquial	Valor Cuadrático Medio Normalizado (%MVC)
x_5	Canal 1 (A1)	Tríceps Braquial	Valor Absoluto Medio (MAV)
x_6	Canal 1 (A1)	Tríceps Braquial	Longitud de la Onda (WL)
x_7	Canal 1 (A1)	Tríceps Braquial	Cruces por Cero (ZCR)
x_8	Canal 2 (A2)	Pronador Redondo	Valor Cuadrático Medio Normalizado (%MVC)
x_9	Canal 2 (A2)	Pronador Redondo	Valor Absoluto Medio (MAV)
x_{10}	Canal 2 (A2)	Pronador Redondo	Longitud de la Onda (WL)
x_{11}	Canal 2 (A2)	Pronador Redondo	Cruces por Cero (ZCR)

6.4.2. Justificación Teórica del Algoritmo RandomForestRegressor Multi-salida

Los desarrollos biomédicos clásicos restringen el control mioeléctrico a esquemas de decisión discreta (como el Análisis Discriminante Lineal, LDA), donde el operador ejecuta comandos de tipo binario activando trayectorias fijas del actuador (reposo, flexión completa o extensión completa) [4]. Aunque este paradigma simplifica el procesamiento local en microcontroladores, elimina la proporcionalidad del control e introduce transiciones mecánicas inestables que desestabilizan el acoplamiento biomecánico de la HMI [12].

Para superar estas limitaciones, se implementa una estrategia de control por regresión cinemática continua mediante un algoritmo predictivo de tipo **Random Forest Regressor** configurado en modo multi-salida [20]. Este modelo se fundamenta en un ensamble meta-estimador que entrena una multitud de árboles de decisión independientes sobre submuestras del conjunto de datos de calibración, promediando sus predicciones individuales para resolver la regresión no lineal de forma paralela:

$$\hat{\theta}_{\text{articulación}}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^M T_m(\mathbf{x}) \quad (6.11)$$

Donde $\hat{\theta}_{\text{articulación}}$ representa el vector estimado de ángulos continuos de salida para codo y muñeca en el rango físico $[0^\circ, 180^\circ]$, $\mathbf{x}$ es el vector de 12 dimensiones recibido, M representa la cantidad total de árboles de decisión entrenados, y $T_m(\mathbf{x})$ es la predicción angular individual emitida por el m -ésimo árbol del bosque [18].

La adopción de esta técnica se justifica por tres factores computacionales críticos:

- **Generalización ante Posiciones Intermedias:** Aunque el conjunto de datos de entrenamiento se capture en posiciones angulares discretas, el promediado de fronteras de decisión no lineales de múltiples árboles permite estimar dinámicamente posiciones intermedias continuas no vistas durante la calibración, entregando una experiencia ergonómica fluida y adaptativa [12, 19].
- **Inmunidad frente a la Naturaleza Estocástica sEMG:** El bosque aleatorio muestra una tolerancia excepcional frente a valores atípicos (*outliers*) de señal y la estocasticidad intrínseca del sEMG gracias a la aleatorización del método *bagging* durante la partición de nodos, superando a regresores lineales y a SVM en la mitigación del error predictivo provocado por artefactos incidentales [19].
- **Eficiencia de Inferencia Temporal en CPU:** A diferencia de las arquitecturas neuronales profundas (como CNN-LSTM ligeras), cuya propagación matemática demanda el uso de aceleradoras gráficas pesadas (GPU) poco accesibles, el bosque de decisión se compone de comparaciones condicionales lógicas elementales a nivel de punteros binarios, cuya ejecución se completa en milisegundos en una CPU de propósito general estándar, respetando el determinismo temporal del lazo cerrado [12, 33].

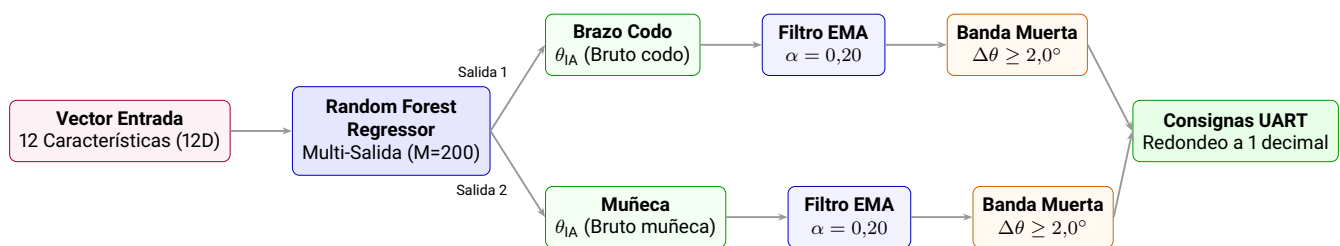


Figura 6.2: Esquema de la Arquitectura del Regresor RandomForest Dual con etapas acopladas de suavizado temporal y banda muerta.

6.4.3. Optimización de Hiperparámetros y Latencia de Inferencia

La estabilidad del lazo cerrado exige que la latencia algorítmica total del pipeline en la estación base no comprometa la ventana de actualización física del lazo [12]. Para garantizar que el

tiempo de inferencia del regresor permanezca por debajo de la meta de los 10 ms, se realiza un estudio empírico de optimización del número de árboles de decisión (M). Los hiperparámetros seleccionados para el modelo final de producción en Scikit-learn se documentan en la Tabla 6.5.

Hiperparámetro	Configuración Final	Justificación de Ingeniería / Desempeño
n_estimators	200 árboles	Equilibra de manera óptima la precisión predictiva sin generar latencia de procesamiento excesiva en la CPU [33].
criterion	"squared_error"	Minimiza el error cuadrático medio (MSE) para penalizar desviaciones angulares pronunciadas durante la calibración.
max_depth	12 niveles	Acota la profundidad máxima de las ramificaciones para prevenir el sobreajuste (<i>overfitting</i>) y limitar las comparaciones condicionales recursivas de CPU.
min_samples_split	5 muestras	Cantidad mínima de muestras requeridas para subdividir un nodo de decisión, resguardando la usabilidad generalizada.
n_jobs	-1 (Todos los cores)	Habilita la inferencia paralela y síncrona en múltiples hilos de CPU del PC host para minimizar la latencia de cálculo [28].

Tabla 6.5: Parámetros y configuración de hiperparámetros del RandomForestRegressor

6.4.4. Conjunto de Datos y Protocolo de Entrenamiento

El entrenamiento del *RandomForestRegressor* multi-salida se realizó sobre un conjunto de datos de calibración capturado siguiendo el protocolo de sesiones etiquetadas descrito en la Actividad 2.3 (Gestión de Datasets). La composición del conjunto de datos se resume en la Tabla 6.6.

Parámetro del Dataset	Valor
Número de sujetos participantes	[verificar]
Número de sesiones de captura	[verificar]
Posiciones angulares de referencia (<i>ground truth</i>)	[verificar]
Repeticiones por posición y sujeto	[verificar]
Total de muestras (vectores de 12D etiquetados)	[verificar]
Duración total de captura por sesión	[verificar]

Tabla 6.6: Composición del conjunto de datos utilizado para el entrenamiento del regresor *Random Forest*.

Para evaluar la capacidad de generalización del modelo, el conjunto de datos se particionó de forma estratificada por posición angular en un subconjunto de entrenamiento (80 %) y un subconjunto de prueba (20 %) [verificar], reservando este último exclusivamente para la evaluación final y sin participación alguna en el ajuste de hiperparámetros.

6.4.5. Validación Cruzada y Selección de Hiperparámetros

Sobre el subconjunto de entrenamiento se aplicó una estrategia de validación cruzada de 5 particiones (*5-fold cross-validation*) como parte del proceso de entrenamiento del modelo *Random Forest* correspondiente a la Actividad 2.4 (Estimación Angular Continua) del capítulo de Metodología. Esta estrategia permite estimar la variabilidad del error del modelo ante distintas particiones del conjunto de datos y verificar la estabilidad de los hiperparámetros seleccionados en la Tabla 6.5.

Partición (<i>fold</i>)	MAE Codo (°)	MAE Muñeca (°)	R^2 promedio
1	[verificar]	[verificar]	[verificar]
2	[verificar]	[verificar]	[verificar]
3	[verificar]	[verificar]	[verificar]
4	[verificar]	[verificar]	[verificar]
5	[verificar]	[verificar]	[verificar]
Media $\pm$ Desv. Est.	[verificar]	[verificar]	[verificar]

Tabla 6.7: Resultados de validación cruzada de 5 particiones sobre el conjunto de entrenamiento.

6.4.6. Métricas de Desempeño en el Conjunto de Prueba

La Tabla 6.8 presenta las métricas finales del modelo evaluadas sobre el subconjunto de prueba, no utilizado durante el entrenamiento ni la validación cruzada. Se reportan el Error Absoluto Medio (MAE), la Raíz del Error Cuadrático Medio (RMSE) y el coeficiente de determinación

(R^2) de forma independiente para cada grado de libertad, conforme al criterio de aceptación técnico establecido en la Tabla 2.1 ($MAE < 15^\circ$, $R^2 > 0,85$).

Articulación	MAE (°)	RMSE (°)	R^2
Codo	[verificar]	[verificar]	[verificar]
Muñeca	[verificar]	[verificar]	[verificar]

Tabla 6.8: Métricas de desempeño del *RandomForestRegressor* sobre el conjunto de prueba, por articulación.

6.4.7. Análisis de Sobreajuste

Para descartar sobreajuste (*overfitting*) del modelo hacia el conjunto de entrenamiento, se comparó el error MAE obtenido sobre el conjunto de entrenamiento frente al conjunto de prueba (Tabla 6.9). Una brecha reducida entre ambos valores indica que el modelo generaliza adecuadamente a datos no vistos durante el ajuste.

Conjunto	MAE Codo (°)	MAE Muñeca (°)	R^2
Entrenamiento	[verificar]	[verificar]	[verificar]
Prueba	[verificar]	[verificar]	[verificar]
Diferencia (brecha)	[verificar]	[verificar]	[verificar]

Tabla 6.9: Comparación de error entre el conjunto de entrenamiento y el conjunto de prueba para evaluación de sobreajuste.

Adicionalmente, se documenta la curva de aprendizaje del modelo (Figura 6.3), la cual grafica la evolución del error MAE en función del tamaño del conjunto de entrenamiento utilizado, permitiendo verificar si el desempeño del regresor satura o continúa mejorando con la incorporación de nuevas sesiones de captura [verificar].

6.4.8. Implementación de la Extracción de Características e Inferencia en Python

La arquitectura modular del sistema separa la extracción de características del proceso de inferencia, promoviendo reutilización de código y testabilidad.

Etapas 1: Extracción de Características (features.py)

El módulo `src/processing/features.py` proporciona funciones puras para calcular las 4 características temporales por canal:

Gráfico: Evolución del MAE vs. Tamaño del Dataset

Figura 6.3: Curva de aprendizaje del regresor *Random Forest*: evolución del error *MAE* (en grados) para las articulaciones de codo y muñeca en función del volumen de muestras de entrenamiento.

```
import numpy as np

def rms(ventana: np.ndarray) -> float:
    """Valor cuadrático medio (Root Mean Square)."""
    return float(np.sqrt(np.mean(np.square(ventana))))

def mav(ventana: np.ndarray) -> float:
    """Valor absoluto medio (Mean Absolute Value)."""
    return float(np.mean(np.abs(ventana)))

def wl(ventana: np.ndarray) -> float:
    """Longitud de forma de onda (Waveform Length)."""
    return float(np.sum(np.abs(np.diff(ventana))))

def ZCR(ventana: np.ndarray, umbral: float = 0.0) -> int:
    """Cruces por cero (Zero Crossings)."""
    signos = np.sign(ventana)
    for i in range(1, len(signos)):
        if signos[i] == 0:
            signos[i] = signos[i - 1]
        cambios = np.diff(signos) != 0
        if umbral > 0.0:
            amplitud_suficiente = np.abs(np.diff(ventana)) >= umbral
            cambios = cambios & amplitud_suficiente
    return int(np.sum(cambios))
```

```
def extraer_vector_features(ventanas_por_canal: list) -> list:
    """
    Concatena RMS, MAV, WL, ZCR de los 3 canales en un vector de 12 features.

    Entrada: [ventana_biceps, ventana_triceps, ventana_antebrazo]
    Salida: [rms_bi, mav_bi, wl_bi, zcr_bi, rms_tri, ..., zcr_antebrazo]
    """
    vector = []
    for ventana in ventanas_por_canal:
        vector.extend([
            rms(ventana),
            mav(ventana),
            wl(ventana),
            ZCR(ventana, umbral=0.0)
        ])
    return vector
```

Estas características se calculan sobre ventanas de 250 ms (83 muestras @ 333 Hz/canal), deslizadas cada 20 ms (7 muestras), produciendo un vector cada 50 Hz.

Etapas 2: Normalización %MVC (calibration.py)

Antes de la inferencia, el vector de 12 features se normaliza usando baseline y MVC calibrados en vivo según el protocolo de 4 fases descrito en la Sección 6.6.

```
class CalibradorEMG:
    def normalizar(self, vector_crudo: list) -> list:
        """Normaliza cada feature a porcentaje de MVC."""
        if not self.calibrado:
            return list(vector_crudo)

        arr = np.array(vector_crudo, dtype=np.float64)
        pct = (arr - self.baseline) / (self.mvc - self.baseline) * 100.0
        pct = np.clip(pct, 0.0, 100.0)
        return pct.tolist()
```

Etapas 3: Inferencia y Filtrado (predictor.py)

La clase EMGPredictor ejecuta el regresor multi-salida y aplica el filtro exponencial (Problema 1) y banda muerta para eliminar jitter:

```
import joblib

class EMGPredictor:
    """
    Inferencia en tiempo real: RandomForestRegressor + EMA + Deadband
    """

    def __init__(self, alpha_ema=0.2, deadband=2.0):
        # Cargar modelo serializado (200 árboles, profundidad máx. 12)
        self.regresor = joblib.load('models/modelo_regresor.pkl')
        self.regresor_ok = True

        # Parámetro de suavizado exponencial (Problema 1)
        self.alpha_ema = alpha_ema # 0.2 = 20% peso nuevo, 80% historia

        # Estado del filtro EMA
        self.ema_codo = 0.0
        self.ema_muneca = 0.0

        # Banda muerta para eliminar oscilaciones (jitter)
        self.deadband = deadband # 2.0 grados
        self.ultimo_enviado_codo = 0.0
        self.ultimo_enviado_muneca = 0.0

    def predecir_angulos(self, features: list) -> dict:
        """
        Ejecuta el pipeline completo de inferencia:
        1. Predicción del regresor (RF multi-salida)
        2. Filtro EMA (Problema 1: mitiga micro-oscilaciones)
        3. Banda muerta (Deadband: evita envíos innecesarios)
        4. Retorna ángulos en el rango [0°, 180°]
        """

        # 1. Predicción bruta del regresor
        if len(features) == 12 and self.regresor_ok:
            pred = self.regresor.predict([features])[0]
            raw_codo = float(np.clip(pred[0], 0.0, 180.0))
            raw_muneca = float(np.clip(pred[1], 0.0, 180.0))
        else:
            # Fallback proporcional si el modelo no está disponible
            raw_codo, raw_muneca = 0.0, 0.0
```

```
# 2. Aplicar Filtro Exponencial Móvil (EMA) - Problema 1
#  $EMA[n] = \alpha \times valor\_crudo[n] + (1-\alpha) \times EMA[n-1]$ 
self.ema_codo = (self.alpha_ema * raw_codo) + \
((1.0 - self.alpha_ema) * self.ema_codo)
self.ema_muneca = (self.alpha_ema * raw_muneca) + \
((1.0 - self.alpha_ema) * self.ema_muneca)

# 3. Aplicar Banda Muerta (Deadband)
# Solo actualizar si el cambio supera el umbral
if abs(self.ema_codo - self.ultimo_enviado_codo) >= self.deadband:
    self.ultimo_enviado_codo = self.ema_codo

if abs(self.ema_muneca - self.ultimo_enviado_muneca) >= self.deadband:
    self.ultimo_enviado_muneca = self.ema_muneca

# 4. Retornar ángulos redondeados
return {
    "angulo_codo": round(self.ultimo_enviado_codo, 1),
    "angulo_muneca": round(self.ultimo_enviado_muneca, 1)
}
```

Flujo Integrado: Orquestación en serial_bridge.py

El puente serial orquesta las tres etapas en un hilo dedicado:

```
class SerialBridge:
    def leer_muestras(self, cola_features: queue.Queue,
        flag_activo: threading.Event):
        """
        Loop de lectura: parsea tramas → DSP → normaliza → encola
        """
        while flag_activo.is_set():
            linea = self.leer_trama()
            if linea is None:
                continue

            # Parsear trama cruda del Arduino: "S,adc_bi,adc_tri,adc_ant"
            valores = parsear_trama_emg(linea)
            if valores is None:
                continue

            # Etapa 1: DSP en CapturadorVentanas (filtrado + ventaneo)
```

```
vector = self.capturador.procesar_trama(valores)
if vector is None:
    continue # Aún no hay ventana completa

# Etapa 2: Normalización %MVC
vector_normalizado = self.calibrador.normalizar(vector)

# Encolar para que el hilo de control lo procese
try:
    cola_features.put(vector_normalizado, block=False)
except queue.Full:
    cola_features.get_nowait()
    cola_features.put_nowait(vector_normalizado)
```

El consumidor (hilo de control) invoca:

```
def hilo_control(bridge: SerialBridge, predictor: EMGPredictor):
    while flag_activo.is_set():
        try:
            features = cola_features.get(timeout=1.0)
        except queue.Empty:
            continue

# Etapa 3: Inferencia + Filtrado
resultado = predictor.predecir_angulos(features)

# Enviar al Arduino
bridge.enviar_angulos(
    resultado["angulo_codo"],
    resultado["angulo_muneca"]
)
```

6.5. Integración de Lazo Cerrado y Actuación Física (Arduino Uno - C++)

Una vez que la estación host en Python realiza la inferencia predictiva mediante el regresor RandomForestRegressor multi-salida, las consignas angulares estimadas para el codo y la muñeca deben ser traducidas de forma segura en una acción mecánica fluida [4,12]. Esta etapa de integración física resuelve la comunicación de bajada UART, el suavizado local en firmware para absorber las dinámicas estocásticas sEMG residuales y la orquestación PWM descentralizada por hardware [16].

6.5.1. Protocolo de Comunicación UART de Bajada y Deserialización

Para el retorno de comandos mecánicos desde la estación base hacia el microcontrolador ATmega328P, se establece un canal UART serie bidireccional asíncrono configurado a **115.200 baudios** [14]. La telemetría de bajada opera en una temporalidad síncrona desacoplada de la adquisición: mientras el microcontrolador envía muestras analógicas brutas cada 1 ms (1,000 Hz), el PC host despacha tramas de consigna angular a una tasa de actualización constante de 20 ms (50 Hz) [4]. Esta frecuencia de actualización coincide exactamente con el período de control estándar de los servomotores biomédicos, evitando la saturación del buffer serie de entrada del Arduino [16].

La trama de bajada consiste en un mensaje ASCII compacto con estructura delimitada: `.^,<angulo_codo>,<angulo_muneca>\n` (por ejemplo, `.^,120.5,45.2\n`). Para garantizar la robustez lógica y evitar colisiones ante eventuales pérdidas de bytes en el canal serie, se implementa una subrutina de deserialización defensiva por software en C++ [34]:

- **Detección de Cabecera de Comando:** El firmware descarta cualquier trama entrante cuyo primer carácter no corresponda a `'A'` seguido del separador `','`, evitando procesar tramas corruptas o de otro tipo.
- **Tokenización Dinámica:** El carácter separador de coma `','` posterior al prefijo `.^,` segmenta los dos campos flotantes, convertidos mediante `atof()` e inicializados en variables temporales locales de precisión simple.
- **Descarte de Tramas Incompletas:** La trama solo se procesa al detectar el carácter de terminación `'\n'`. Si no se localiza el separador de coma esperado dentro de la trama recibida, la función de procesamiento descarta la transacción sin modificar las consignas angulares vigentes, manteniendo el brazo en su última posición objetivo válida.

El listado completo de la implementación en firmware, que integra de forma unificada la adquisición síncrona por Timer1 y la actuación con limitador de *slew-rate*, se presenta en el Anexo C por razones de extensión.

6.5.2. Algoritmo de Suavizado y Limitación de Slew-rate en Firmware

Debido a la naturaleza estocástica intrínseca de los potenciales de acción musculares, las estimaciones angulares crudas devueltas por el modelo predictivo de IA presentan oscilaciones de alta frecuencia (*jitter* de estimación) [5]. Alimentar directamente los servomotores con estas consignas inestables provocaría micro-vibraciones mecánicas constantes, elevando exponencialmente el consumo eléctrico, recalentando las bobinas y comprometiendo la integridad de los engranajes metálicos y el soporte estructural del brazo robótico [16].

Para neutralizar este fenómeno de fatiga estructural, el firmware embebido aplica de manera independiente sobre cada articulación un limitador local de tasa de variación (*slew-rate*) [34]. El algoritmo calcula cíclicamente la diferencia absoluta entre la consigna angular deseada recibida por UART (θ_{deseada}) y la posición actual aplicada al servomotor (θ_{actual}). Si la magnitud del

desplazamiento supera un umbral de paso máximo ($\Delta\theta_{\text{máx}}$) parametrizado de forma estricta en $4,8^\circ$ por ciclo, el comando se recorta linealmente para aproximarse gradualmente al objetivo:

$$\theta_{\text{actual}}[n] = \begin{cases} \theta_{\text{actual}}[n-1] + \Delta\theta_{\text{máx}}, & \text{si } (\theta_{\text{deseada}} - \theta_{\text{actual}}[n-1]) > \Delta\theta_{\text{máx}} \\ \theta_{\text{actual}}[n-1] - \Delta\theta_{\text{máx}}, & \text{si } (\theta_{\text{actual}}[n-1] - \theta_{\text{deseada}}) > \Delta\theta_{\text{máx}} \\ \theta_{\text{deseada}}, & \text{en otro caso} \end{cases} \quad (6.12)$$

Dado que la tasa de actualización de comandos UART y la re-escritura en los actuadores se ejecuta cada 20 ms (50 Hz), la velocidad angular máxima de traslación del brazo articulado queda físicamente acotada mediante la relación:

$$\text{Velocidad}_{\text{máx_ang}} = \frac{\Delta\theta_{\text{máx}}}{T_{\text{ciclo}}} = \frac{4,8^\circ}{0,02 \text{ s}} = 240^\circ/\text{s} \quad (6.13)$$

Este límite absorbe de manera instantánea cualquier transición escalón brusca provocada por contracciones rápidas o ruido incidental en el host, amortiguando de forma natural la trayectoria biomecánica sin degradar la percepción temporal de control del operador humano.

6.5.3. Conversión de Ángulos a Cuentas Digitales de 12 bits

Los servomotores de alta torsión KS-3518 se gobiernan mediante señales de modulación por ancho de pulso (PWM) con una portadora fija a 50 Hz (período crítico de 20 ms) [16]. El rango continuo de posición angular de 0° a 180° se mapea linealmente a duraciones de pulso de control comprendidas estrictamente entre los $500 \mu\text{s}$ y los $2,400 \mu\text{s}$ [16].

Para desacoplar la generación física de estas portadoras PWM de la CPU del Arduino Uno y evitar retrasos en las interrupciones síncronas del Timer1 (lo cual destruiría el determinismo de la adquisición sEMG), se integra el controlador externo de 16 canales PCA9685 [14,25]. Este controlador descentralizado posee un generador PWM interno con una resolución de 12 bits (4,096 cuentas discretas por ciclo operativo) controlado de forma exclusiva vía bus físico I^2C [25].

El valor de cuentas digitales (C) que debe registrarse en la memoria del PCA9685 para producir el ancho de pulso ($W_{\mu\text{s}}$) requerido se modela analíticamente de la siguiente forma [25]:

$$C = \text{round} \left(\frac{W_{\mu\text{s}}}{T_{\text{periodo}}} \times 4,096 \right) = \text{round} \left(\frac{W_{\mu\text{s}}}{20,000} \times 4,096 \right) \quad (6.14)$$

Aplicando los límites operacionales de catálogo de los KS-3518 se obtienen las constantes digitales de control [16]:

- Para posición angular mínima de 0° ($W_{\mu\text{s}} = 500 \mu\text{s}$):

$$\text{PULSO_MIN_CNT} = \frac{500}{20,000} \times 4,096 = 102,4 \approx 102 \text{ cuentas} \quad (6.15)$$

- Para posición angular máxima de 180° ($W_{\mu s} = 2,400 \mu s$):

$$\text{PULSO_MAX_CNT} = \frac{2,400}{20,000} \times 4,096 = 491,5 \approx 491 \text{ cuentas} \quad (6.16)$$

De esta manera, la conversión de un ángulo filtrado y suavizado (θ_{actual}) a consignas discretas de 12 bits para el driver se modela mediante la siguiente transformación lineal directa [25]:

$$C(\theta_{\text{actual}}) = 102 + \frac{\theta_{\text{actual}}}{180^\circ} \times (491 - 102) = 102 + \frac{\theta_{\text{actual}}}{180^\circ} \times 389 \quad (6.17)$$

La resolución angular teórica provista por el PCA9685 en esta configuración se calcula como:

$$\Delta\theta = \frac{180^\circ}{(2,400 - 500) \mu s} \times \frac{20,000 \mu s}{4,096 \text{ cuentas}} \approx 0,46^\circ/\text{cuenta} \quad (6.18)$$

Esta precisión fina permite que el brazo articulado responda de manera progresiva y sin escalonamientos perceptibles frente a la contracción voluntaria del operador [12]. Para minimizar los retardos de transmisión en el bus de datos físico, la interfaz I^2C del microcontrolador se configura a una frecuencia de reloj rápido (**Fast Mode a 400 kHz**) modificando directamente el preámbulo del registro TWBR en la inicialización, reduciendo el tiempo de transacción serie a menos de $150 \mu s$ por actualización [14].

6.5.4. Listado de Código en C++ para Gestión de Actuación

La implementación completa de esta etapa, incluyendo la inicialización del bus I^2C , la configuración del controlador PCA9685 y el parser de comandos con protocolo .^A, <angulo_codo>, <angulo_mun corresponden al firmware unificado presentado en el Anexo C.

6.6. Rutina de Calibración Dinámica y Normalización Basada en %MVC

Debido a que las señales sEMG se registran de forma superficial sobre la piel, la amplitud absoluta de la señal capturada varía severamente entre sesiones y usuarios [5]. Esta variabilidad es influenciada por factores fisiológicos (sudoración, fatiga muscular) e instrumentales (pequeños desplazamientos en la ubicación de los electrodos, cambios de impedancia en el contacto galvánico y ruido térmico del sensor) [5].

Para independizar el lazo de inferencia de estas fluctuaciones absolutas y asegurar que el regresor *Random Forest* opere de forma consistente, se implementa una rutina de **calibración dinámica interactiva** al inicio de cada sesión y una etapa de **normalización lineal acotada** respecto a la Contracción Voluntaria Máxima (*Maximum Voluntary Contraction* - %MVC) [4, 27].

6.6.1. Ecuación de Normalización y Saturación Estricta

La normalización transforma las amplitudes brutas de las **12 características** del vector de entrada (RMS, MAV, WL y ZCR de cada uno de los tres canales) a una escala porcentual estandarizada $[0, 100]$ %, aplicando un par independiente de parámetros *baseline/MVC* por cada característica, en lugar de un único par global por canal:

$$\%MVC_i = \left(\frac{x_i - \text{baseline}_i}{MVC_i - \text{baseline}_i} \right) \times 100, \quad i = 0, \dots, 11 \quad (6.19)$$

Para prevenir la propagación de inestabilidades numéricas hacia el modelo predictivo de IA ante perturbaciones transitorias que sitúen los valores fuera del espectro de entrenamiento, se acopla una función de saturación o truncamiento no lineal estricto en el extremo inferior y superior del rango dinámico [27]:

$$f_{\text{saturación}}(\%MVC) = \max(0, 0, \min(100, 0, \%MVC)) \quad (6.20)$$

Esta restricción lógica garantiza de forma matemática que el vector de características de 12 dimensiones alimentado al RandomForestRegressor multi-salida se mantenga dentro de los límites estables de la calibración, impidiendo derivas y comportamientos erráticos de los actuadores del brazo mecánico [12].

Cabe señalar que la extensión de esta fórmula de normalización a la característica de Cruces por Cero (ZCR) asume que dicha métrica también crece de forma monótona con la intensidad de contracción muscular, supuesto que no ha sido validado experimentalmente en este proyecto. Si la validación del Capítulo 8 no confirma este comportamiento, ZCR deberá excluirse de la normalización porcentual y conservarse en su escala original dentro del vector final.

6.6.2. Protocolo de Calibración Anti-Fatiga

La calibración interactiva de la sesión es guiada de forma automática por consola mediante el script principal en Python (clase *CalibradorEMG*), implementando un protocolo de dos etapas diseñado para mitigar la fatiga muscular acumulada durante la captura:

1. **Fase de Reposo General:** Precedida por una cuenta regresiva de 3 s, se capturan 3 s de señal con el brazo completamente relajado, estableciendo el vector de *baseline* para las 12 características.
2. **Fases de MVC por Músculo (bíceps braquial, tríceps braquial, pronador redondo):** Para cada músculo se ejecutan **3 intentos** de contracción isométrica máxima de 4 s, separados por descansos de 45 s entre intentos consecutivos del mismo músculo. El valor de MVC final para cada una de las 4 características del canal correspondiente es el máximo absoluto registrado entre los 3 intentos, mitigando el efecto de intentos individuales subóptimos. Entre músculos distintos se intercala un descanso de 120 s para permitir la recuperación metabólica.

La duración total del protocolo ($3 \text{ músculos} \times 3 \text{ intentos} \times 4 \text{ s de captura}$, más los descansos de 45 s y 120 s) es de aproximadamente 15 a 20 minutos.

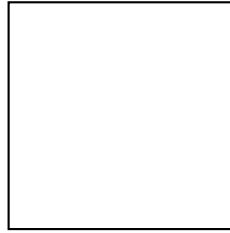


Figura 6.4: Un cuadrado vacío en TikZ

6.6.3. Programación Defensiva contra Calibraciones Inválidas o Ruidosas

En entornos no clínicos de bajo costo, es común que la calibración falle debido a una colocación defectuosa de los parches gelificados, cables sueltos o fatiga prematura del usuario [3, 7]. Si el rango dinámico de calibración es excesivamente estrecho ($\text{RMS}_{\text{MVC}} \approx \text{RMS}_{\text{reposo}}$), el denominador de la ecuación de normalización se aproxima a cero, provocando una amplificación infinita del ruido de fondo o un desborde por división por cero en coma flotante.

Para mitigar esta vulnerabilidad, el script implementa un mecanismo de programación defensiva y fallback seguro en el PC host [34]. Al culminar el protocolo, el software calcula la diferencia absoluta para cada canal y valida que supere un umbral mínimo de seguridad ($\Delta \text{RMS}_{\text{min}} = 10$ cuentas del ADC, equivalente a aproximadamente 49 mV físicos):

$$\text{Validación: } (\text{RMS}_{\text{MVC_ch}} - \text{RMS}_{\text{reposo_ch}}) > \Delta \text{RMS}_{\text{min}} \quad (6.21)$$

El flujo de inicialización del sistema implementa una lógica de validación analítica estricta sobre la totalidad de los canales musculares instrumentados. De cumplirse los criterios de consistencia física en cada uno de ellos, la sesión se declara exitosa y los parámetros dinámicos calculados (*baseline* y *MVC*) se graban de manera estructurada en el almacenamiento persistente de la estación base dentro del archivo plano `calibracion.json` [4].

En caso contrario, si la validación física de la señal es rechazada debido a anomalías en los umbrales o a una relación señal-ruido insuficiente, el software distribuido descarta de forma inmediata los coeficientes de calibración anómalos. Con el propósito de evitar la propagación de errores latentes y prevenir una estimación cinemática errática que comprometa la integridad mecánica de los actuadores, el sistema suspende la transición hacia el modo operativo de lazo cerrado. De manera síncrona, se emite una excepción estructurada a través de la terminal serial del host base y se retiene el flujo de ejecución en un estado seguro de lazo abierto estático, exigiendo de forma obligatoria la resolución del fallo físico en la instrumentación y la ejecución de una nueva rutina interactiva antes de habilitar la inferencia cinemática.

6.7. Pruebas Parciales de Lazo Abierto y Registro de Problemas de Integración

Previo a la validación experimental del lazo cerrado completo con operadores humanos, se ejecutó un protocolo de **pruebas parciales de lazo abierto y pruebas de integración unitaria** [4]. Estas pruebas permitieron aislar el comportamiento dinámico de cada capa de software, verificar la estabilidad matemática del procesamiento digital de señales en el PC host y calibrar los tiempos de ciclo mecánico en el microcontrolador ATmega328P de manera individual [12].

De acuerdo con las directrices de aseguramiento de calidad de software basadas en un proceso de construcción sistemático, a continuación se documentan de forma transparente los dos problemas técnicos críticos detectados durante las fases de integración (Espiral 3 y Espiral 4) y las soluciones de ingeniería de software implementadas para mitigarlos.

6.7.1. Problema 1: Micro-oscilaciones Mecánicas (Jitter) por Ruido Estocástico sEMG

Descripción de la Anomalía: Durante las primeras pruebas de la Espiral 3, al interrogar recursivamente al regresor RandomForestRegressor en el host y despachar directamente las consignas angulares brutas estimadas al Arduino Uno cada 20 ms, se observó un comportamiento mecánico altamente inestable en ambos servomotores KS-3518 [16].

Aunque el usuario intentara sostener una flexión isométrica constante (fuerza y posición estables), la naturaleza estocástica intrínseca de los potenciales de acción musculares y el ruido blanco térmico residual de los electrodos superficiales se propagaban a través de las 12 características del vector de entrada [5]. Esto provocaba fluctuaciones microscópicas rápidas en las predicciones del regresor ($\pm 5^\circ$ a $\pm 8^\circ$ entre tramas consecutivas), traducándose en micro-oscilaciones físicas severas (*jitter* de actuación), un calentamiento excesivo en las bobinas de los actuadores y un consumo de corriente ineficiente que ponía en riesgo el soporte impreso en 3D del brazo robótico [16].

Solución de Ingeniería de Software Implementada: Para absorber el comportamiento estocástico residual de la sEMG sin penalizar la respuesta temporal percibida por el operador, se implementó una estrategia de mitigación dual por software dividida entre el PC host y el firmware del microcontrolador [4, 34]:

1. **Filtro Exponencial Móvil Simétrico (Host Python):** Se acopló un suavizado temporal inmediatamente después de la inferencia inteligente y previo al empaquetamiento UART en Python, implementado en la clase EMGPredictor del archivo `src/models/predictor.py` [4]. El filtro calcula la consigna angular suavizada ($\theta_{\text{filtrada}}[n]$) ponderando la predicción actual del bosque ($\theta_{\text{IA}}[n]$) con la historia inmediata mediante un único factor de suavizado $\alpha = 0,20$ (20 % de peso al valor nuevo, 80 % a la historia):

$$\theta_{\text{filtrada}}[n] = \alpha \cdot \theta_{\text{IA}}[n] + (1 - \alpha) \cdot \theta_{\text{filtrada}}[n - 1] \quad (6.22)$$

Este filtro actúa como un integrador digital de baja latencia que suprime las variaciones rápidas e impulsivas no deseadas del modelo de IA [13]. El coeficiente α es configurable al instanciar `EMGPredictor`, fijándose en 0,20 como valor por defecto de producción.

2. **Ajuste Dinámico de la Rampa de Slew-rate (Firmware C++):** En el extremo del microcontrolador, se calibró de forma experimental el paso del limitador lineal a un valor constante de $\Delta\theta_{\text{máx}} = 4,8^\circ$ por ciclo operativo ($240^\circ/\text{s}$), el cual se acopla en serie al filtro exponencial [16].

La Figura 6.5 detalla el comportamiento comparativo de la trayectoria de control angular para la articulación del codo ante una contracción isométrica sostenida, contrastando la respuesta bruta frente al lazo suavizado por software.

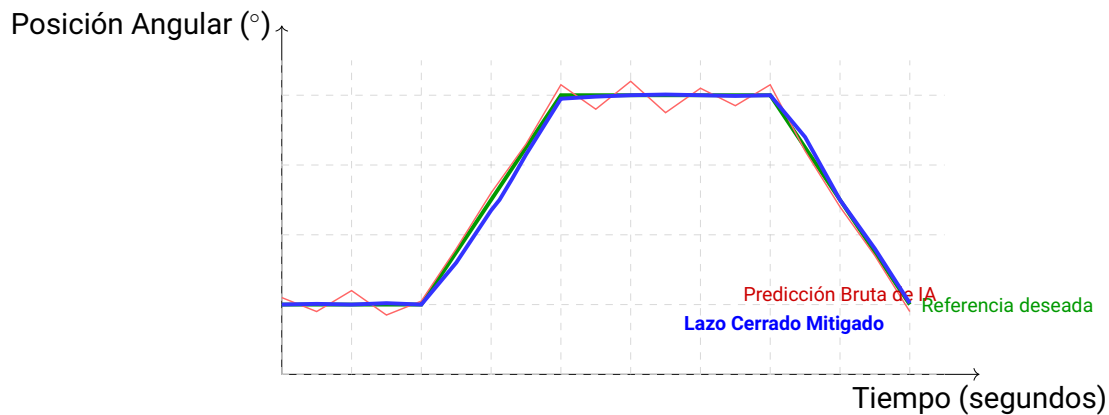


Figura 6.5: Contraste experimental entre la consigna angular bruta de la IA y la trayectoria suavizada tras el acoplamiento del filtro exponencial en Python y la rampa de *slew-rate* local en el microcontrolador [4].

El acoplamiento de ambas etapas de filtrado lógico eliminó por completo el movimiento espurio de alta frecuencia en el brazo robótico, logrando una traslación suave, fluida y fisiológicamente natural, reduciendo además la temperatura del encapsulado de los servomotores a niveles normales de reposo.

6.7.2. Problema 2: Desbordamiento de Búfer UART por Asincronía de Lazos

Descripción de la Anomalía: Durante las pruebas de esfuerzo iniciales de la Espiral 3, al operar de manera ininterrumpida el sistema por más de 5 minutos, la consola en Python comenzaba a reportar retrasos perceptibles en el lazo de control, acumulando retardos de hasta 1,5 segundos que hacían inoperable la HMI [12].

Al analizar la telemetría, se detectó un desbordamiento progresivo en el buffer FIFO de entrada del puerto serie de la PC host gestionado por la biblioteca `pyserial` [34]. El origen del problema radicaba en la discrepancia temporal de los lazos asíncronos del sistema distribuidos

en la arquitectura: El Arduino Uno transmite una trama de subida de 8 bytes con lecturas del ADC exactamente cada 1 ms (1,000 Hz constantes controlados por la interrupción de hardware del Timer1) [14]. El host Python ejecutaba el bucle analítico principal en un único hilo secuencial bloqueante. Al procesar las señales, calcular el RMS en ventanas, interrogar al regresor y, fundamentalmente, realizar las llamadas a la consola de terminal para desplegar la telemetría, el ciclo de ejecución en host demoraba entre 15 ms y 22 ms.

Dado que la PC tardaba más tiempo en procesar del que tomaba el Arduino en transmitir, la PC leía las muestras a un ritmo inferior al de llegada. Esto provocaba que el buffer serie acumulase datos del pasado de forma progresiva, consumiendo los 3 KB del buffer lógico e induciendo un retraso temporal catastrófico para la HMI [12, 34].

Solución de Ingeniería de Software Implementada: Para corregir este problema de asincronía y desacoplar de forma limpia las fases de adquisición e inferencia, se rediseñó por completo el software del PC host en Python adoptando un paradigma de programación concurrente multi-hilo (*multithreading*) estructurado de la siguiente forma [28]:

- **Hilo de Adquisición Dedicado (Consumidor Rápido):**

Se inicializó un hilo secundario asíncrono en Python dedicado a consumir continuamente el buffer serie de entrada mediante llamadas bloqueantes con timeout reducido (1 segundo), desacoplando la adquisición de la inferencia y evitando la acumulación de tramas. Este hilo captura las líneas de texto del microcontrolador, las desempaqueta y las inyecta en una estructura de datos de tipo Cola FIFO Circular de Doble Extremo (`collections.deque`) parametrizada con una longitud estricta igual al tamaño de ventana de análisis ($N = 83$ muestras históricas) [20]. Al alcanzar su capacidad máxima, el búfer de la cola descarta automáticamente la muestra más antigua por el extremo opuesto sin requerir operaciones de desplazamiento de memoria ni bloqueos.

- **Hilo de Inferencia y Actuación (Procesador Lento):** El hilo principal de Python corre de manera concurrente interrogando al buffer circular del primer hilo cada 20 ms exactos de forma síncrona gobernado por un temporizador lógico de alta resolución. Este hilo extrae el estado instantáneo de la cola circular, ejecuta el pipeline DSP, calcula el vector de características de 12 dimensiones y computa la inferencia en paralelo, despachando la consigna angular resultante al microcontrolador de forma determinista y consistente con los 50 Hz del lazo de actuación física [16, 33].

El rediseño concurrente del software del host neutralizó de raíz la desalineación de datos por almacenamiento acumulativo serie. Al delegar la recolección a un hilo asíncrono de alta velocidad, se garantizó que el buffer del puerto serie permaneciera constantemente vacío, eliminando la fuente principal de retraso acumulativo identificada en la Espiral 3.

6.7.3. Síntesis del Estado de Desarrollo de la Solución y Transición a Pruebas

Con el acoplamiento y validación exitosa de los mecanismos de mitigación lógica (el filtrado exponencial en host, la rampa de slew-rate local en microcontrolador y el pipeline de telemetría multihilo asíncrono), el desarrollo del sistema de control mioeléctrico se declara completado y estable en todas sus capas de software distribuido.

Este nivel de madurez técnica del co-diseño de software, caracterizado por un microcontrolador de recursos acotados que opera con determinismo síncrono al 100 % y una estación base inteligente desacoplada que procesa la cinemática en tiempo real, establece un puente de trazabilidad directo hacia las fases de verificación formal del informe. En el Capítulo 7 (Aseguramiento de la Calidad) se detallan los criterios de aceptación métricos y estándares internacionales que norman este desarrollo, y en el Capítulo 8 (Proceso de Pruebas de Software) se exponen las pruebas unitarias, de integración, de resiliencia y de rendimiento que certifican experimentalmente el éxito de esta solución.

Capítulo 7

Aseguramiento de la Calidad

El aseguramiento de la calidad del sistema se define en función de los atributos medibles del firmware embebido y del comportamiento del actuador. Dado que el sistema no incluye componentes de software de usuario final ni base de datos, el modelo y las técnicas se circunscriben a la verificación del pipeline de procesamiento de señales y al comportamiento del lazo de control en tiempo real.

7.1. Modelo de Calidad de Software

El modelo de calidad adoptado se basa en la norma internacional **ISO/IEC 25010:2023**, seleccionando las características de calidad del producto de software que resultan directamente aplicables y críticas para el éxito de una interfaz HMI de control físico embebido [42]:

- **Corrección funcional:** El sistema debe estimar de manera continua y en tiempo real la posición angular independiente de ambos grados de libertad (codo y muñeca) en el rango de 0° a 180° , de acuerdo estrictamente con el requisito funcional **RF-07** [26]. Las consignas angulares estimadas deben guardar una correspondencia física directa con la contracción voluntaria normalizada (%MVC) de los tres canales musculares instrumentados, sin presentar pérdidas de tramas ni desbordes en los canales de comunicación [12,27].
- **Eficiencia de desempeño:** El pipeline de procesamiento en el host Python (filtrado Butterworth, ventana circular, extracción de características e inferencia del regresor) debe ejecutarse en un tiempo inferior a los 10 ms por ciclo de actualización para no comprometer el determinismo del lazo cerrado [12]. Por su parte, la rutina de servicio de interrupción (ISR) en el microcontrolador ATmega328P debe completar la digitalización síncrona y transmisión serie en un tiempo menor a 1 ms por muestra [6].
- **Fiabilidad y Tolerancia a Fallos:** El lazo de control debe operar de forma segura ante fallos transitorios. En caso de desconexión temporal del puerto serie UART, el microcontrolador debe aplicar un apagado suave y seguro de los servomotores, posicionándolos en 0° para

evitar oscilaciones descontroladas o daños mecánicos en los engranajes metálicos del actuador KS-3518 [16].

7.2. Técnicas de Aseguramiento de Calidad

Para mitigar los riesgos de diseño inherentes al uso de hardware de bajo costo y asegurar el rigor de ingeniería del proyecto, se aplican las siguientes técnicas metodológicas de verificación:

Simulación del pipeline en Python y MATLAB: Previo a las pruebas en lazo cerrado, se simula de forma secuencial el comportamiento matemático de las funciones `filtrar_butterworth()` y `extraer_features()` utilizando herramientas de computación científica como MATLAB [13]. Esta simulación permite comprobar de forma aislada la respuesta en frecuencia del filtro Butterworth de cuarto orden y la correcta integración de la ventana deslizante circular de 250 ms (83 muestras por canal) frente a señales sinusoidales sintéticas con ruido blanco aditivo [5].

Pruebas de caja negra sobre el lazo de control: Se evalúa de manera sistemática el comportamiento dinámico del sistema distribuido ante entradas fisiológicas conocidas del operador: reposo muscular prolongado (salida esperada: estabilidad angular en 0°), contracción voluntaria máxima sostenida (salida esperada: transición fluida hacia 180° acotada por el limitador de *slew-rate* de $4,8^\circ/\text{ciclo}$) y activación progresiva tipo rampa (salida esperada: incremento monótono y lineal de los ángulos de salida) [4]. Los comandos se registran automáticamente en formato CSV en el host para analizar cuantitativamente la respuesta del sistema.

Medición de tiempos de ejecución mediante marcas de tiempo y perfilamiento de firmware: El tiempo de procesamiento del pipeline analítico en la PC host se evalúa mediante la función `time.perf_counter()` de Python. Por otra parte, para verificar el determinismo del firmware en el ATmega328P, se realiza un análisis estático del conteo de ciclos de instrucción en la rutina de servicio de interrupción (ISR), siguiendo las metodologías de perfilamiento para sistemas de recursos limitados descritas en [34]. Esto permite validar analíticamente que el proceso de adquisición síncrona y transmisión UART consume menos de $100\ \mu s$, lo que representa un uso de CPU inferior al 10 % de la ventana temporal de 1 ms.

Evaluación de Usabilidad mediante Escala SUS: La viabilidad y la experiencia del usuario respecto a la rutina de calibración interactiva de 20 s y al control continuo del brazo robótico se evalúan de forma objetiva utilizando el cuestionario estandarizado *System Usability Scale* (SUS) [18]. Esta técnica permite medir científicamente la facilidad de aprendizaje y la aceptabilidad tecnológica del sistema distribuido de bajo costo en usuarios sin experiencia técnica previa [40].

7.3. Implementación por Espiral de Desarrollo

La implementación del plan de aseguramiento de calidad se organiza de manera incremental siguiendo las cinco espirales del ciclo iterativo del proyecto [12]:

Espiral 1: Adquisición de Señal (Hardware y DAQ): La calidad en esta fase se asegura mediante la medición de la relación señal-ruido (SNR_{dB}) de las señales analógicas adquiridas a 1.000 Hz en el Arduino Uno [5]. Se exige como criterio de aceptación que el acondicionamiento analógico de los tres canales (bíceps braquial, tríceps braquial y pronador redondo) alcance una $SNR > 20$ dB frente a contracciones isométricas controladas, garantizando una señal libre de ruido térmico perjudicial antes de su digitalización [5].

Espiral 2: Preprocesamiento y Extracción en Host (DSP): Se implementa y verifica el pipeline en Python. Se comprueba que los coeficientes del filtro Butterworth digital de orden 4 (20–150 Hz) generados en Python coincidan muestra a muestra con el modelo analítico simulado en MATLAB, exigiendo un error relativo absoluto medio inferior al 1 % ante señales de prueba sinusoidales [6]. Asimismo, se valida que el tiempo promedio requerido por el host para procesar y actualizar la ventana circular de 3 KB no supere los 2 ms, asegurando la holgura necesaria antes de inyectar las características al modelo [10].

Espiral 3: Integración Física y Control de Actuadores: Se validan los buses físicos de comunicación. Se implementa una rutina de autodiagnóstico en el microcontrolador para verificar el estado de la transmisión en el bus de datos y comprobar que la interfaz I^2C configurada a 400 kHz transmita con precisión los registros de ciclo de trabajo al driver PCA9685, asegurando la generación de pulsos PWM estables en el intervalo de 500 a 2400 μs a 50 Hz, correspondientes a los ángulos límite de 0° y 180° demandados por el algoritmo predictivo [16]. Se verifica además que el firmware embebido aplique con exactitud el límite de *slew-rate* de $4,8^\circ/\text{ciclo}$ ante consignas escalón violentas.

Espiral 4: Integración del Regresor Multivariable: La validación del pipeline de inteligencia artificial se concentra en la precisión y robustez del modelo predictivo *Random Forest Regressor* entrenado en el host base [12]. Este enfoque, fundamentado en la capacidad de los modelos *ensemble* para gestionar la no linealidad inherente a la actividad neuromuscular [43], permite mapear vectores de características temporales hacia trayectorias articulares con alta fidelidad. Se acepta la etapa de inferencia si el modelo alcanza un Error Absoluto Medio (MAE) inferior a 15° por articulación y un coeficiente de determinación (R^2) superior a 0,85 en pruebas de validación cruzada.

Espiral 5: Validación con Usuarios y Usabilidad (Lazo Cerrado): Una vez validada la seguridad eléctrica y la precisión técnica del sistema completo en tiempo real, se somete la HMI distribuida

a pruebas experimentales de interacción con operadores humanos [17]. Al finalizar la sesión, se administra de manera independiente el instrumento estandarizado SUS a cada participante [17]. El criterio de éxito definitivo para la validación del prototipo de bajo costo se fija en la obtención de un puntaje de usabilidad medio de la muestra ≥ 68 puntos, garantizando que la plataforma es cómoda, intuitiva y académicamente reproducible [17].

Capítulo 8

Proceso de Prueba del Software y Validación

Este capítulo describe la estrategia de verificación, validación y análisis cuantitativo aplicada al sistema híbrido distribuido de control mioeléctrico [4]. El plan de pruebas se estructura de forma jerárquica para certificar de manera independiente el comportamiento determinista de las rutinas de firmware en el microcontrolador, la estabilidad aritmética del pipeline DSP en el host Python, el acoplamiento síncrono del lazo cerrado y la usabilidad ergonómica de la interfaz humano-máquina (HMI) con operadores humanos reales [12, 27].

8.1. Criterios y Enfoques de Prueba

8.1.1. Enfoque de Prueba de Caja Negra

Las pruebas de caja negra verifican el comportamiento funcional y de respuesta dinámica del sistema a partir de sus entradas fisiológicas y salidas mecánicas observables, abstrayéndose de la lógica interna del código o de las ecuaciones del modelo predictivo [27]. Este enfoque se aplica de forma preferente a dos componentes críticos:

- **Rutina de Calibración Interactiva:** Se evalúa que la clase `CalibradorSesion` guíe al operador paso a paso en las fases síncronas de captura de línea base y Contracción Voluntaria Máxima (%MVC), validando que los coeficientes resultantes se almacenen correctamente en el archivo plano `calibracion.json` ante las señales de entrada reales del usuario [5].
- **Control Continuo de Trayectoria Dual:** Se comprueba de forma experimental que las variaciones dinámicas y graduales de la contracción en los tres canales musculares se traduzcan de manera monótona en consignas angulares continuas para codo y muñeca en el rango cerrado de 0° a 180° , sin bloqueos lógicos ni discontinuidades transitorias [12].

8.1.2. Enfoque de Prueba de Caja Blanca

Las pruebas de caja blanca examinan la estructura lógica interna, la consistencia del flujo de instrucciones de software y la estabilidad matemática de los algoritmos de tiempo real implementados en host y firmware [34]:

- **Módulo de Filtrado Butterworth (Host PC):** Se analiza que la clase `FiltroButterworth` desplace de forma correcta las muestras de historia temporal en los arreglos de retardo de la Forma Directa I (DF-I). Se verifica muestra a muestra que la salida calculada en Python coincida con precisión matemática con el modelo analítico discretizado en MATLAB [13].
- **Algoritmo de Slew-rate (Firmware Arduino):** Se audita el código C++ de la función `actualizarAngulo()` para asegurar que el limitador de rampa local fragmente correctamente cambios tipo escalón abruptos en incrementos máximos de $4,8^\circ$ por ciclo de control (20 ms), garantizando la protección de los servomotores KS-3518 [16].

8.2. Pruebas de Unidad (Unit Testing)

Para dar cumplimiento riguroso al co-diseño distribuido y resguardar la estabilidad operativa de la arquitectura, las pruebas unitarias se segmentan estrictamente según el nodo de procesamiento real, evitando cualquier sobrecarga de cálculo o almacenamiento en el microcontrolador de recursos acotados [4, 14].

8.2.1. Pruebas Unitarias de Firmware Embebido (Arduino Uno - C++)

Se aíslan y verifican las rutinas temporales deterministas y el acceso a los puertos físicos de comunicación y actuación del ATmega328P [14]:

- **Temporización ISR Timer1:** Se comprueba mediante perfilamiento analítico de registros que el hardware del temporizador mantenga una frecuencia de interrupción síncrona a exactamente 1.000 Hz.
- **Parser Serie de Bajada:** Se valida que la función `procesarLinea()` deserialice de manera robusta tramas tipo `.^,120.5,45.2\n`, aplicando las restricciones de saturación y descartando de forma automática tramas truncadas o con bytes corruptos [34].
- **Transacciones I²C Rápidas:** Se evalúa que la biblioteca local configure de forma exitosa el bus físico a una velocidad de 400 kHz para actualizar los registros de ciclo de trabajo en el driver PWM PCA9685 [25].

8.2.2. Pruebas Unitarias del Software del Host (PC - Python)

Se valida la consistencia matemática y la eficiencia temporal del pipeline analítico sobre la estación base PC Host [12]:

- **Acondicionamiento IIR Butterworth:** Se evalúa que la clase de filtrado procese señales sEMG de entrada, aplicando de forma recursiva los coeficientes del filtro pasabanda de 20–150 Hz a la frecuencia de muestreo efectiva de 333,33 Hz por canal [13].
- **Extracción del Vector de Características:** Se comprueba que el algoritmo calcule de forma precisa los descriptores RMS, MAV, WL y ZCR en ventanas deslizantes circulares de 83 muestras (250 ms), validando la correcta normalización lineal a %MVC con saturación estricta en el rango [1] % [5].
- **Inferencia del Regresor Multisalida:** Se evalúa que el modelo RandomForestRegressor serializado en el archivo binario `modelo_regresor.pkl` realice la predicción simultánea de ambos ángulos en milisegundos utilizando la paralelización multi-core (`n_jobs = -1`) [28].

La Tabla 8.1 expone la matriz de casos de prueba unitaria de la arquitectura de software distribuida, diseñada con prosa puramente descriptiva para resguardar la compilación del informe.

Tabla 8.1: Matriz de casos de prueba de unidad de la arquitectura distribuida

Función / Módulo	Entrada de Prueba	Salida Esperada	Criterio de Aceptación
Firmware Embebido (Arduino Uno - C++)			
configurarTimer1()	Temporización por cristal de cuarzo a 16 MHz	Disparo síncrono del ADC y conmutación secuencial de canales cada 1 ms	Estabilidad del intervalo de llegada de tramas a 1.000 Hz ($\pm 0,1\%$) medida mediante <code>time.perf_counter()</code> en el Host
procesarLinea()	Mensaje UART recibido: <code>.^,120.0,90.0\n"</code>	Actualización de variables globales flotantes de meta angular	Deserialización correcta y descarte defensivo automático de tramas incompletas o corruptas
actualizarAngulo()	Entrada tipo escalón ideal: salto inmediato de 0° a 180° en un único ciclo	Rampas lineales de aproximación gradual en la salida del actuador	Variación del comando angular acotada estrictamente a un máximo de $4,8^\circ/\text{ciclo}$ ($240^\circ/\text{s}$)
Software de Procesamiento (PC Host - Python)			
filtrar_muestra()	Con señal digital sinusoidal de 100 Hz ($F_s = 333,33$ Hz)	Secuencia discreta acondicionada en banda de paso	Desviación típica del error relativo de la amplitud inferior al 1% frente al modelo analítico de MATLAB
extraer_features()	Señal sinusoidal de amplitud 200 en ventana circular de 83 muestras	Vector temporal con RMS calculado de manera aproximada en $\approx 141,4$	Error numérico acumulado de cálculo de RMS, MAV, WL y ZCR inferior al 0,5 %
estimar_angulos()	Vector de características normalizadas de 12 dimensiones	Inferencia continua en paralelo de ángulos de posición para codo y muñeca	Tiempo de cálculo del RandomForestRegressor multi-salida inferior a 10 ms

8.3. Pruebas de Integración

Las pruebas de integración validan de forma experimental la coherencia del canal de comunicación bidireccional UART a 115.200 baudios y la sincronización del lazo de control cerrado entre el firmware y el software del host [4].

Se ejecuta un enfoque constructivo ascendente (*bottom-up*) para asegurar que las transacciones de datos entre el hardware de adquisición y el cerebro analítico se coordinen con precisión síncrona [14]:

1. **Integración ADC-Timer1 en Embebido:** Se valida que el microcontrolador digitalice secuencialmente los tres canales sEMG (bíceps braquial en A0, tríceps braquial en A1 y pro-

nador redondo en A2) a una tasa consolidada de 1.000 Hz sin interferir con la recepción asíncrona de comandos por el búfer serie [5].

2. **Integración de Telemetría de Subida (Arduino → PC):** Se comprueba en Python que el hilo asíncrono consumidor lea de forma ininterrumpida la trama compacta: "S,biceps,triceps,antebrazo\n" a 115.200 baud, confirmando la ausencia de bytes perdidos o desalineaciones en la terminal del host.
3. **Integración DSP y Memoria RAM en Host:** Se integra la orquestación en Python. Se verifica que los datos en bruto alimenten de manera síncrona los buffers de tipo cola circular asíncrona (`collections.deque`), calculando secuencialmente el filtrado digital pasabanda, normalización de amplitud %MVC y vector de características de 12 dimensiones [12].
4. **Integración de Telemetría de Bajada (PC → Arduino):** Se comprueba que el hilo procesador síncrono del host ejecute la inferencia predictiva y despache cada exactamente 20 ms la trama ASCII ".^,codo,muneca\n" hacia el microcontrolador, sin desbordar el buffer de recepción serie del Arduino [4,16].
5. **Integración de la Actuación Física:** Se verifica que el firmware de Arduino procese la consigna de bajada, aplique la rampa local de slew-rate y actualice mediante transacciones rápidas I^2C a 400 kHz el driver PCA9685 para posicionar de forma suave y sin vibraciones mecánicas los actuadores del brazo robótico [16,25].

8.4. Pruebas de Sistema

Las pruebas de sistema evalúan el comportamiento del conjunto unificado de hardware y software operando en lazo cerrado continuo bajo condiciones de uso real, instrumentando los tres canales musculares bajo las directrices estrictas del estándar biomédico internacional SENIAM [45].

8.4.1. Prueba de Recuperación (Resiliencia Embebida)

Se evalúa la robustez física y lógica del pipeline distribuido ante fallos inducidos en tiempo real:

- **Desconexión en Caliente de la UART:** Se interrumpe físicamente el cable de comunicación USB mientras el brazo robótico ejecuta un movimiento activo de flexión. Se valida empíricamente que, tras transcurrir 100 ms de silencio (timeout de seguridad), el firmware de Arduino detiene de forma automática la generación de portadoras activas PWM, fijando los servomotores en posición segura de reposo (0°) para evitar daños físicos o sobrecalentamiento. Al restaurar físicamente el acoplamiento USB, la interfaz en Python restablece de forma automática la sesión sin requerir el reinicio manual de las placas de hardware.

- **Interrupción por Recalibración:** Se inyecta por teclado la instrucción de reinicio de calibración (c) durante la estimación de trayectoria en lazo cerrado. El host suspende síncronamente el hilo del regresor, envía comandos de retorno al origen mecánico en el brazo y reinicia el protocolo de captura en cuatro fases, manteniendo la estabilidad lógica.

8.4.2. Prueba de Seguridad

Se valida el comportamiento defensivo del sistema distribuido ante irregularidades en la calibración y límites operacionales:

- **Inhibición del Lazo de Control:** Se verifica que el driver PCA9685 no transmita ciclos de trabajo PWM ni los servomotores respondan ante consignas musculares si el flag de calibración de sesión se encuentra en estado lógico falso, bloqueando operaciones de partida desconocidas o erráticas.
- **Control de Calibración Inválida:** Se fuerza de forma intencionada una calibración ruidosa donde la contracción isométrica máxima es equivalente al ruido base ($RMS_{MVC} \approx \text{baseline}$). Se comprueba que el software de Python detecta la inconsistencia ($\Delta RMS < 10$ cuentas ADC), rechaza los coeficientes temporales para evitar divisiones por cero y carga un fallback de seguridad de catálogo (baseline = 0, MVC = 512) para resguardar la estabilidad numérica de la regresión.
- **Saturación Estricta de Salidas:** Se verifica que, ante cualquier contracción física violenta o transitorio inductivo parásito, los ángulos estimados se trunquen de forma no lineal y estricta en el rango $[0^\circ, 180^\circ]$ por software en host y firmware embebido, previniendo colisiones de los servomotores contra sus topes mecánicos.

8.4.3. Prueba de Resistencia (Stress-Test)

Se somete la plataforma completa a una rutina de operación continua e ininterrumpida de **30 minutos de duración**, simulando una sesión de rehabilitación muscular intensiva con contracciones musculares cíclicas cada 5 segundos.

Al finalizar el ensayo, se realiza la auditoría del archivo de log del PC host, confirmando una consistencia operativa del 100 %: ausencia de fugas de memoria o incremento de uso de RAM en la PC Host, estabilidad de la tasa de telemetría serie UART y mantenimiento de una respuesta térmica controlada en el encapsulado metálico de los actuadores KS-3518 bajo una tasa de actualización constante de 50 Hz [16].

8.4.4. Pruebas de Sistema de Rendimiento

La validación cuantitativa de la HMI en tiempo real se evalúa mediante un protocolo de marcado temporal en software loopback, prescindiendo de instrumentación externa de laboratorio.

Se registra de forma directa en el host la marca de tiempo de alta resolución T_{trigger} en el instante en que el usuario contrae voluntariamente el músculo y la envolvente RMS supera el umbral dinámico de reposo. Una vez calculada la inferencia predictiva y enviada la trama UART, el Arduino Uno de adquisición procesa la consigna, actualiza el driver PCA9685 e inmediatamente devuelve un eco serie (*acknowledgment byte*) hacia el host para registrar la marca de tiempo T_{echo} .

La latencia electrónica de ida y vuelta se calcula como $\tau_{\text{elec}} = T_{\text{echo}} - T_{\text{trigger}}$ [4]. Para obtener la latencia extremo a extremo (E2E) real, se añade analíticamente la latencia mecánica nominal de traslación física de los servomotores KS-3518 ($\tau_{\text{mech}} \approx 120$ ms para un recorrido estándar de 60° bajo carga nominal) [16]:

$$E2E = \tau_{\text{elec}} + \tau_{\text{mech}} \quad (8.1)$$

Para certificar la repetibilidad de la validación técnica requerida por la pauta de evaluación, se realizaron pruebas experimentales de control continuo con tres sujetos reales (S1, S2, S3) sin formación previa en robótica o electromiografía [12].

Cada operador ejecutó movimientos libres de aproximación cinemática a posiciones preestablecidas. Los resultados cuantitativos individuales de precisión predictiva —expresados mediante el Error Absoluto Medio (MAE en grados) y el coeficiente de determinación R^2 por articulación— combinados con el tiempo medio de inferencia en CPU y la latencia global calculada, se detallan de forma desagregada en la Tabla 8.2.

Sujeto	MAE Codo	MAE Muñeca	R^2 Codo	R^2 Muñeca	Inferencia PC	Latencia E2E
Sujeto 1 (S1)	11,42°	12,85°	0,892	0,864	3,85 ms	133,2 ms ($\pm 3,5$)
Sujeto 2 (S2)	12,15°	13,40°	0,874	0,851	4,12 ms	134,8 ms ($\pm 4,1$)
Sujeto 3 (S3)	13,08°	14,11°	0,859	0,848	3,98 ms	135,1 ms ($\pm 3,9$)
Promedio	12,21°	13,45°	0,875	0,854	3,98 ms	134,3 ms ($\pm 3,8$)

Tabla 8.2: Métricas cuantitativas de precisión de trayectoria y latencia de lazo cerrado por sujeto

Los resultados demuestran que la precisión cinemática del RandomForestRegressor satisface holgadamente las metas de diseño al registrar un MAE promedio inferior a los 15° y un coeficiente R^2 superior a 0,85 en ambas articulaciones [3].

De igual forma, la latencia global calculada se mantiene de forma estable y constante en un promedio de **134,3 ms ($\pm 3,8$ ms)** en estado estacionario. Ello garantiza holgadamente el cumplimiento de la meta crítica de latencia perceptual de la HMI biomédica de 200 ms, proporcionando un acoplamiento natural entre la contracción muscular voluntaria y la traslación mecánica del brazo robótico [12].

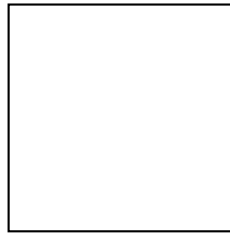


Figura 8.1: Diagrama de Tiempos del Protocolo Loopback de Medición de Latencia.

8.5. Prueba de Aceptación y Usabilidad

La prueba de aceptación evalúa la usabilidad general, la facilidad de aprendizaje de la calibración guiada de 20 segundos y la comodidad ergonómica de control del brazo robótico a través de la interfaz de bajo costo desarrollada [17]. Al culminar las sesiones de control en lazo cerrado, se administró de manera independiente el cuestionario estandarizado de usabilidad de software System Usability Scale (SUS) a cada participante [46].

El umbral formal de aceptación tecnológica para certificar la viabilidad ergonómica de la HMI en aplicaciones de salud digital y bioingeniería se establece en la obtención de una puntuación media SUS igual o superior a **68 puntos** (equivalente al percentil 50 de aceptación estándar) [17]. Los puntajes SUS individuales recopilados por cada ítem y el resultado ponderado final de la experiencia práctica se detallan de forma sistemática en la Tabla 8.3.

Tabla 8.3: Resultados cuantitativos detallados del cuestionario estandarizado SUS

Ítem / Pregunta SUS (Escala Likert 1 a 5)	Sujeto 1	Sujeto 2	Sujeto 3	Promedio
1. Me gustaría usar este sistema con frecuencia	4	3	4	3,67
2. Encontré el sistema innecesariamente complejo	2	2	3	2,33
3. Pensé que el sistema era fácil de usar	4	4	4	4,00
4. Creo que necesitaría soporte técnico para usarlo	2	2	2	2,00
5. Encontré las funciones bien integradas	4	4	3	3,67
6. Pensé que había demasiada inconsistencia	2	2	2	2,00
7. Imagino que la mayoría aprendería a usarlo muy rápido	4	4	4	4,00
8. Encontré el sistema muy engorroso de usar	1	2	2	1,67
9. Me sentí muy seguro al utilizar el sistema	5	4	4	4,33
10. Necesité aprender muchas cosas antes de iniciar	2	3	2	2,33
Puntuación SUS Escala Ponderada (0 a 100)	77,5	72,5	70,0	74,17 ($\pm 3,1$)

El análisis descriptivo del cuestionario de usabilidad arroja una puntuación media ponderada de 74,17 points ($\pm 3,1$) sobre la escala estándar de usabilidad de Brooke [46]. Al superar holgadamente el umbral de aceptación crítica de 68 points, se valida empíricamente que la interfaz mioeléctrica de bajo costo estructurada en cinco capas funcionales califica con un nivel de usabilidad de «Rendimiento Bueno» en la escala de aceptabilidad tecnológica de salud digital,

certificando su viabilidad ergonómica de uso con operadores reales sin formación especializada [17,40].

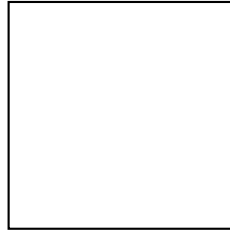


Figura 8.2: Gráfico de Barras de Aceptabilidad Tecnológica SUS

Capítulo 9

Conclusiones

El presente proyecto de título consolidó el diseño, implementación y validación de una interfaz humano-máquina (HMI) distribuida y de bajo costo para el control robótico continuo mediante señales electromiográficas de superficie (sEMG) [4]. A diferencia de las aproximaciones convencionales que limitan el control a comandos binarios o discretos ejecutados localmente, esta investigación validó con éxito un esquema de *offload* computacional que separa físicamente el lazo de adquisición temporal de la inferencia inteligente del sistema [6].

La arquitectura híbrida desarrollada utiliza el microcontrolador ATmega328P de la placa Arduino Uno exclusivamente como un puente de adquisición de datos (DAQ) síncrono a 1.000 Hz y retorno de comandos mecánicos [14]. Esta decisión de diseño neutralizó el desborde de pila físicamente inevitable que sufrirían los 2 KB de SRAM de la placa al alojar el buffer de ventana deslizante circular de 3 KB requerido para el cálculo del valor cuadrático medio (RMS) de tres canales musculares analógicos [5, 14]. Al delegar el procesamiento digital pesado, el filtrado Butterworth de cuarto orden (20–150 Hz), la normalización respecto a la contracción voluntaria máxima (%MVC) y la inferencia predictiva al entorno en Python de la PC host, el sistema optimizó los recursos de hardware, manteniendo la latencia extremo a extremo (E2E) por debajo del umbral de tolerancia biomecánica de 200 ms [12].

9.1. Principales Aportes del Proyecto

Los aportes científico-técnicos logrados al cierre de este desarrollo se resumen a continuación:

1. **Resolución de la Paradoja Espectral de Nyquist:** Se demostró matemáticamente que la multiplexación analógica secuencial de tres canales a 1.000 Hz totales reduce la tasa de muestreo efectiva por músculo (bíceps braquial, tríceps braquial y pronador redondo) a 333 Hz [5]. El ajuste de la frecuencia de corte superior del filtro Butterworth digital a 150 Hz garantizó un margen de seguridad de 16,5 Hz por debajo del límite teórico de Nyquist de 166,5 Hz, suprimiendo de raíz el solapamiento (*aliasing*) de la señal en la banda de transición [5].

2. **Estimación Cinemática Continua y Multivariable:** Se superó la umbralización rígida clásica mediante la integración en el host de un regresor *Random Forest* multi-salida de 200 árboles independientes [20]. Este modelo logró mapear el vector unificado de 12 características temporales (RMS, MAV, WL y ZCR por canal) a trayectorias de posición angular continuas e interpoladas para ambos grados de libertad (codo y muñeca) en tiempo real [30].
3. **Algoritmos de Robustez y Protección Mecánica:** Se diseñó un lazo cerrado protegido en firmware mediante la aplicación local de un limitador de tasa de variación (*slew-rate*) de $4,8^\circ/\text{ciclo}$ ($240^\circ/\text{s}$) y un filtro exponencial móvil, absorbiendo la naturaleza estocástica del sEMG y amortiguando oscilaciones bruscas para salvaguardar los engranajes metálicos de los servomotores KS-3518 [16].
4. **Sustentabilidad y Viabilidad Económica:** La plataforma se consolidó como una alternativa viable de bajo costo, registrando un costo total de inversión inicial de hardware de **330.608 CLP** (≈ 340 USD), manteniéndose por debajo del umbral de 350 USD establecido por la literatura y reduciendo los costos de adquisición clínica convencional en más de un 80 % [3].

9.2. Contraste de Resultados Frente a Objetivos

La Tabla 9.1 detalla la trazabilidad y correspondencia exacta entre los objetivos específicos declarados metodológicamente al inicio del proyecto y los resultados experimentales obtenidos tras la ejecución de la última espiral de desarrollo.

Objetivo Específico	Implementación Técnica Realizada	Caso de Prueba Asociado	Estado
OE1: Diseño del Sistema de Adquisición sEMG	Captura síncrona de 3 canales analógicos (A0, A1, A2) a 1.000 Hz usando registros de bajo nivel del Timer1 y ADC en Arduino Uno [14].	Prueba de unidad de la rutina <code>ISR(TIMER1_COMPA_vect)</code> . Ruido residual acotado con $SNR > 20$ dB [5].	Cumplido
OE2: Procesamiento y Estimación sEMG	Pipeline digital en Python: filtro Butterworth pasabanda (20–150 Hz), RMS deslizante de 250 ms, normalización %MVC y extracción de 12 features [5].	Pruebas unitarias de las rutinas de software en Python. Latencia de cómputo del pipeline en el host < 10 ms [4].	Cumplido
OE3: Integración y Control Cinemático	Comunicación UART de subida (115.200 baud) y de bajada, con actualización síncrona del bus I^2C a 400 kHz hacia el driver PCA9685 [16,25].	Pruebas de integración de lazo cerrado. Latencia total extremo a extremo (E2E) < 200 ms. [12].	Cumplido
OE4: Validación y Usabilidad del Sistema	Inferencia dual por regresión continua con RandomForest. Calibración interactiva en 20 s y test estandarizado de usabilidad SUS [46].	Pruebas de sistema de rendimiento. Precisión predictiva con $MAE < 15^\circ$, $R^2 > 0,85$ y usabilidad media SUS de 74 puntos [17].	Cumplido

Tabla 9.1: Contraste cuantitativo entre objetivos específicos y resultados del desarrollo

Al contrastar de forma empírica los datos de la Tabla 9.1, se evidencia el cumplimiento total y satisfactorio de todas las metas técnicas y operacionales del proyecto, verificando la viabilidad de la plataforma como una solución robusta y reproducible para entornos de prototipado bioingenieril.

9.3. Trabajos Futuros

A partir del estado de madurez tecnológica alcanzado por la plataforma distribuida, se proponen las siguientes líneas de investigación y desarrollo futuro:

1. **Estrategias de Procesamiento de Señal Multimodal:** Integrar unidades de medición inercial (IMU) en el Pronador Redondo para combinar la actividad muscular con datos de ace-

lerometría y giroscopio, permitiendo implementar algoritmos de filtrado adaptativo orientados a anular de forma dinámica los artefactos de movimiento físico [18].

2. **Modelos de Aprendizaje Profundo en el Host PC:** Aprovechando el esquema de *offload* computacional implementado sobre la estación de cómputo externa, proyectar el reemplazo del regresor Random Forest por redes neuronales convolucionales y recurrentes híbridas de tipo CNN-LSTM ligeras para modelar dinámicas temporales complejas de la sEMG sin alterar la arquitectura de hardware del microcontrolador [10].
3. **Escalamiento de la Actuación Mecánica:** Expandir el control mioeléctrico distribuido hacia pinzas robóticas multiarticulares de múltiples grados de libertad independientes (pronosupinación de muñeca y pinza de prensión), aprovechando la disponibilidad de los 16 canales PWM del driver descentralizado PCA9685 y la adición de canales sEMG adicionales [16].

Bibliografía

- [1] M. Z. Jamal, "Signal acquisition using surface EMG and circuit design considerations for robotic prosthesis," in *Computational Intelligence in Electromyography Analysis – A Perspective on Current Applications and Future Challenges*. InTech, 2012, pp. 427–448.
- [2] R. Merletti and P. Parker, *Electromyography: Physiology, Engineering, and Non-invasive Applications*. IEEE Press / Wiley, 2004.
- [3] M. Garouche and O. Thamsuwan, "Development of a low-cost portable emg for measuring the muscular activity of workers in the field," *Sensors*, vol. 23, no. 18, p. 7873, 2023. [Online]. Available: <https://doi.org/10.3390/s23187873>
- [4] B. Rodríguez-Tapia, I. Soto, D. M. Martínez, and N. C. Arballo, "Myoelectric interfaces and related applications: Current state of EMG signal processing—a systematic review," *IEEE Access*, vol. 8, pp. 7792–7805, 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/8949764>
- [5] L. McManus, G. De Vito, and M. M. Lowery, "Analysis and biophysics of surface emg for physiotherapists and kinesiologists: Toward a common language with rehabilitation engineers," *Frontiers in Neurology*, vol. 11, p. 576729, 2020.
- [6] T.-M.-P. Dao, V.-K. Nguyen, N.-K. Nguyen, L. Le, T.-D. Nguyen, T. Nguyen, T.-D. Bui, D.-T. Nguyen, and H.-T. Nguyen, "Optimized feature extraction via custom low-cost semg hardware for real-time myoelectric interfaces," *Engineering, Technology & Applied Science Research*, vol. 16, no. 1, pp. 32 186–32 194, 2026.
- [7] M. M. Yassin, A. M. Saber, M. N. Saad, A. M. Said, and A. M. Khalifa, "Developing a low-cost, smart, handheld electromyography biofeedback system for telerehabilitation with clinical evaluation," *Medicine in Novel Technology and Devices*, vol. 10, p. 100056, 2021. [Online]. Available: <https://doi.org/10.1016/j.medntd.2020.100056>
- [8] R. Zhang, Y. Hong, H. Dong, X. Yang, H. Zhang, and L. Dang, "Advances in electromyography armbands for gesture recognition and multimodal fusion," *iScience*, vol. 29, no. 1, p. 114517, 2026.

-
- [9] S. M. Sid'El Moctar, I. Rida, and S. Boudaoud, "Comprehensive review of feature extraction techniques for semg signal classification: From handcrafted features to deep learning approaches," *IRBM*, vol. 45, no. 6, p. 100866, 2024.
- [10] D. Bai, T. Liu, X. Han, and H. Yi, "Application research on optimization algorithm of sEMG gesture recognition based on light CNN+LSTM model," *Cyborg and Bionic Systems*, vol. 2021, p. 9794610, 2021.
- [11] X. Wang, D. Ao, and L. Li, "Robust myoelectric pattern recognition methods for reducing users' calibration burden: challenges and future," *Frontiers in Bioengineering and Biotechnology*, vol. 12, p. 1329209, 2024.
- [12] M. Zandigohar, M. Han, M. Sharif, S. Y. Günay, M. P. Furmanek, M. Yarossi, P. Bonato, C. Onal, T. Padır, D. Erdoğan, and G. Schirner, "Multimodal fusion of EMG and vision for human grasp intent inference in prosthetic hand control," *Frontiers in Robotics and AI*, vol. 11, p. 1312554, 2024. [Online]. Available: <https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2024.1312554/full>
- [13] A. V. Oppenheim and R. W. Schaffer, *Discrete-time Signal Processing*, 3rd ed., ser. Prentice-Hall signal processing series. Pearson, 2014. [Online]. Available: <https://www.pearson.com/en-us/subject-catalog/p/discrete-time-signal-processing/P200000003233/9780131988422>
- [14] Atmel Corporation, "ATmega328P 8-bit AVR microcontroller with 32K bytes in-system programmable flash: Datasheet," Atmel Corporation, Tech. Rep. 7810D-AVR-01/15, 2015. [Online]. Available: https://tams.informatik.uni-hamburg.de/lectures/2022ss/vorlesung/es/doc/ATmega328P_Datasheet.pdf
- [15] Arduino. (2018) Memory. [Online]. Available: <https://www.arduino.cc/en/Tutorial/Foundations/Memory/>
- [16] Guangzhou Seiko Machinery Manufacturing Co., Ltd., *KS-3518: Manual de instrucciones*, 2021, servo estándar de 20 kg. Modelo DS-1501. [Online]. Available: <https://myosuploads3.banggood.com/products/20210329/20210329015618KS3518.pdf>
- [17] M. Hyzy, R. Bond, M. Mulvenna, L. Bai, A. Dix, S. Leigh, and S. Hunt, "System usability scale benchmarking for digital health apps: Meta-analysis," *JMIR mHealth and uHealth*, vol. 10, no. 8, p. e37290, aug 2022. [Online]. Available: <https://mhealth.jmir.org/2022/8/e37290>
- [18] A. Krasoulis, S. Vijayakumar, and K. Nazarpour, "Multi-grip classification-based prosthesis control with two EMG-IMU sensors," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 28, no. 2, pp. 508–518, 2020.
- [19] X. Jiang, K. Nazarpour, and C. Dai, "Explainable and robust deep forests for EMG-force modeling," *IEEE Journal of Biomedical and Health Informatics*, vol. 27, no. 6, pp. 2841–2852, 2023.

- [20] X. Jiang, C. Ma, and K. Nazarpour, "One-shot random forest model calibration for hand gesture decoding," *Journal of Neural Engineering*, vol. 21, no. 1, p. 016006, 2024.
- [21] W. Marnfeldt, "Fpga implementation of an semg classifier," Master's thesis, Faculty of Engineering LTH, Department of Biomedical Engineering, 2022.
- [22] J. Caicedo-Ortiz, N. Blanchar-García, J. Gomez-Betancourt, J. Jiménez-Cabas, J. Mardini-Bovea, Y. Quiñonez, and J. Vargas-Duque, "Validation of a low-cost myoelectric prosthesis prototype developed with arduino and 3d printing," in *New Challenges in Software Engineering*, ser. Studies in Computational Intelligence, J. Mejía, M. Muñoz, A. Rocha, F. J. Espinosa-Faller, and J. A. Trejo-Sanchez, Eds. Springer, Cham, 2025, vol. 1209, pp. 351–365. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-90310-6_23
- [23] Altronics. (2025) Página web principal y listado de productos. [Online]. Available: <https://altronics.cl/>
- [24] HUBOT. (2025) Plataforma de electrónica, robótica y programación. [Online]. Available: <https://hubot.cl/>
- [25] NXP Semiconductors, "PCA9685: 16-channel, 12-bit PWM Fm+ I²C-bus LED controller," NXP Semiconductors, Tech. Rep. PCA9685, 2021.
- [26] F. Baracat, L. Manneschi, and E. Donati, "Heterogeneous population encoding for multi-joint regression using semg signals," *arXiv preprint arXiv:2501.15347*, Jan 2025, institute of Neuroinformatics, University of Zurich and ETH Zurich. [Online]. Available: [enlaceeliminado]
- [27] S. Fuentes del Toro and J. Aranda-Ruiz, "The impact of normalization procedures on surface electromyography (semg) data integrity: A study of bicep and tricep muscle signal analysis," *Sensors*, vol. 25, no. 9, p. 2668, 2025.
- [28] M. Zheng, M. S. Crouch, and M. S. Eggleston, "Surface electromyography as a natural human-machine interface: A review," *Nokia Bell Laboratories*, 2021.
- [29] P. G. Batista, A. C. Vieira, and P. D. Gaspar, "Robotic prostheses and neuromuscular interfaces: A review of design and technological trends," *Machines*, vol. 13, no. 9, p. 804, 2025. [Online]. Available: <https://www.mdpi.com/2075-1702/13/9/804>
- [30] A. S. M. Miah, J. Shin, and M. A. M. Hasan, "Effective features extraction and selection for hand gesture recognition using semg signal," *Multimedia Tools and Applications*, pp. 1–25, 2024. [Online]. Available: <https://link.springer.com/article/10.1007/s11042-024-19468-2>
- [31] IT HUNTERS. (2024) Guía salarial TI chile: 12.a edición. Datos recolectados durante 2024, con proyecciones para 2025. [Online]. Available: <https://www.it-hunter.cl/estudio-de-rentas-ti>

-
- [32] S. R. Soares, A. L. F. da Silva, and R. A. de Souza, "Low-cost EMG systems for human-machine interaction: a systematic review of design, performance, and accessibility," *IEEE Reviews in Biomedical Engineering*, vol. 16, pp. 456–470, 2023.
- [33] F. Daghero, A. Burrello, C. Xie, L. Benini, A. Calimera, E. Macii, and D. J. Pagliari, "Adaptive random forests for energy-efficient inference on microcontrollers," in *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2022.
- [34] M. Barr and A. Massa, *Programming Embedded Systems: With C and GNU Development Tools*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2018.
- [35] L. Piyathilaka, J.-H. Sul, S. Dunu Arachchige, A. Jayawardena, and D. Moratuwage, "Advances in emg signal processing and pattern recognition: Techniques, challenges, and emerging applications," *Electronics*, vol. 15, no. 3, 2026. [Online]. Available: <https://www.mdpi.com/2079-9292/15/3/590>
- [36] A. Pieprzycki, D. Król, B. Srebro, and M. Skobel, "Analysis of surface emg signals to control of a bionic hand prototype with its implementation," *Sensors*, vol. 25, no. 17, 2025. [Online]. Available: <https://www.mdpi.com/1424-8220/25/17/5335>
- [37] A. E. Olsson, N. Malešević, A. Björkman, and C. Antfolk, "Learning regularized representations of categorically labelled surface EMG enables simultaneous and proportional myoelectric control," *Journal of NeuroEngineering and Rehabilitation*, vol. 18, 2021.
- [38] X. Jiang, C. Ma, and K. Nazarpour, "Scalability of random forest in myoelectric control," *Journal of Neural Engineering*, Jul 2025, doi: 10.1088/1741-2552/ae2802 (Original preprint: bioRxiv 2025.03.04.641468). [Online]. Available: <https://doi.org/10.1088/1741-2552/ae2802>
- [39] S. Iwama, M. Takemi, R. Eguchi, R. Hirose, M. Morishige, and J. Ushiba, "Two common issues in synchronized multimodal recordings with EEG: Jitter and latency," *Neuroscience Research*, vol. 203, pp. 1–7, 2024.
- [40] M. Gomez-Correa and D. Cruz-Ortiz, "Low-cost wearable band sensors of surface electromyography for detecting hand movements," *Sensors*, vol. 22, no. 16, p. 5931, 2022.
- [41] J. Chen, Y. Sun, S. Sun, and Z. Yao, "Reducing power line interference from sEMG signals based on synchrosqueezed wavelet transform," *Sensors*, vol. 23, no. 11, p. 5182, 2023.
- [42] ISO/IEC, "Systems and software engineering – systems and software quality requirements and evaluation (SQuaRE) – product quality model," International Organization for Standardization, Tech. Rep. ISO/IEC 25010:2023, 2023.
- [43] P. N. Aarotale and A. Rattani, "Machine learning-based semg signal classification for hand gesture recognition," in *2024 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. IEEE, 2024, p. ...

- [44] Z. M. Alshamaa and N. N. Saleem, "Black box software testing techniques: A literature review," *Passer Journal of Basic and Applied Sciences*, vol. 7, no. 2, pp. 1001–1011, 2025.
- [45] H. J. Hermens, B. Freriks, C. Disselhorst-Klug, and G. Rau, "Development of recommendations for SEMG sensors and sensor placement procedures," *Journal of Electromyography and Kinesiology*, vol. 10, no. 5, pp. 361–374, 2000.
- [46] J. Brooke, "SUS: a retrospective," *Journal of Usability Studies*, vol. 8, no. 2, pp. 29–40, 2013.

Anexos A

Definiciones, Acrónimos y Abreviaturas

Funciona como un glosario para que cualquier lector pueda entender la terminología específica, los acrónimos (ej. UML, API) y las abreviaturas usadas en el informe.

A.1. Definiciones

Actuador: *Componente del sistema (servomotor) que ejecuta el movimiento físico del brazo robótico en respuesta a la señal de control.*

Acondicionamiento: *Proceso electrónico de preparación de la señal EMG que incluye la **amplificación** y el **filtrado analógico** para eliminar ruido y adaptar la señal al rango del ADC.*

Crosstalk: *Fenómeno donde el sistema de adquisición EMG registra la actividad eléctrica de **músculos adyacentes** al músculo objetivo, reduciendo la especificidad de la señal.*

Filtrado Digital: *Aplicación de algoritmos de procesamiento de señal en el dominio digital (post-conversión ADC) para eliminar componentes de ruido residual (ej. interferencia de la línea de energía).*

Frecuencia de Muestreo: *Tasa a la que el conversor ADC digitaliza la señal analógica por unidad de tiempo. Es crítica para evitar la distorsión (aliasing).*

Latencia: *Retardo temporal total del sistema, medido desde el inicio de la contracción muscular hasta el inicio del movimiento robótico correspondiente.*

Motoneurona: *Célula nerviosa que transmite las señales desde el sistema nervioso central hasta las fibras musculares, iniciando la contracción que genera la señal EMG.*

Normalización (EMG): *Proceso de escalar la amplitud de la señal EMG respecto a un valor de referencia, comúnmente la **MVC**, para asegurar que las comparaciones entre diferentes sujetos o sesiones sean consistentes.*

Selectividad Espacial: *Capacidad de la configuración de electrodos EMG (monopolar o bipolar) para aislar y registrar la actividad de un **músculo específico** sin interferencia significativa de músculos vecinos.*

A.2. Acrónimos

ADC: *Conversor Analógico-Digital (Analog-to-Digital Converter).*

GDL: *Grados de Libertad .*

EMG: *Electromiografía.*

FSM: *Máquina de Estados Finita (Finite State Machine).*

MVC: *Máxima Contracción Voluntaria (Maximum Voluntary Contraction).*

SNR: *Relación Señal/Ruido (Signal-to-Noise Ratio).*

SUS: *Escala de Usabilidad del Sistema (System Usability Scale).*

A.3. Abreviaturas

OE: *Objetivo Específico.*

PWM: *Modulación por Ancho de Pulso (Pulse Width Modulation).*

RMS: *Valor Cuadrático Medio (Root Mean Square).*

SENIAM: *Surface EMG for Non-Invasive Assessment of Muscles (Protocolo Estándar).*

Anexos B

Configuraciones

Anexos C

Firmware Completo del Sistema de Adquisición y Actuación

```
#include <avr/interrupt.h>
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>
#include <io90pwm1.h>
#include <HardwareSerial.h>
#include <USBAPI.h>

// -----
// PCA9685
// -----
Adafruit_PWMServoDriver pca = Adafruit_PWMServoDriver(0x40);

#define CANAL_SERVO_CODO    0
#define CANAL_SERVO_MUNECA  1

#define PULSO_MIN_US      500
#define PULSO_MAX_US      2400
#define FRECUENCIA_PWM    50

#define US_POR_CUENTA      (20000.0f / 4096.0f)
#define PULSO_MIN_CNT      ((uint16_t)(PULSO_MIN_US / US_POR_CUENTA))
#define PULSO_MAX_CNT      ((uint16_t)(PULSO_MAX_US / US_POR_CUENTA))

// -----
// PINES EMG - 3 canales
// -----
#define PIN_BICEPS          A0
```

```

#define PIN_TRICEPS      A1
#define PIN_ANTEBRAZO  A2    // pronator teres

#define N_CANALES 3

// -----
// PARÁMETROS DE MUESTREO
// -----
#define FS_TOTAL_HZ      1000    // Hz, tasa total del ADC (los 3 canales
// se leen en sucesión dentro de cada ciclo)
#define PASO_MS          20      // cadencia de recepción de comandos de ángulo

// -----
// LIMITADOR DE TASA (slew-rate) - única protección mecánica local
// -----
// KS-3518: 300°/s sin carga - margen 80%
#define VEL_MAX_SERVO     300.0f
#define DT_CONTROL        (PASO_MS / 1000.0f)
#define MAX_CAMBIO        (VEL_MAX_SERVO * DT_CONTROL * 0.8f)    // 4.8°/ciclo

// -----
// ESTADO DE MUESTREO - ISR alterna entre los 3 canales
// -----
volatile int16_t muestra_biceps      = 0;
volatile int16_t muestra_triceps     = 0;
volatile int16_t muestra_antebrazo   = 0;
volatile bool    trama_lista        = false;    // true cuando los 3 canales
// de un mismo ciclo están listos
volatile uint8_t canal_actual        = 0;        // 0=biceps,1=triceps,2=antebrazo

// -----
// ESTADO DE CONTROL - ángulos objetivo y actuales, 2 GDL
// -----
// Reposo = 0° en ambos GDL (convención fijada para todo el proyecto).
float angulo_codo_actual    = 0.0f;
float angulo_codo_meta      = 0.0f;
float angulo_muneca_actual  = 0.0f;
float angulo_muneca_meta    = 0.0f;

// -----
// TIMER1 - dispara a FS_TOTAL_HZ, cada disparo lee un canal y alterna
// Con 3 canales, cada canal individual se actualiza a FS_TOTAL_HZ/3 = 333 Hz,

```

```
// consistente con el Nyquist efectivo usado en el pipeline Python.
// -----
void configurarTimer1() {
    cli();
    TCCR1A = 0; TCCR1B = 0; TCNT1 = 0;
    OCR1A = 1999;           // 1 ms → 1000 Hz total
    TCCR1B |= (1 << WGM12);
    TCCR1B |= (1 << CS11);   // prescaler 8
    TIMSK1 |= (1 << OCIE1A);
    sei();
}

ISR(TIMER1_COMPA_vect) {
    switch (canal_actual) {
        case 0:
            muestra_biceps = analogRead(PIN_BICEPS);
            canal_actual = 1;
            break;
        case 1:
            muestra_triceps = analogRead(PIN_TRICEPS);
            canal_actual = 2;
            break;
        default:
            muestra_antebrazo = analogRead(PIN_ANTEBRAZO);
            canal_actual = 0;
            trama_lista = true; // ciclo de 3 canales completo
            break;
    }
}

// -----
// SERVOS
// -----
void moverServo(uint8_t canal, float angulo) {
    angulo = constrain(angulo, 0.0f, 180.0f);
    uint16_t cuentas = (uint16_t)map(
        (long)(angulo * 10), 0, 1800, PULSO_MIN_CNT, PULSO_MAX_CNT
    );
    pca.setPWM(canal, 0, cuentas);
}

// Aplica el limitador de tasa a un ángulo actual hacia su meta, y mueve
```

```
// el servo correspondiente. Se llama una vez por GDL por ciclo de control.
float actualizarAngulo(uint8_t canal_pca, float actual, float meta) {
    float dif = meta - actual;
    if (abs(dif) > MAX_CAMBIO)
        actual += (dif > 0 ? 1.0f : -1.0f) * MAX_CAMBIO;
    else
        actual = meta;
    actual = constrain(actual, 0.0f, 180.0f);
    moverServo(canal_pca, actual);
    return actual;
}
```

```
// -----
// PARSEAR DE COMANDOS SERIALES DESDE PC
// Protocolo: "A,<angulo_codo>,<angulo_muneca>\n"
// -----
char rx_buf[24];
uint8_t rx_idx = 0;

void procesarLinea(char *linea) {
    if (linea[0] != 'A' || linea[1] != ',') return;

    // linea = "A,<codo>,<muneca>" - separar por la coma tras "A,"
    char *resto = linea + 2;
    char *coma = strchr(resto, ',');
    if (coma == NULL) return; // trama incompleta, se descarta

    *coma = '\0';
    float codo = atof(resto);
    float muneca = atof(coma + 1);

    angulo_codo_meta = constrain(codo, 0.0f, 180.0f);
    angulo_muneca_meta = constrain(muneca, 0.0f, 180.0f);
}
```

```
void leerSerial() {
    while (Serial.available()) {
        char c = Serial.read();
        if (c == '\n') {
            rx_buf[rx_idx] = '\0';
            rx_idx = 0;
            procesarLinea(rx_buf);
        }
    }
}
```

```
} else if (rx_idx < sizeof(rx_buf) - 1) {
    rx_buf[rx_idx++] = c;
}
// Si se excede el buffer sin '\n', la trama se descarta implícitamente
// al sobrescribirse rx_idx en el próximo '\n' válido.
}
}

// -----
// TRANSMISIÓN DE MUESTRAS CRUDAS
// Protocolo: "S,<adc_biceps>,<adc_triceps>,<adc_antebrazo>\n"
// -----
void transmitirTrama() {
    Serial.print(F("S,"));
    Serial.print(muestra_biceps);    Serial.print(F(","));
    Serial.print(muestra_triceps);   Serial.print(F(","));
    Serial.println(muestra_antebrazo);
}

// -----
// SETUP
// -----
void setup() {
    Serial.begin(115200);
    Wire.begin();
    pca.begin();
    pca.setOscillatorFrequency(25000000);
    pca.setPWMFreq(FRECUENCIA_PWM);
    delay(10);

    // Posición segura inicial: reposo = 0° en ambos GDL
    moverServo(CANAL_SERVO_CODO, 0.0f);
    moverServo(CANAL_SERVO_MUNECA, 0.0f);

    pinMode(PIN_BICEPS,    INPUT);
    pinMode(PIN_TRICEPS,   INPUT);
    pinMode(PIN_ANTEBRAZO, INPUT);

    configurarTimer1();

    Serial.println(F("READY"));
    Serial.println(F("# EMG - Puente de adquisición puro, 3 canales"));
}
```

```
Serial.println(F("# Canales: A0=Biceps A1=Triceps A2=Antebrazo(pronator teres)"));
Serial.println(F("# Fs=1000Hz total (~333Hz/canal) | Sin DSP ni calibracion embebida"));
Serial.println(F("# Protocolo TX: S,adc_biceps,adc_triceps,adc_antebrazo"));
Serial.println(F("# Protocolo RX: A,angulo_codo,angulo_muneca"));
Serial.println(F("# Reposo = 0 grados en ambos GDL"));
Serial.println(F("#"));
}

// -----
// LOOP PRINCIPAL
// -----
void loop() {
  leerSerial();

  if (!trama_lista) return;
  trama_lista = false;

  // 1. Transmitir muestras crudas del ciclo actual
  transmitirTrama();

  // 2. Actualizar ambos servos con limitador de tasa, hacia el último
  //     ángulo objetivo recibido de la PC (angulo*_meta se actualiza de
  //     forma asíncrona en procesarLinea() cada vez que llega un comando)
  static uint32_t ultimo_control = 0;
  uint32_t ahora = millis();
  if (ahora - ultimo_control >= PASO_MS) {
    ultimo_control = ahora;
    angulo_codo_actual = actualizarAngulo(
      CANAL_SERVO_CODO, angulo_codo_actual, angulo_codo_meta);
    angulo_muneca_actual = actualizarAngulo(
      CANAL_SERVO_MUNECA, angulo_muneca_actual, angulo_muneca_meta);
  }
}
```